



Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera



A Graduation Project Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mohamed Ashraf
Mazen Adel

Nesma Osama
Mohamed Khater

Supervised by

Dr. Salma Abdel Monem

April 17, 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the authors or department.

Abstract

Accelerera is a hybrid Python/C++ machine learning framework that combines a high-level Python interface with native execution components. The system is organized around six modules: Accelerera Pipe, the Code Parallelizer, Auto Preprocessing, AutoML, Deployment, and the Benchmark Platform. Accelerera Pipe represents machine-learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, and emits OpenMP-annotated C++ code. This report documents the design, methodology, and experimental results available for the implemented modules, while retaining the required report structure for the remaining modules.

Arabic Abstract

مشروع أكسيليرا هو إطار عمل هجين للتعلم الآلي مبني باستخدام بايثون وسي بلس بلس، ويجمع بين واجهة بايثون عالية المستوى ومكونات تنفيذ أصلية. يتكون النظام من ست وحدات، وهي أكسيليرا بايب، ووحدة موازنة الشفرة، والمعالجة المسبقة التلقائية، والتعلم الآلي التلقائي، والنشر، ومنصة القياس المرجعي. تمثل أكسيليرا بايب سير عمل التعلم الآلي في صورة عقد ضمن رسم بياني موجه للمعالجة المسبقة، وبناء النماذج، والتنبؤ، وتقييم المقاييس، والتفرع، والدمج. تحلل وحدة موازنة الشفرة شفرات سي وسي بلس بلس وشفرات بايثون المدعومة، وتستخرج خصائص الحلقات، وتتنبأ بفرص التنفيذ المتوازي، وتنتج شفرة سي بلس بلس مزودة بتوجيهات أوبن إم بي. يوثق هذا التقرير التصميم والمنهجية والنتائج التجريبية المتاحة للوحدات المنفذة، مع الحفاظ على هيكل التقرير المطلوب لبقية الوحدات.

Acknowledgment

Contents

Abstract	2
Arabic Abstract	3
Acknowledgment	4
List of Abbreviations	14
List of Symbols	15
1 Introduction	17
1.1 Motivation and Justification	18
1.2 Project Objectives and Problem Definition	20
1.3 Project Outcomes	22
1.4 Document Organization	23
2 Market Visibility Study	25
2.1 Targeted Customers	25
2.2 Market Survey	27
2.2.1 AutoCleanML	27
2.2.2 scikit-learn	28
2.2.3 AutoML Tools	29
2.2.4 Code Acceleration Tools	29
2.2.5 Deployment and Hosting Platforms	30
2.3 Business Case and Financial Analysis	30
2.3.1 Business Case	31
2.3.2 Financial Analysis	32
3 Literature Survey	34
3.1 Background and Previous Work	34
3.1.1 Data Science and Machine Learning	34
3.1.1.1 Data Exploration and Preprocessing [1]	34
3.1.1.2 Model Evaluation Metrics [2]	35
3.1.2 Natural Language Processing (NLP) [3]	37
3.1.2.1 Term Frequency-Inverse Document Frequency (TF-IDF)	37

3.1.3	Data Augmentation	38
3.1.4	Machine Learning Pipelines	38
3.1.5	Graph-Based Pipeline Execution	38
3.1.6	Code Parallelization	38
3.1.7	OpenMP	39
3.1.8	Deployment	39
3.1.9	Model Deployment	39
3.1.10	Machine Learning Deployment	39
3.1.11	Cloud and Server Deployment	39
3.1.12	Web Services, APIs, and REST APIs	40
3.1.13	HTTP, HTTPS, JSON, URLs, and Ports	40
3.1.14	Docker and Containerization	40
3.1.15	Virtual Machines, AWS, and AWS EC2	40
3.1.16	Heroku and Container Hosting	40
3.1.17	SSH and CLI Tools	40
3.1.18	Environment Variables and Configuration Files	41
3.1.19	Pickle Files and Model Artifacts	41
3.1.20	Version Control, Git, GitHub, and Repositories	41
3.1.21	Backend Services, FastAPI, and Uvicorn	41
3.1.22	Deployment Scripts	41
3.2	Comparative Study of Previous Work	42
3.2.1	Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [4]	42
3.2.2	scikit-learn Pipeline	42
3.2.3	OMPar: Automatic Parallelization with AI-Driven Source-to- Source Compilation	42
3.2.4	Automated Machine Learning	43
3.2.5	Search Strategies for AutoML	43
3.2.6	Multi-Fidelity Evaluation and Warm Starts	43
3.2.7	Ensemble Learning in AutoML	44
3.2.8	Model Deployment Platforms	44
3.3	Implemented Approach	45
3.3.1	Auto Preprocessing	45
3.3.2	accelera_pipe	46
3.3.3	Parallelizer	47
3.3.4	Deployment Module Technologies	47
4	System Design and Architecture	49
4.1	Overview and Assumptions	49
4.1.1	Accelera Pipeline Assumptions	49
4.1.2	Auto Preprocessing Assumptions	49
4.1.3	Benchmark Assumptions	49

4.1.4	Deployment Assumptions	49
4.2	System Architecture	51
4.2.1	Block Diagram	51
4.3	Accelerera Pipe Module	59
4.3.1	Functional Description	59
4.3.2	Modular Decomposition	60
4.3.3	Branching and Path Selection	62
4.3.4	Intermediate Memory Management	63
4.3.5	Parallel Graph Execution	64
4.3.6	Design Constraints	65
4.3.7	Summary	66
4.4	Code Parallelizer Module	67
4.4.1	Functional Description	67
4.4.2	Modular Decomposition	69
4.4.3	Design Constraints	71
4.4.4	Summary	72
4.5	Auto Preprocessing Module	73
4.5.1	Functional Description	74
4.5.1.1	Tabular datasets	75
4.5.1.2	Text datasets	75
4.5.1.3	Image datasets	75
4.5.2	Modular Decomposition	75
4.5.2.1	Tabular Data Preprocessing Functions	75
4.5.2.2	Text Data Preprocessing Functions	80
4.5.2.3	Image Data Preprocessing	82
4.5.2.4	Inference and Reusability:	86
4.5.3	Design Constraints	86
4.5.4	Module Summary	86
4.6	AutoML Module	86
4.6.1	Module Responsibility	86
4.6.2	Functional Description	87
4.6.3	Configuration Space Construction	87
4.6.4	Three-Stage Fidelity Schedule	87
4.6.5	Surrogate-Guided Candidate Selection	88
4.6.6	Meta-Learning Warm Starts	88
4.6.7	Parallel Execution and Timeout Control	88
4.6.8	Final Model and Ensemble Selection	88
4.6.9	Modular Decomposition	89
4.6.10	Design Constraints	90
4.6.11	User-Facing Example	90
4.7	Deployment Module	90
4.7.1	Module Responsibility	90

4.7.2	Service Architecture	92
4.7.3	Configuration and Artifact Preparation	93
4.7.4	Runtime Validation, Inference, and Tracking	94
4.7.5	Model Version Control	95
4.7.6	Deployment Orchestration	97
4.7.7	Command-Line Interface	99
4.7.8	Technical Value of the Module	101
4.8	Benchmark Platform Module	101
4.8.1	Functional Description	101
4.8.2	Modular Decomposition	102
4.8.3	Design Constraints	103
4.8.4	Module Summary	104
5	System Testing and Verification	105
5.1	Testing Setup	105
5.2	Testing Plan and Strategy	105
5.2.1	Module Testing	105
5.3	Accelera Pipe Module	105
5.4	Code Parallelizer Module	107
5.4.0.1	Auto Preprocessing Module	110
5.4.0.2	AutoML Module	113
5.4.0.3	Deployment Module	115
5.4.1	Experimental Setup and Test Suite	115
5.4.2	Verification Objectives	116
5.4.3	Subsystem Verification Details	117
5.4.4	Command-Flow Verification	118
5.4.5	Deployment Test Results	119
5.4.5.1	Accelera Pipeline Reporting Module	120
5.4.5.2	Benchmark Platform Module	120
5.4.6	Integration Testing	121
5.4.7	Testing Schedule	121
5.4.8	Comparative Results to Previous Work	121
5.4.8.1	Auto Preprocessing	121
6	Conclusions and Future Work	124
6.1	Faced Challenges	124
6.2	Gained Experience	124
6.3	Conclusions	124
6.4	Future Work	125
	Reference	125
A	Development Platforms and Tools	128

A.1	Hardware Platform	128
A.2	Software Platform	128
B	Use Cases	129
B.1	Auto Preprocessing Use Cases	129
B.2	Accelera Pipe Use Cases	129
B.3	Code Parallelizer Use Cases	130
B.4	Pipeline Reporting Use Cases	131
B.5	Benchmark Platform Use Cases	131
C	User Guide	132
C.1	Using Accelera Pipe	132
C.1.1	Constructing and selecting a graph	132
C.1.2	Saving training graphs and executed graphs	133
C.2	Using the Code Parallelizer	133
C.2.1	Parallelizing files and source strings	133
C.2.2	Parallelizing a Python callable	134
C.2.3	Using automatic pipeline integration	135
C.3	Using Tabular Auto Preprocessing	135
C.4	Using Text Auto Preprocessing	136
C.5	Using Image Auto Preprocessing	137
C.6	Reading Auto Preprocessing Reports	138
C.7	Using Pipeline Reports	138
C.8	Using the Benchmark Platform	139
D	Code Documentation	140
E	Feasibility Study	141

List of Figures

4.1	Overall block diagram of the Accelera project architecture.	51
4.2	Accelera Pipe workflow with custom preprocessing parallelization and pipeline reuse.	60
4.3	Before duplicate first-node sharing	63
4.4	After duplicate first-node sharing	63
4.5	Code Parallelizer module workflow	70
4.6	Integration flow for compiled preprocessing inside Accelera Pipe	72
4.7	Auto Preprocessing Workflow	74
4.8	Tabular Preprocessing Workflow	80
4.9	Text Preprocessing Workflow	82
4.10	Image Preprocessing Workflow	85
4.11	Deployment module overview from workspace artifacts to serving targets.	91
4.12	Model serving architecture and execution flow.	93
4.13	Deployment VCS registry structure.	96
4.14	VCS commit workflow for deployment artifacts.	96
4.15	VCS deploy and rollback workflow.	97
4.16	Multi-target deployment orchestration flow.	98
5.1	Mean classification accuracy and runtime for the evaluated AutoML systems	114
5.2	Mean regression R^2 and runtime for the evaluated AutoML systems . .	115

List of Tables

1.1	Main Accelera modules and their roles.	18
1.2	Problems addressed by Accelera.	21
1.3	Summary of project outcomes.	24
2.1	Targeted customers for Accelera.	26
2.2	Market comparison summary.	27
2.3	Possible business value of Accelera.	31
2.4	Expected cost categories.	32
2.5	Simplified business plan stages.	33
3.1	Selected AutoML systems and their design emphasis.	44
3.2	Deployment-related platforms and Accelera's difference.	45
4.1	Possible paths to the deployment selection switch.	52
4.2	Principal node families in an Accelera Pipe graph.	53
4.3	Parallelizer input dispatch and output behavior.	54
4.4	Parallelizer stages.	56
4.5	Standalone use versus end-to-end use.	58
4.6	Main components of the Accelera Pipe module	61
4.7	Mapping from builder calls to graph node semantics	62
4.8	Supported operations in the Python-to-C++ converter	68
4.9	Main components of the Code Parallelizer module	69
4.10	Detected Column Types in Tabular Data	77
4.11	EDA graphs generated for tabular preprocessing	77
4.12	AutoML implementation units	89
4.13	Deployment module implementation units.	92
4.14	Deployment orchestrator commands.	100
4.15	Deployment VCS commands.	100
5.1	Accelera Pipe latency scaling in seconds	106
5.2	Accelera Pipe RSS memory scaling in MB	106
5.3	Selected candidate and accuracy across sample sizes	107
5.4	Largest Accelera Pipe comparison	107
5.5	OpenMP classifier report	108
5.6	Hard parallelizer evaluation	108
5.7	Linux parallelizer and Accelera Pipe integration benchmark	108

5.8	Linux direct example-script verification runs	109
5.9	Windows direct example correctness and path timing	109
5.10	Windows compilation and process overhead	110
5.11	Datasets Used in Module Testing	112
5.12	Models and Evaluation Metrics Used in Module Testing	113
5.13	AutoML classification benchmark summary	114
5.14	AutoML regression benchmark summary over ten datasets	115
5.15	Deployment module verification objectives.	117
5.16	Verified deployment command flows.	119
5.17	Deployment Module Test Suite Verification Summary	120
5.18	Classification comparison using Logistic Regression and Random Forest	122
5.19	Classification comparison using Decision Tree and KNN	122
5.20	Regression comparison using Linear Regression and Random Forest .	122
5.21	Regression comparison using Decision Tree and KNN	123
B.1	Representative Accelera Pipe use cases.	129
B.2	Representative Code Parallelizer use cases.	130

List of Algorithms

4.1	High-level Accelera Pipe training flow	62
4.2	Consumer-count based intermediate release	64
4.3	Parallel graph execution scheduling	65
4.4	Loop classification and pragma insertion	70
4.5	General tabular preprocessing flow	79
4.6	Text tokenizer flow	81
4.7	Image training preprocessing flow	84
4.8	Using Accelera AutoML for classification	90
4.9	Preparing deployment artifacts	94
4.10	Validating a request, running inference, and tracking the result	94
4.11	Creating a deployment VCS commit	96
4.12	Building and running the local deployment container	98
4.13	Deploying the model service to AWS EC2	98
4.14	Benchmark creation flow	103
4.15	Submission scoring flow	103
5.1	Deployment test-suite execution command	116
C.1	Building and selecting an Accelera Pipe graph	132
C.2	Applying a transformer to selected columns	133
C.3	Saving and loading Accelera Pipe graphs	133
C.4	Parallelizing a source file and a code string	134
C.5	Compiling a custom numerical function	134
C.6	Adding an automatically optimized custom function	135
C.7	Calling tabular training preprocessing	135
C.8	Changing tabular preprocessing to regression	136
C.9	Calling tabular testing preprocessing	136
C.10	Calling text training preprocessing	136
C.11	Calling text testing preprocessing	137
C.12	Calling image training preprocessing	137
C.13	Calling image testing preprocessing	138
C.14	Calling the pipeline graph report	138

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
AWS	Amazon Web Services
CI/CD	Continuous Integration and Continuous Deployment
CLI	Command-Line Interface
CPU	Central Processing Unit
Docker	Containerization Platform
EC2	Elastic Compute Cloud
Git	Distributed Version Control System
GUI	Graphical User Interface
Heroku	Cloud Application Platform
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
OpenMP	Open Multi-Processing
REST	Representational State Transfer
RSS	Resident Set Size
SSH	Secure Shell
SVC	Support Vector Classifier
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
VCS	Version Control System
VM	Virtual Machine
VMS	Virtual Memory Size

List of Symbols

X	Input feature matrix
y	Target vector
P	Source program
L	Extracted loop
$F(L)$	Feature representation of loop L

Contacts

Team Members

Name	Email	Phone Number
Mohamed Ashraf	mohamed.abdelaziz03@eng-st.cu.edu.eg	+2 01004538215
Nesma Osama	nesma.ahmed03@eng-st.cu.edu.eg	+2 01067193062
Mazen Adel	mez3n1151@gmail.com	+2 01152794877
Mohamed Khater	abc1@email.com	+2 01xxxxxxx

Supervisor

Name	Email	Number
Salma Abdelmoniem	<u>abc1@email.com</u>	+2 01114302362

Chapter 1: Introduction

Accelerera is a hybrid Python/C++ machine-learning framework designed to support the complete data-science workflow, starting from data preparation and model selection, and extending to graph-based execution, native code acceleration, and benchmark comparison. The project is not a single isolated algorithm. Instead, it is a collection of connected modules that cooperate to make machine-learning experiments easier to build, easier to repeat, and faster to execute when possible.

The main idea behind Accelerera is simple: Python gives developers a productive environment for experimentation, but real machine-learning workflows often need more structure and better execution control. A typical workflow may include data cleaning, preprocessing, feature engineering, model selection, prediction, metric evaluation, reporting, and comparison between several alternatives. When these steps are written as separate scripts, repeated work, unclear execution order, and performance bottlenecks become common problems.

Accelerera addresses this by combining six main units: `auto_preprocessing`, `accelera_automl`, `accelera_pipe`, `parallelizer`, the benchmark platform, and the Deployment module. Table 1.1 gives a quick overview of these modules and how they contribute to the full system.

Table 1.1: Main Accelera modules and their roles.

Module	Main role	Why it matters
<code>auto_preprocessing</code>	Prepares tabular, text, and image data using automated pre-processing steps.	Reduces repeated manual data-cleaning work and produces reusable preprocessing artifacts and reports.
<code>accelera_automl</code>	Searches for suitable classification and regression models under time and trial budgets.	Helps users compare model candidates systematically instead of manually testing each model.
<code>accelera_pipe</code>	Builds graph-based machine-learning workflows with preprocessing, model, prediction, metric, merge, and branch nodes.	Exposes workflow structure, supports branch execution, and allows fitted paths to be reused through an executed graph.
<code>parallelizer</code>	Analyzes supported Python/C++ loop-heavy code, inserts safe OpenMP pragmas, and compiles native callable extensions.	Targets custom preprocessing bottlenecks that would otherwise remain slow in pure Python.
Benchmark platform	Provides a prototype web platform for datasets, metrics, submissions, and leaderboard-style comparison.	Connects the framework to a future environment for fair pipeline evaluation and community comparison.
Deployment	Transforms trained models and generated artifacts into deployable services using configuration files, model packaging, runtime validation, and deployment commands for local and cloud/server targets.	Bridges the gap between model development and real-world usage by making trained outputs accessible through service endpoints rather than leaving them only as local files.

1.1 Motivation and Justification

Machine-learning development is often presented as a simple process of choosing a model and calling `fit`. In practice, the model is only one stage in a larger workflow. Before training, the data must be cleaned, split, transformed, encoded, scaled, and sometimes reduced. After training, the output must be evaluated, compared, reported, and possibly deployed or submitted to a benchmark system. This creates several prac-

tical problems.

- **Repeated preprocessing code:** Similar cleaning, encoding, scaling, and feature-selection steps are often rewritten for every dataset.
- **Manual model comparison:** Users may need to try many models and configurations before finding a good solution.
- **Linear workflow limitations:** Traditional scripts or simple pipelines may duplicate work when several branches share early operations.
- **Python loop bottlenecks:** Custom preprocessing functions written in Python may become slow when they contain large numeric loops.
- **Weak experiment visibility:** Without reports, graphs, and saved artifacts, it becomes harder to understand what preprocessing and evaluation steps were applied.
- **Lack of fair comparison:** Pipeline results need a structured place where metrics, submissions, and benchmarks can be compared.
- **Deployment gap:** A trained model is not useful to external users until it is packaged, served through an API, configured for its runtime environment, and exposed on a local or remote host.

These problems justify the need for Accelerera. The project provides automation where the workflow is repetitive, graph execution where the workflow contains branches, native acceleration where supported custom code is slow, benchmark components where results must be compared, and deployment support where trained models must become usable services. The motivation is not only speed; it is also organization, reproducibility, usability, and practical delivery.

The `auto_preprocessing` module is motivated by the fact that data preparation usually consumes a large part of the development effort. It handles common preprocessing decisions for tabular datasets, such as dropping duplicate records, detecting column types, imputing missing values, encoding categorical features, applying robust scaling, selecting features, saving preprocessors, and generating reports. It also supports text preprocessing using cleaning, tokenization, stemming, TF-IDF vectorization, and target encoding. For image workflows, it provides preparation paths for classification tasks. This makes the project useful before the model-selection stage even begins.

The `accelera_automl` module is motivated by the difficulty of manual model selection. A user may not know which estimator, preprocessing choice, or hyperparameter setting will perform best. AutoML reduces this burden by evaluating candidate configurations under controlled budgets. This makes the search process more systematic and provides useful outputs such as the best model, score, leaderboard, and run history.

The `accelera_pipe` module is motivated by the need to represent machine-learning workflows as graphs rather than only as linear chains. In branch-heavy experiments, several alternatives may share the same preprocessing steps. A graph representation makes these dependencies explicit. It also allows the system to reason about execution order, branch selection, reuse of fitted paths, and intermediate-data lifetime. This gives the user a high-level Python interface while the backend manages a more structured execution model.

The `parallelizer` module is motivated by the performance gap between Python and native compiled code. Python is easy to write, but loop-heavy Python functions can be slow. The parallelizer attempts to optimize supported custom functions by converting them to C++, extracting loops, computing loop features, classifying safe parallelization opportunities, applying rule-based safety checks, inserting OpenMP pragmas, and compiling the result as a native Python-callable extension. If the optimization path fails, the original function is preserved, which keeps the workflow correct.

The benchmark platform is motivated by the need for organized comparison. A machine-learning framework becomes more useful when users can compare pipelines on common datasets and metrics. The benchmark module therefore represents a future-facing part of the project, where submissions, metrics, and leaderboard-style results can help users evaluate their solutions fairly.

Finally, the Deployment module is motivated by the final step that many machine-learning projects postpone: making the trained model available outside the notebook or local script. Deployment normally requires packaging artifacts, building an API layer, preparing dependencies, exposing ports, configuring servers, and checking that the service responds correctly. Accelera includes this step so that the workflow can continue from model creation to a reachable prediction service.

1.2 Project Objectives and Problem Definition

The main objective of Accelera is to provide an integrated framework for constructing, preprocessing, executing, evaluating, optimizing, and comparing machine-learning workflows. The project aims to keep the familiar Python programming style while adding automation, graph execution, native acceleration, and benchmark-oriented evaluation.

The main objectives of the project are:

1. Build an automated preprocessing module for tabular, text, and image datasets.
2. Support data cleaning, missing-value handling, categorical encoding, robust scaling, TF-IDF text representation, image preparation, feature selection, and report generation.
3. Implement AutoML classifiers and regressors that search across supported model configurations using scoring metrics, cross-validation, time budgets, trial budgets, and optional ensemble construction.

Table 1.2: Problems addressed by Accelera.

Problem	Accelera component	Expected benefit
Manual and repetitive data preparation	auto_preprocessing	Automates common preprocessing and produces saved artifacts and reports.
Time-consuming model selection	accelera_automl	Searches candidate models and configurations under controlled budgets.
Repeated work in branch-heavy workflows	accelera_pipe	Represents the workflow as a graph and exposes shared execution structure.
Slow custom Python loops	parallelizer	Converts supported functions to native code and inserts safe OpenMP pragmas.
Difficult result comparison	Benchmark platform	Stores benchmark information, metrics, submissions, and leaderboard views.
Local-only trained models	Deployment module	Packages saved models into validated API services that can run in Docker, Heroku, or AWS EC2 workflows.

4. Provide a graph-based pipeline module that represents workflow steps as executable graph nodes rather than only as a sequential list.
5. Support pipeline operations such as preprocessing, model fitting, prediction, metric evaluation, branching, merging, and executed-graph reuse.
6. Integrate custom preprocessing functions with the parallelizer so that supported loop-heavy functions can be optimized automatically.
7. Implement a conservative code parallelization path that uses C++, OpenMP, loop-feature extraction, classification, rule-based safety checks, and native compilation.
8. Provide reporting tools that make preprocessing outputs, model metrics, and pipeline structure more visible to the user.
9. Build a benchmark platform prototype for datasets, submissions, users, metrics, and leaderboard-based comparison.
10. Provide a Deployment module that packages trained models and generated artifacts into usable services, supports local and remote deployment workflows,

manages deployment configuration, and reduces the infrastructure knowledge required to expose a trained model through an API.

The problem addressed by the project can be stated as follows:

Machine-learning workflows in Python are easy to write but become difficult to organize, repeat, compare, and optimize when they include many preprocessing steps, model alternatives, custom functions, evaluation paths, and deployment requirements. Accelerera aims to preserve Python usability while adding automated preprocessing, budgeted model search, graph-based execution, safe loop-level acceleration, and benchmark-oriented evaluation, while also supporting the final step of turning trained artifacts into accessible services.

This problem has two sides. The first side is **workflow complexity**. As the number of preprocessing steps, branches, and candidate models increases, the workflow becomes harder to manage manually. The second side is **execution performance**. Even if the workflow is correct, it may repeat shared work or spend too much time in slow Python loops. Accelerera addresses both sides by combining high-level automation with lower-level execution support.

1.3 Project Outcomes

The project outcomes are the implemented modules and the integration between them. Each module provides a different part of the final system.

Outcome 1: Automated preprocessing

The `auto_preprocessing` module prepares datasets for machine-learning models. For tabular data, it handles duplicates, missing values, column-type detection, categorical encoding, robust scaling, target processing, feature selection, and report generation. For text data, it applies cleaning, tokenization, stemming, TF-IDF vectorization, and label encoding. For image data, it supports classification preparation. The outcome is a set of processed arrays, saved preprocessors, selected features, and reports.

Outcome 2: AutoML model search

The `accelera_automl` module provides estimator-style AutoML classes for classification and regression. It searches candidate configurations, evaluates them with cross-validation, supports warm-starting from dataset metafeatures, and can build voting or stacked ensembles. The outcome is a fitted final model, best score, leaderboard, and run history.

Outcome 3: Graph-based pipeline execution

The `accelera_pipe` module provides a graph-based pipeline interface. It represents preprocessing, model, prediction, metric, branch, and merge operations as graph nodes. The output of a pipeline run includes predictions and an `ExecutedGraph`, which stores the selected fitted path for later use. This outcome gives the project its workflow-execution layer.

Outcome 4: Source-level parallelization

The `parallelizer` module accelerates supported loop-heavy code. It can process C/C++ code, supported Python code, custom Python functions, and transformer methods. It extracts loops, computes features, classifies parallelization opportunities, applies safety rules, inserts OpenMP pragmas, and compiles native Python-callable extensions when possible. If any step fails, it falls back to the original Python implementation.

Outcome 5: Pipeline and parallelizer integration

One of the most important outcomes is the connection between `accelera_pipe` and the `parallelizer`. When a preprocessing node contains a supported custom Python function, the pipeline attempts to optimize it before adding it to the graph. This allows the system to combine graph-level workflow execution with code-level acceleration.

Outcome 6: Reporting and benchmark support

The project includes reports for preprocessing, model metrics, and graph visualization. It also includes a benchmark platform prototype with backend and frontend components for datasets, metrics, users, submissions, and leaderboards. These outcomes improve visibility and comparison.

Outcome 7: Deployment support

The Deployment module extends Accelera beyond experimentation by converting saved machine-learning artifacts into runnable services. It organizes `config.json` files, model pickle files, serving code, schema validation, telemetry logging, Docker packaging, and target-specific commands for local containers, Heroku, and AWS EC2. This outcome is important because it allows Accelera to support the complete journey from building a model to making the model reachable through an endpoint.

1.4 Document Organization

The rest of this report is organized into seven main parts: Chapter 2 presents the market visibility study, Chapter 3 discusses the literature survey and related work, Chapter 4 explains the methodology and system design of the main Accelera modules, Chap-

Table 1.3: Summary of project outcomes.

Outcome	Description
Automated preprocessing	The project provides reusable preprocessing support for tabular, text, and image data, with generated reports and saved preprocessing artifacts.
Automated model selection	The AutoML module helps search and compare candidate classification and regression models under time and trial constraints.
Graph-based pipeline execution	Accelerera Pipe represents machine-learning workflows as executable graphs, supporting preprocessing, modelling, prediction, metrics, branching, and merging.
Code-level acceleration	The Parallelizer attempts to optimize supported loop-heavy Python/C++ code using static analysis, safety checks, OpenMP pragmas, compilation, and caching.
Benchmarking support	The Benchmark Platform provides a foundation for dataset-based evaluation, prediction submission, metric calculation, and leaderboard-style comparison.
Deployment support	The Deployment module packages trained models and supporting artifacts into validated API services that can run locally in Docker or be deployed to supported cloud/server targets.

ter 5 presents testing, verification, and experimental results, and Chapter 6 concludes the report by summarizing the contributions, limitations, and possible future improvements. The appendices include supporting material such as documentation, usage instructions, feasibility details, and additional implementation notes.

Chapter 2: Market Visibility Study

This chapter studies the market need for Accelera and compares it with existing tools that solve related problems. The goal is to show who can benefit from the project, what alternatives already exist, and why Accelera provides a different value. The chapter also presents a simplified business case and financial analysis to clarify how the project could be developed into a product or service.

Accelera targets the data-science workflow as a complete process, not as one isolated task. Some existing tools focus on preprocessing, others focus on model selection, others focus on competition platforms or code acceleration, and others focus only on hosting or cloud deployment. Accelera combines these directions through six connected parts: `auto_preprocessing`, `accelera_automl`, `accelera_pipe`, `parallelizer`, the benchmark platform, and the Deployment module.

2.1 Targeted Customers

The targeted customers for Accelera are users and organizations that build, test, compare, and optimize machine-learning workflows. These customers can be divided into several groups, as shown in Table 2.1.

The most important customer segment is the data-science and machine-learning user who wants to move from a raw dataset to an evaluated model with less manual work. For beginners, Accelera provides guidance and automation. For experienced users, it provides a more structured execution model and opportunities for acceleration.

Accelera can also be useful in educational environments. In many machine learning courses, students repeat the same steps: clean the data, encode features, scale numerical values, split the dataset, train models, calculate metrics, and compare results. Accelera can make these steps easier to understand because it provides reports and separates the workflow into clear modules.

For professional users, the value is different. They care more about reproducibility, execution time, reusable pipeline structure, and fair comparison. The `accelera_pipe` and benchmark modules are especially useful here because they organize workflow execution and comparison. The `parallelizer` adds value when the workflow includes supported custom preprocessing functions that are slow in Python.

Customer Value Proposition

The main value proposition of Accelera is that it reduces the gap between experimentation and structured execution. Instead of forcing the user to choose between simple scripts and large industrial platforms, Accelera provides a research-friendly framework with practical automation and optimization features.

Table 2.1: Targeted customers for Accelerera.

Customer group	Main need	How Accelerera helps
Students and beginners	Need an easier way to build complete machine-learning workflows without writing every step manually.	Provides automated preprocessing, model search, reports, and a pipeline interface that organizes the workflow.
Data scientists	Need to prepare data, compare models, and repeat experiments quickly.	Reduces repetitive preprocessing, supports AutoML search, and gives reports and leaderboards for comparison.
Machine-learning engineers	Need structured, reusable, and optimized workflows that can be inspected and executed reliably.	Provides graph-based pipeline execution, executed-graph reuse, and integration with custom preprocessing acceleration.
Researchers	Need to compare approaches, measure results, and experiment with workflow and code optimization.	Provides modular components for benchmarking, graph execution, OpenMP parallelization, and performance evaluation.
Small teams and startups	Need a practical toolkit that reduces development time without building an internal ML platform from scratch.	Combines preprocessing, AutoML, pipeline execution, reports, and benchmark support in one project, with deployment support for turning selected models into accessible services.

- **Less manual work:** preprocessing and model search are partially automated.
- **Better organization:** workflows can be represented as graph pipelines instead of scattered scripts.
- **Better visibility:** reports and benchmark views help users inspect results.
- **Possible speedup:** supported loop-heavy preprocessing code can be accelerated through C++ and OpenMP.
- **Fairer comparison:** benchmark components help compare pipelines using common datasets and metrics.
- **Practical delivery:** deployment commands help move saved models from local artifacts to runnable API services.

2.2 Market Survey

Many existing tools provide features related to Accelerera, but most of them focus on only one part of the workflow. Some tools focus on preprocessing, some on AutoML, some on competitions, and some on code acceleration. Accelerera is different because it tries to connect these ideas into one toolkit.

Table 2.2: Market comparison summary.

Tool or platform	Main focus	Limitation	Accelerera difference
<code>scikit-learn</code>	Machine-learning models, preprocessing tools, and linear pipelines.	Pipeline execution is mainly linear and does not focus on graph-based branch execution or custom code parallelization.	Adds graph-based pipeline execution and integration with the Parallelizer.
AutoCleanML	Automatic data cleaning and preprocessing.	Focused mainly on data cleaning and does not cover the whole workflow.	Adds AutoML, graph pipelines, parallelization, reports, and benchmarks.
Auto-sklearn / TPOT / Auto-Gluon	Automated model selection and machine-learning search.	Focus mainly on model search, not graph execution or source-level code parallelization.	Combines AutoML with preprocessing, graph pipelines, and the Parallelizer.
Numba / Cython	Python performance acceleration.	Require user knowledge of acceleration syntax and do not manage ML workflow structure.	Provides an automatic attempt to optimize supported preprocessing functions inside the pipeline.
Heroku	Cloud application hosting and container deployment.	Provides hosting infrastructure but does not manage ML artifacts, schema validation, or model version snapshots as part of the training workflow.	Connects model packaging, serving code, validation, telemetry, and Heroku container commands inside one ML-oriented deployment module.
AWS EC2	General-purpose virtual machines for cloud hosting.	Powerful but requires manual server access, SSH, Docker setup, port exposure, and operational knowledge.	Automates EC2-oriented file synchronization, remote Docker build/run commands, and health checks for Accelerera model services.

2.2.1 AutoCleanML

AutoCleanML is a Python tool focused on automating data cleaning and preprocessing tasks for machine-learning datasets. It is useful because data preparation is one of the most repetitive and time-consuming parts of the machine-learning workflow.

Pros:

- Automates common data-cleaning and preprocessing decisions.
- Helps beginners prepare datasets more easily.
- Provides reporting that explains some preprocessing decisions.

Cons:

- Focuses mainly on preprocessing, not the complete workflow.
- Does not provide a graph-based pipeline execution engine.
- Does not target custom code acceleration or OpenMP parallelization.
- Does not provide a benchmark platform for comparing full pipelines.

Compared with AutoCleanML, Accelera includes preprocessing but does not stop there. It also includes AutoML search, graph pipeline execution, source-level parallelization, reports, and benchmark support. Therefore, Accelera aims to be closer to a complete workflow framework.

2.2.2 `scikit-learn`

`scikit-learn` is one of the strongest tools in the machine-learning market. It provides a wide range of models, preprocessing tools, metrics, and a simple `Pipeline` abstraction. For this reason, it is the most important competitor for `accelera_pipe`.

Pros:

- Mature and widely used.
- Provides many models, preprocessing utilities, and metrics.
- Has a simple and reliable pipeline interface.
- Integrates well with the Python data-science ecosystem.

Cons:

- The standard pipeline is mainly linear.
- Branch-heavy workflows may require repeated pipelines or external search utilities.
- It does not focus on native graph scheduling for pipeline branches.
- It does not automatically convert supported custom preprocessing functions to C++ and OpenMP.

Accelera uses `scikit-learn` as a strong baseline rather than ignoring it. The difference is that `accelera_pipe` represents the workflow as a graph and integrates with the Parallelizer for supported custom preprocessing acceleration.

2.2.3 AutoML Tools

AutoML tools such as Auto-sklearn, TPOT, and AutoGluon focus on automating model selection and hyperparameter search. They are useful because they reduce the manual effort needed to test many models.

Pros:

- Search over many candidate models and configurations.
- Reduce manual model-selection effort.
- Often provide strong baseline performance.
- Some tools support ensembles.

Cons:

- Usually focus on model search more than workflow execution.
- Do not mainly target graph-based pipeline execution.
- Do not automatically accelerate custom preprocessing loops using OpenMP.
- Can be heavy or complex for beginners.

Accelera includes `accelera_automl`, but also connects AutoML with preprocessing, graph execution, reports, and parallelization. This makes the AutoML module one part of a larger framework rather than the only feature.

2.2.4 Code Acceleration Tools

Tools such as Numba, Cython, and manual C++ extensions can speed up Python code. They are relevant because Accelera also tries to accelerate supported custom preprocessing functions.

Pros:

- Can significantly improve performance for numeric code.
- Useful for loop-heavy workloads.
- Provide more control for experienced developers.

Cons:

- Often require users to understand extra syntax, compilation, or type constraints.
- Do not automatically decide where acceleration fits inside an ML pipeline.
- Do not provide pipeline graphs, AutoML, reports, or benchmark features.

The Accelera Parallelizer is different because it is designed to work with the pipeline module. It attempts to optimize supported custom preprocessing functions and attach the optimized callable back to the pipeline node. This makes code acceleration part of the workflow instead of a separate manual task.

2.2.5 Deployment and Hosting Platforms

Deployment platforms such as Heroku, AWS EC2, Render, Railway, and Google Cloud Run are relevant to Accelera because they help developers publish software services. Their main value is infrastructure: they provide servers, routing, containers, or managed execution environments. This is important, but it does not by itself solve the machine-learning-specific work of collecting model artifacts, preserving deployment configuration, validating input schemas, and connecting a trained model to an inference API.

Pros:

- Provide public or cloud-hosted environments for running applications.
- Reduce the need to own physical servers.
- Support common deployment units such as containers, web services, or virtual machines.

Cons:

- Usually require the user to prepare the ML serving code and runtime dependencies separately.
- Do not automatically understand Accelera model artifacts, `config.json`, schema validation, or prediction telemetry.
- General-purpose platforms may be difficult for beginners because they involve CLI tools, SSH, ports, environment variables, and deployment logs.

Accelera's Deployment module is different because it is not only a hosting target. It is part of the ML workflow itself. It receives the outputs produced by training or pipeline modules, copies model binaries, writes deployment requirements, builds a Docker service, exposes prediction endpoints, validates requests, records telemetry, and provides commands for local Docker, Heroku, and AWS EC2. This integration increases Accelera's market value because it makes the system closer to an end-to-end ML platform rather than only a model training toolkit.

2.3 Business Case and Financial Analysis

Accelera can be developed as an open-source framework with optional hosted services around benchmarking, reporting, team collaboration, managed execution, and deployment assistance. The open-source part helps adoption because students, researchers, and developers can use the framework freely. The business value can come from hosted benchmark features, team dashboards, managed experiment tracking, deployment templates, and support services.

2.3.1 Business Case

The main business case is based on reducing time spent in repetitive data-science work. Users and teams spend time preparing data, comparing models, rewriting scripts, debugging pipeline steps, and measuring results. Accelera can reduce this time by providing one framework that handles multiple stages of the workflow. The Deployment module strengthens this business case because organizations do not only need trained models; they need trained models to become usable services. Automating deployment-related steps can reduce manual DevOps effort, reduce time-to-market, and lower the technical barrier for users who understand ML but are less experienced with Docker, SSH, cloud machines, or service exposure.

Table 2.3: Possible business value of Accelera.

Business value	Reason	Possible customer
Faster experimentation	Automated preprocessing and AutoML reduce repeated manual trials.	Students, data scientists, and small teams.
Better organization	Graph pipelines and executed graphs make workflows easier to inspect and reuse.	ML engineers and research teams.
Performance improvement	Parallelizer can speed up supported loop-heavy preprocessing code.	Teams with custom preprocessing bottlenecks.
Better comparison	Benchmark platform supports submissions, metrics, and leaderboards.	Universities, communities, and organizations.
Shorter deployment path	Deployment support helps package model artifacts, build a serving API, and run the service on supported targets.	Small teams, prototypes, and applied ML projects.
Lower entry barrier	Reports and automation make ML workflows easier for beginners.	Education and training programs.

The first possible product direction is a hosted benchmark platform. Users can submit pipelines, compare results on public datasets, and view leaderboards. A free tier can support public datasets and basic submissions, while paid tiers can support private benchmarks, team spaces, larger storage, and advanced reports.

The second possible product direction is a managed workflow dashboard. This dashboard can visualize preprocessing results, model leaderboards, pipeline graphs, and runtime comparisons. Teams can use it to organize experiments and share results

without relying only on notebooks.

The third possible product direction is professional support and integration. Organizations that want to use Accelera internally may need help adapting the framework to their datasets, infrastructure, or benchmark requirements. This creates a possible support and consulting model.

The Deployment module also creates a practical commercial path for a managed deployment tier. A hosted product could provide prepared deployment templates, private model-serving environments, monitored endpoints, and simplified rollback support. Even when the current implementation remains lightweight and developer-controlled, its presence makes Accelera easier to present as a complete platform because it connects experimentation with operational use.

2.3.2 Financial Analysis

The financial analysis can be divided into expected costs and possible revenue streams. Since Accelera is mainly a software project, the capital expenditure is limited compared with hardware-based products. Most costs come from development, hosting, cloud resources, storage, and maintenance.

Table 2.4: Expected cost categories.

Cost category	Description	Notes
Development cost	Time spent implementing, testing, documenting, and maintaining the framework.	Main cost during early stages.
Cloud hosting	Servers for benchmark back-end, database, frontend, and possible job execution.	Can start small and scale with users.
Storage	Datasets, submissions, reports, and model artifacts.	Increases with benchmark usage.
Compute resources	Running benchmark submissions or managed experiments.	Can be limited by quotas or paid tiers.
Maintenance and support	Bug fixes, user support, documentation, and security updates.	Needed for long-term usage.

Possible revenue streams include:

- **Free open-source adoption:** increases user base and community trust.
- **Hosted benchmark subscriptions:** paid private benchmarks, team workspaces, and larger storage limits.

- **Managed experiment dashboards:** paid reporting and workflow monitoring for teams.
- **Enterprise support:** integration, customization, and technical support for organizations.
- **Education packages:** university or training-center packages for teaching machine-learning workflows.

A simple financial plan can start with low-cost hosting and manual support. In the first stage, the focus should be adoption and validation. The open-source framework can attract users, while the benchmark platform can collect early feedback. In the second stage, private benchmark spaces and team dashboards can be introduced. In the third stage, enterprise support and managed compute can be added if the user base grows.

Table 2.5: Simplified business plan stages.

Stage	Focus	Expected result
Stage 1	Release open-source framework, documentation, examples, and local benchmark prototype.	Build trust, collect users, and validate the technical direction.
Stage 2	Launch hosted benchmark features, public leaderboards, and private team workspaces.	Create first revenue stream through hosted services.
Stage 3	Add managed experiment dashboards, larger storage, and paid support.	Serve teams and organizations with more advanced workflow needs.
Stage 4	Provide enterprise integration and managed compute for benchmark execution.	Scale the platform into a commercial ML workflow service.

The expected break-even point depends on hosting cost, development effort, and the number of paying users. Since the software can begin as open source, the initial financial risk is lower than products that require manufacturing or special hardware. The main challenge is user adoption. If Accelera can attract students, researchers, and small teams through the free framework, the benchmark and reporting services can become the commercial layer that supports future development.

Chapter 3: Literature Survey

In this chapter, we give an overview of the topics, algorithms, and techniques needed to understand the project as a whole. We discuss the technical subjects and background knowledge, and then summarize the research papers related to our project.

3.1 Background and Previous Work

This section introduces the main background needed to understand the Accelera project as a whole. It explains the important concepts, techniques, and tools that were used in different parts of the system. These background topics help clarify how the project supports data preparation, pipeline execution, report generation, model evaluation, and benchmarking. The goal of this section is to give the reader enough context before moving to the methodology and implementation chapters.

3.1.1 Data Science and Machine Learning

Data science is the field that uses data, statistics, programming, and domain knowledge to understand a problem and extract useful insights from it. It is usually used when we have raw data and we want to turn it into useful information or predictions.

Machine learning is one of the main parts of data science. Instead of writing fixed rules for every case, we train a model on examples, and the model learns patterns from these examples. After training, the model can use what it learned to make predictions on new data.

In supervised machine learning, the training data contains input features and a target label. The model learns the relationship between the features and the target. If the target is a class, such as spam or not spam, the problem is called classification. If the target is a continuous number, such as price or temperature, the problem is called regression.

3.1.1.1 Data Exploration and Preprocessing [1]

Exploratory Data Analysis, or EDA, is the step where we inspect the dataset before building the model. We summarize columns, draw visualizations, check missing values, and try to understand relationships between variables. This step helps us notice problems early instead of discovering them after training the model.

Data preprocessing is the stage of transforming raw data into a clean and usable format for machine learning models. In real datasets, the data may contain missing values, duplicated rows, different feature scales, and categorical columns that cannot be used directly by many models. Because of this, preprocessing is usually considered an essential part of a machine learning workflow.

Missing value imputation is used when some values in a dataset are empty. Instead of removing all rows that contain missing values, the missing values can be replaced using a suitable value, such as the mean, median, mode, or another estimated value. Robust scaling is a scaling technique that uses the median and the interquartile range instead of the mean and standard deviation. It is useful when the data contains outliers, because the median and interquartile range are less affected by extreme values:

$$x' = \frac{x - \text{median}(X)}{\text{IQR}(X)}$$

where x' is the scaled value, x is the original value, $\text{median}(X)$ is the median of the feature values, and $\text{IQR}(X)$ is the interquartile range.

Categorical encoding is the process of converting text categories into numeric values. Many traditional machine learning models and preprocessing pipelines expect numerical input. Two examples are binary encoding and frequency encoding. Binary encoding first gives each category a number, then converts this number into binary form and stores the binary digits in separate columns. Frequency encoding replaces each category with how often it appears in the dataset:

$$f(c) = \frac{\text{count}(c)}{N}$$

where $f(c)$ is the frequency of category c , $\text{count}(c)$ is the number of rows containing this category, and N is the total number of rows.

3.1.1.2 Model Evaluation Metrics [2]

After training a model, we need metrics to check whether the model is performing well or not. The selected metric depends on the type of problem. Classification models are evaluated using metrics that compare predicted classes with actual classes, while regression models are evaluated using metrics that measure the difference between predicted numerical values and real values.

Classification metrics: Classification metrics are used when the output is a class or category, such as spam or not spam, positive or negative, or one image class from many classes. The confusion matrix is the base idea behind many classification metrics. It compares the actual classes with the predicted classes. For binary classification, it contains four values: true positives, true negatives, false positives, and false negatives.

- **True Positive (TP):** The model predicts positive, and the actual class is positive.
- **True Negative (TN):** The model predicts negative, and the actual class is negative.
- **False Positive (FP):** The model predicts positive, but the actual class is negative.

- **False Negative (FN):** The model predicts negative, but the actual class is positive.

Accuracy measures how many predictions are correct out of all predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is true positives, TN is true negatives, FP is false positives, and FN is false negatives. Accuracy is simple and useful when the classes are balanced, but it may be misleading when one class is much larger than the other.

Precision measures how many predicted positive samples are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

Precision is useful when false positives should be reduced.

Recall measures how many actual positive samples were correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

Recall is useful when false negatives should be reduced.

The F1-score combines precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It is useful when both precision and recall are important, especially with imbalanced datasets.

Regression metrics: Regression metrics are used when the output is a numerical value, such as price, score, or temperature.

Mean Absolute Error measures the average absolute difference between the real values and predicted values:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of samples. MAE is easy to understand because it gives the average error in the same unit as the target value.

Mean Squared Error gives more penalty to large errors:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

This means large mistakes affect the score more than small mistakes.

Root Mean Squared Error is the square root of MSE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE is easier to read than MSE because it returns the error to the same unit as the target value.

R-squared score measures how well the regression model explains the variation in the actual values:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and $\bar{y}$ is the mean of the actual values. A value close to 1 means the model explains the data well, while a value close to 0 means the model does not explain the data well.

3.1.2 Natural Language Processing (NLP) [3]

Natural Language Processing is the field that helps computers understand and work with human language. It is used in many applications such as text classification, sentiment analysis, spam detection, and chatbots. Text data cannot be used directly by most machine learning models, so it needs to be cleaned and converted into numbers first.

In text preprocessing, the text may be converted to lowercase, cleaned from unnecessary symbols, split into words, and simplified using stemming. After that, methods such as TF-IDF are used to transform the text into numerical features. In Accelera, these steps are used to prepare text datasets so they can be used for machine learning models.

3.1.2.1 Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency-Inverse Document Frequency is one common technique used to convert text into numerical features. It gives each word a weight based on how important it is in one document and across the full dataset. Term Frequency measures how often a word appears in a document. Inverse Document Frequency gives lower weight to words that appear in many documents, because these words are usually less useful for classification:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

where $TFIDF(t, d)$ is the weight of term t in document d , $TF(t, d)$ is the frequency of the term in the document, and $IDF(t)$ measures how rare the term is across all documents.

3.1.3 Data Augmentation

Data augmentation is a technique used to increase the diversity of a dataset without actually collecting new data. It works by applying transformations to existing samples to create new modified versions. For example, in image classification, augmentation may include flipping, rotating, cropping, or changing brightness. This can help the model generalize better because it sees more variations during training.

3.1.4 Machine Learning Pipelines

A machine learning pipeline is a sequence of steps used to prepare data, train a model, make predictions, and evaluate results. Pipelines are useful because they organize the full workflow in one structure instead of writing each step separately.

A typical pipeline may include preprocessing steps such as scaling, encoding, or feature selection, followed by a machine learning model. This makes experiments easier to repeat and reduces mistakes, because the same preprocessing steps are applied during training and testing.

One of the most common tools for building machine learning pipelines is `scikit-learn`. It provides a `Pipeline` object that connects transformers and estimators together. In this project, `scikit-learn` is considered the main competitor for the `accelera_pipe` module because both systems aim to help users build machine learning workflows in Python.

3.1.5 Graph-Based Pipeline Execution

Traditional machine learning pipelines usually execute steps in a linear order. This is suitable for simple workflows, but it can become inefficient when several branches share the same preprocessing steps or when multiple models are compared.

Graph-based execution represents the workflow as nodes and edges. Each node represents an operation, such as preprocessing, model training, prediction, or metric evaluation. Each edge represents the dependency between operations. This representation can help avoid repeated work, because shared steps can be executed once and reused by different branches.

The `accelera_pipe` module follows this idea by representing the pipeline internally as an execution graph. This allows the system to support branching, merging, parallel execution of independent paths, and better management of intermediate data.

3.1.6 Code Parallelization

Code parallelization is the process of running parts of a program at the same time to reduce execution time. In many programs, the most expensive part is a loop that repeats the same operation over a large amount of data. If the loop iterations are independent, they may be executed in parallel.

Manual parallelization is difficult because the developer must check that there are

no unsafe dependencies between iterations. If a loop writes to shared variables or depends on results from previous iterations, parallel execution may produce wrong results. Because of this, automatic parallelization tools try to analyze code and decide whether a loop can be safely parallelized.

3.1.7 OpenMP

OpenMP is a programming interface used to add parallel execution to C and C++ programs. It allows developers to use compiler directives, called pragmas, to tell the compiler that some parts of the code can run in parallel.

For example, the directive `#pragma omp parallel for` can be used before a loop to distribute loop iterations across multiple threads. OpenMP is useful because it does not require rewriting the whole program. However, the pragma must be inserted only when the loop is safe to parallelize.

3.1.8 Deployment

Deployment is the process of moving software from a development environment to an environment where users or other systems can access it. In machine-learning projects, deployment is important because the trained model must be connected to software that receives input, runs inference, and returns predictions.

3.1.9 Model Deployment

Model deployment means packaging a trained model together with the code and dependencies needed to use it. A deployed model is usually exposed through a service endpoint so that other applications can send data and receive predictions. This is different from only saving a model file, because a saved file still requires a runtime service before it becomes usable by external users.

3.1.10 Machine Learning Deployment

Machine-learning deployment adds extra concerns beyond ordinary software deployment. The system must preserve preprocessing steps, load model binaries, validate input schemas, handle prediction errors, and sometimes track requests for monitoring. These concerns explain why Accelera treats deployment as a module connected to the ML workflow rather than only as a cloud-hosting task.

3.1.11 Cloud and Server Deployment

Cloud deployment runs a service on infrastructure provided by a cloud provider. Server deployment runs the service on a machine that may be local, virtual, or cloud-hosted. In both cases, the deployed service must receive network traffic on a configured port and must be started with the correct dependencies and environment variables.

3.1.12 Web Services, APIs, and REST APIs

A web service is a program that exposes functionality over the network. An API defines how other programs interact with that service. A REST API commonly uses HTTP methods and URLs to exchange structured data. In the Deployment module, prediction requests are sent to API endpoints such as `/predict`, while status information is exposed through endpoints such as `/health`.

3.1.13 HTTP, HTTPS, JSON, URLs, and Ports

HTTP is the common protocol used by web clients and servers. HTTPS is the secure version of HTTP. JSON is a text format frequently used to send structured data between clients and APIs. A URL identifies where a service can be reached, and a port identifies the network entry point used by the service on a machine. These concepts are needed to understand deployed prediction endpoints.

3.1.14 Docker and Containerization

Docker is a containerization tool that packages an application with its runtime environment. A container helps the same service run consistently across machines because dependencies, files, and startup commands are described explicitly. The Deployment module uses Docker to package the FastAPI serving application, model files, configuration, and required Python libraries into one runnable service image.

3.1.15 Virtual Machines, AWS, and AWS EC2

A virtual machine is a software-defined computer that runs on physical cloud hardware. AWS is a major cloud provider, and EC2 is its virtual-machine service. Deploying to EC2 gives the user flexible control over the server, but it also requires practical infrastructure steps such as SSH access, file synchronization, Docker installation, port exposure, and health checking.

3.1.16 Heroku and Container Hosting

Heroku is a platform for deploying web applications with less direct server management. It can run containerized applications through its container registry. In Accelera, Heroku is treated as a deployment target where the prepared Docker service can be pushed and released through Heroku CLI commands.

3.1.17 SSH and CLI Tools

SSH is a secure protocol for connecting to remote machines and running commands. A CLI is a text-based interface used to execute commands from a terminal. The Deployment module uses CLI workflows for local Docker, Heroku, and EC2. For EC2, SSH is used to run remote commands, build Docker images, start containers, and read logs.

3.1.18 Environment Variables and Configuration Files

Environment variables are runtime values provided outside the source code, such as the port on which a server should listen. Configuration files store structured settings used by the program. Accelerera uses `config.json` to locate model artifacts, define input schema rules, and configure tracking behavior.

3.1.19 Pickle Files and Model Artifacts

Pickle files with the `.pkl` extension are commonly used in Python to serialize trained objects such as Scikit-learn models or preprocessors. In the Deployment module, these files are treated as model artifacts. They are copied into the deployment workspace, loaded by the model service, and used during inference.

3.1.20 Version Control, Git, GitHub, and Repositories

Version control records changes over time and helps users return to previous states. Git is a widely used version-control system, GitHub hosts Git repositories, and a repository stores a project's files and history. Accelerera also implements a lightweight model VCS for deployment artifacts. This VCS is not a replacement for Git; it snapshots model files and deployment configuration so that earlier model versions can be restored.

3.1.21 Backend Services, FastAPI, and Uvicorn

A backend service runs server-side logic and responds to client requests. FastAPI is a Python framework for building APIs, and Uvicorn is an ASGI server used to run FastAPI applications. The Deployment module uses FastAPI to define prediction and health endpoints, while Uvicorn starts the application inside the Docker container.

3.1.22 Deployment Scripts

Deployment scripts automate repeated infrastructure commands. Instead of asking the user to copy files, build containers, start services, and check endpoints manually, a script can execute these steps in sequence. Accelerera's `deployment.py` script provides commands for preparing files, building Docker images, running locally, deploying to Heroku, and deploying to EC2.

3.2 Comparative Study of Previous Work

3.2.1 Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [4]

Koukaras and Tjortjis discuss the role of data preprocessing and feature engineering in machine learning and data mining projects. They explain that preprocessing is not only a preparation step before model training, but also an important part of the complete machine learning pipeline. The paper also presents a general preprocessing framework that includes data cleaning, data transformation, feature construction, feature selection, and dimensionality reduction. For each stage, it explains different techniques and discusses the advantages and disadvantages of each one.

3.2.2 `scikit-learn` Pipeline

`scikit-learn` is one of the most widely used machine learning libraries in Python [9]. It provides many preprocessing tools, machine learning models, evaluation metrics, and utilities for building complete machine learning workflows.

The `Pipeline` object in `scikit-learn` allows users to connect several processing steps together. Each step is usually a transformer, and the final step is usually an estimator. This makes the workflow cleaner and easier to reuse.

In this project, `scikit-learn` is the main competitor for `accelera_pipe`. Both systems provide a way to build machine learning workflows. However, `scikit-learn` pipelines are mainly linear, while `accelera_pipe` uses a graph-based backend. This allows `accelera_pipe` to support branching, shared intermediate results, and parallel execution of independent parts of the workflow.

3.2.3 OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation

OMPPar is a related work that focuses on automatic parallelization using AI-driven source-to-source compilation [10]. The main idea of OMPPar is to analyze C and C++ code, detect loops that can be parallelized, and generate OpenMP pragmas automatically.

OMPPar is important for this project because it inspired the general direction of the Parallelizer module. Both approaches use a staged process that includes loop analysis, loop classification, and pragma insertion. This makes the problem easier to handle because the system does not treat parallelization as one single step.

However, there is an important difference between OMPPar and our Parallelizer. OMPPar uses an AI-based pragma generation approach. In our project, the pragma generator was replaced with a rule-based generator. This decision was made because the

available training data was limited, and some of the available examples were synthetic data. Depending completely on generated or biased data could lead to unsafe pragma generation.

Therefore, our Parallelizer uses AI mainly to support loop classification, while the final pragma insertion is controlled by deterministic safety rules. This makes the system more conservative, but also safer and easier to explain.

3.2.4 Automated Machine Learning

Automated machine learning treats model-family selection, preprocessing choices, and hyperparameter tuning as one optimization problem. Instead of manually testing a small set of estimators, an AutoML system builds a search space and evaluates configurations under a consistent validation protocol. The combined algorithm selection and hyperparameter optimization problem can be written as

$$(a^*, \lambda^*) = \arg \min_{a \in \mathcal{A}, \lambda \in \Lambda_a} \widehat{L}(a, \lambda; D),$$

where $\mathcal{A}$ is the set of candidate algorithms, Λ_a is the hyperparameter domain of algorithm a , and $\widehat{L}$ is the validation loss on dataset D . This formulation was popularized by Auto-WEKA [5].

3.2.5 Search Strategies for AutoML

Grid search evaluates a fixed Cartesian product and becomes expensive as the number of dimensions increases. Random search is often stronger when only some hyperparameters strongly affect performance [13]. Sequential model-based optimization improves on random sampling by fitting a surrogate model to observed configuration costs. SMAC uses random forests for algorithm configuration and acquisition-function based candidate selection [12].

3.2.6 Multi-Fidelity Evaluation and Warm Starts

Full cross-validation on the complete dataset is costly, especially for ensembles and iterative models. Multi-fidelity optimization first evaluates candidates using cheaper approximations, such as smaller samples, fewer folds, or reduced model budgets. Poor candidates can be discarded early, while promising candidates are promoted to more expensive stages.

Meta-learning is another way to reduce cold-start cost. A new dataset can be described using metafeatures, compared with historical datasets, and initialized with configurations that worked well on similar tasks. Auto-sklearn combines Bayesian optimization, dataset metafeatures, and ensemble construction for Scikit-learn workflows [6].

3.2.7 Ensemble Learning in AutoML

Ensembles reduce the risk of selecting one unstable model. Voting combines predictions or probabilities from several models. Stacked generalization trains a meta-model on out-of-fold predictions from base models [14]. The out-of-fold step is important because it prevents the meta-model from learning on predictions made by base models on their own training rows.

Table 3.1: Selected AutoML systems and their design emphasis.

System	Design emphasis
Auto-WEKA	Combined algorithm selection and hyperparameter optimization over WEKA components [5].
Auto-sklearn	Bayesian optimization, dataset metafeatures, and post-hoc ensembles for Scikit-learn workflows [6].
TPOT	Evolutionary search over tree-structured machine-learning pipelines [7].
FLAML	Cost-aware lightweight AutoML for limited resource budgets [8].
AutoGluon Tabular	Multi-layer stacking and ensembling with a high-level tabular prediction interface [11].
Accelera AutoML	Conditional model spaces, staged fidelity, surrogate-guided sampling, packaged warm starts, and voting or stacking inside Accelera.

3.2.8 Model Deployment Platforms

Deployment platforms such as Heroku, AWS EC2, AWS Elastic Beanstalk, Render, Railway, Google Cloud Run, and Hugging Face Spaces provide ways to host applications or model demos. Their role is important because they give developers infrastructure for running web services. However, these platforms mainly provide the hosting layer. The user usually remains responsible for building the ML serving application, preparing dependency files, loading model artifacts, validating inputs, logging predictions, and maintaining model versions.

Accelera's Deployment module is different in scope. It is not a general cloud provider; it is an ML-oriented deployment layer inside the Accelera workflow. It takes the generated configuration and model pickle files, prepares a FastAPI serving container, exposes prediction endpoints, runs schema validation, records telemetry, and provides commands for local Docker, Heroku, and AWS EC2 deployment. Therefore, Accelera complements infrastructure platforms by preparing the ML service itself and connecting it to the artifacts created by the rest of the framework.

Table 3.2: Deployment-related platforms and Accelera's difference.

Platform	Main focus	Accelera difference
Heroku	Application hosting with container deployment support.	Accelera prepares the ML serving container and provides Heroku-specific commands for pushing and releasing it.
AWS EC2	General-purpose virtual machines with full server control.	Accelera automates file synchronization, remote Docker build/run steps, and health checks for model services.
Render, Railway, or Cloud Run	Managed hosting for web services or containers.	Accelera focuses on creating the ML API, validation, telemetry, and deployment package before such hosting is considered.
Hugging Face Spaces	Hosted ML demos and interactive applications.	Accelera emphasizes integration with local model artifacts and repeatable deployment workflows inside the project.

3.3 Implemented Approach

3.3.1 Auto Preprocessing

In this module we decided to implement an automated preprocessing approach for tabular datasets. The approach was designed to prepare data for machine learning models by applying suitable preprocessing steps automatically according to the type and characteristics of each column and some user configurations. The chosen approach follows a classical automated preprocessing pipeline. The main preprocessing decisions are:

- **Missing values:** Median imputation is used for numerical features because it is simple and more robust to outliers than mean imputation. For categorical features, most-frequent imputation is used because it replaces missing values with valid existing categories.
- **Numerical scaling:** `RobustScaler` is used for numerical features because tabular datasets may contain outliers or skewed values. It depends on the median and interquartile range, which makes it more stable than standard scaling when extreme values exist.
- **Categorical encoding:** The encoding method is selected according to the fea-

ture type and cardinality. Binary columns are encoded using ordinal encoding, low-cardinality categorical columns are encoded using binary encoding, and high-cardinality categorical columns are encoded using frequency encoding. This helps reduce the number of generated features and avoids high dimensionality.

- **Feature selection:** Mutual information is used because it is a filter-based method that can work with both classification and regression tasks. It helps keep the most informative features and remove weak features before model training.

This approach was chosen because it gives a good balance between automation, simplicity, and speed. More advanced techniques, such as K-nearest neighbour imputation, autoencoders, and wrapper feature selection, can be useful in specific cases, but they need more computation.

One of the important additions in this project is that the module is not only used for data preparation. It also supports exploratory data analysis report.

Also the module was extended to support text and image data preparation, not only tabular data.

3.3.2 accelera_pipe

In this module, we decided to implement a graph-based machine learning pipeline approach. The goal of this module is to make machine learning workflows easier to build while also improving execution when the workflow contains branches or repeated shared steps.

The general framework of the approach is as follows:

1. The user builds a pipeline using high-level Python methods.
2. The pipeline stores each operation as a node in an internal graph.
3. Preprocessing, model training, prediction, metric evaluation, branching, and merging are represented as graph operations.
4. The graph executor determines the correct order of execution.
5. Independent branches can be executed in parallel when possible.
6. Shared intermediate results can be reused instead of recomputed.
7. Intermediate data can be released when it is no longer needed.
8. The executed graph can be stored and reused for inspection or later execution.

This approach was chosen because it keeps the pipeline idea familiar to users who already know `scikit-learn`, but changes the internal execution model. Instead of treating the workflow as only a sequence of steps, `accelera_pipe` treats it as a graph. This helps the system handle more complex workflows that include multiple models, shared preprocessing, and branch comparisons.

3.3.3 Parallelizer

In this module, we decided to implement a conservative automatic parallelization approach. The goal is to speed up supported loop-heavy code by converting it to C++, analyzing its loops, inserting safe OpenMP pragmas, and compiling it into a native Python-callable extension.

The general framework of the approach is as follows:

1. Take the input source code, file, function, or transformer method.
2. Convert supported Python code into C++ when needed.
3. Extract loops using a compiler-based analysis stage.
4. Compute loop features such as memory access, dependency indicators, reduction patterns, branches, and function calls.
5. Classify whether the loop may be parallelized.
6. Apply rule-based safety checks before inserting any pragma.
7. Insert `#pragma omp parallel for` or a reduction pragma only when the loop is considered safe.
8. Compile the transformed C++ code into a native extension.
9. Return the optimized callable to be used from Python.

The chosen approach follows the same general idea as AI-assisted parallelization systems, but with a safer final pragma generation stage. The pragma generator is rule-based instead of fully AI-generated because the available data was limited and partly synthetic. This reduces the risk of generating unsafe OpenMP directives.

The Parallelizer is also connected to `accelera_pipe`. When a preprocessing node contains a supported custom Python function, the pipeline can attempt to optimize this function using the Parallelizer. This allows the project to combine graph-level workflow execution with code-level acceleration.

3.3.4 Deployment Module Technologies

The Deployment module uses a set of practical software-engineering technologies to move trained artifacts toward a runnable service. This section introduces the tools used by the module; the detailed module architecture is described later in Section 4.7.

Git-style versioning inspired the internal model VCS. The module does not replace Git for source code, but it applies the same idea of named snapshots to deployment artifacts. The VCS stores model files and `config.json` inside commit directories, records metadata in an index file, and allows a previous model snapshot to be restored before deployment.

FastAPI and Uvicorn form the serving layer. FastAPI defines HTTP endpoints for single prediction, CSV prediction, health checks, tracking summaries, and the interactive GUI. Uvicorn runs the application inside the container and listens on the configured service port.

Docker is used as the portability layer. The deployment script writes a `Dockerfile`, copies serving files, installs runtime dependencies, adds the saved model artifacts, exposes the service port, and starts the Uvicorn server. This keeps the runtime environment more repeatable than launching the service directly from an arbitrary local Python environment.

Great Expectations supports the runtime validation layer. The configuration file defines expected feature names, data types, required fields, allowed values, and numeric bounds. Incoming requests are checked and coerced before they reach the model service, which reduces the risk of invalid payloads causing hidden prediction errors.

SSH and `rsync` are used for AWS EC2 deployment. The script connects to the remote machine, creates the target directory, synchronizes the `Dockerfile`, configuration, serving source, and model files, builds the container on the remote host, starts it, and checks the health endpoint. Heroku deployment uses Heroku CLI commands to authenticate, push the container image, release it, and open the deployed application.

Configuration files and generated artifacts are the connection point between training and deployment. The `config.json` file tells the service where to find model pickle files, how to validate input data, and whether telemetry should be enabled. The deployment scripts then convert these local artifacts into a concrete service package.

Chapter 4: System Design and Architecture

4.1 Overview and Assumptions

This section introduces the assumptions made for each module in Accelerera and explains why these assumptions were needed during the implementation.

4.1.1 Accelerera Pipeline Assumptions

The system combines Python-facing APIs with native execution components. The design assumes that machine-learning workflows can be represented as a graph of operations and that only a restricted, analyzable subset of custom source code should be translated to parallel C++.

4.1.2 Auto Preprocessing Assumptions

The Auto Preprocessing module was designed based on the following assumptions:

- **Data assumption:** The input data should be valid, meaningful, and one of the supported types, so the module can select the correct preprocessing workflow.
- **Configuration assumption:** The user should provide the needed configuration values, so the module can apply the correct preprocessing steps.

4.1.3 Benchmark Assumptions

The Benchmark Platform was designed based on the following assumptions:

- **Benchmark data assumption:** The user is assumed to provide the benchmark files correctly, including the training data, test data, and separate target file. The user is also responsible for ensuring that the file separation does not cause data leakage, so the platform can evaluate submissions fairly.
- **Metric and submission assumption:** The admin is assumed to define the correct metric information for the benchmark, and the user is assumed to submit predictions in the required format, so the scoring script can calculate the result correctly.

4.1.4 Deployment Assumptions

The Deployment module assumes that the selected model or pipeline has already produced serializable model artifacts and a deployment configuration file. The configuration must describe the model files relative to the deployment workspace, the expected

input schema when validation is enabled, and the tracking settings used by the serving application.

For local execution and cloud/server execution, the module assumes that Docker is available on the target machine or can be installed during the EC2 workflow when the corresponding option is selected. For remote EC2 deployment, the user must provide a reachable host, a valid SSH user, and any required private key. The public port used by the service must also be reachable from the client machine for the final health check and GUI endpoint to work.

4.2 System Architecture

4.2.1 Block Diagram

The block diagram in Figure 4.1 gives the big picture of Accelera. It shows the system as a group of connected modules rather than one single fixed pipeline. This is intentional: a user can use each module alone, or can connect several modules together to build an end-to-end machine-learning workflow.

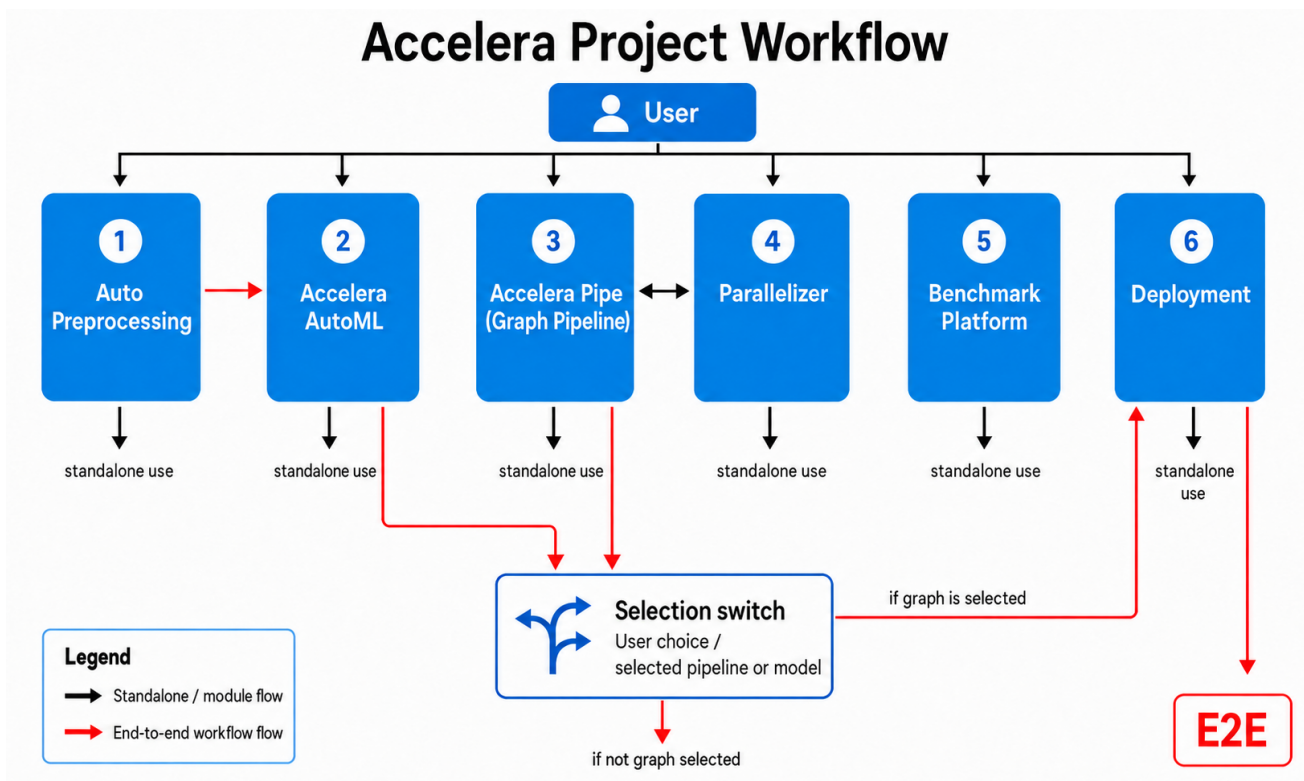


Figure 4.1: Overall block diagram of the Accelera project architecture.

At the highest level, the user can interact directly with the six main parts of the project: Auto Preprocessing, Accelera AutoML, Accelera Pipe, Parallelizer, Benchmark Platform, and Deployment. The blue arrows in the diagram represent standalone or module-level flows, while the red arrows represent the end-to-end workflow path.

The architecture follows three broad stages: data preparation, model or graph construction, and artifact consumption. Auto Preprocessing prepares reusable features and preprocessing artifacts; AutoML and Accelera Pipe provide alternative model-building paths; and Deployment or the Benchmark Platform consumes the resulting model, executed graph, endpoint, or prediction file. The Parallelizer is a supporting optimization module that can operate independently on source code or accelerate supported custom preprocessing inside Accelera Pipe.

The red route represents the complete data-to-service lifecycle, while the blue routes show standalone use. Modules exchange explicit artifacts rather than internal state, allowing users to stop after any stage.

The Deployment module turns a saved model or pipeline into a validated API service by packaging model artifacts, configuration, serving code, and deployment commands. Its outputs may include a model URL, service endpoint, health endpoint, telemetry summary, and optional inference UI.

Auto Preprocessing flow

The Auto Preprocessing block starts from a dataset and tries to prepare it for machine learning usage. The module first profiles the data and detects its type. Then it applies preprocessing steps depending on the type of input.

- **Tabular data:** missing value imputation, categorical encoding, robust scaling, and feature selection.
- **Text data:** cleaning, tokenization, stemming, and TF-IDF feature extraction.
- **Image data:** image preparation for classification tasks.

The output can be used in two ways. In the standalone path, the user can directly use the processed data, reports, and saved preprocessors. In the end-to-end path, the processed data can be passed to the AutoML block for model search.

AutoML and selection flow

The AutoML block receives a dataset, extracts statistics and metafeatures, builds candidate models and preprocessing choices, and runs a search process. Its goal is to return the best model according to the selected metric.

The important point is that AutoML is not forced to be the only way to build a model. The user may choose an AutoML result, or may choose a graph-pipeline result from Accelera Pipe. This is why the diagram contains a selection switch.

Table 4.1: Possible paths to the deployment selection switch.

Path	Meaning
AutoML path	The user lets <code>accelera_automl</code> search for the best model, then chooses the selected model for deployment.
Graph pipeline path	The user builds a workflow using <code>accelera_pipe</code> , executes the graph, and selects the executed graph or best pipeline path for deployment.
Standalone path	The user may stop after any module and use its output directly without continuing to deployment.

Accelera Pipe module

Accelera Pipe is the graph-execution module of the system. Its role is to let the user define a machine-learning workflow as a graph instead of a fixed linear sequence. Each graph node represents one operation in the workflow, such as data input, preprocessing, model training, prediction, metric evaluation, merging, or branching. The edges between nodes describe data dependencies, meaning that a node can execute only after its required source nodes have produced their outputs.

This design allows Accelera Pipe to support workflows that are more flexible than a traditional sequential pipeline. For example, the graph may contain multiple model branches, shared preprocessing nodes, or metric nodes used to select the best path. The module is also responsible for managing execution order, selected-path extraction, persistence, and intermediate-memory lifetime. After training, the selected fitted path can be stored in an executed graph so it can be reused later for inference without keeping unnecessary temporary training data.

Accelera Pipe therefore works at the workflow level. It does not only run a list of steps; it manages how the steps are connected, when they are executed, which branches are selected, and which intermediate results can be released safely.

Formally, a pipeline is represented as a directed acyclic graph

$$G = (V, E), \quad E \subseteq V \times V, \quad (4.1)$$

where each vertex $v \in V$ is an executable node and each edge $(u, v) \in E$ indicates that v consumes the output of u . A node becomes ready only after all its predecessors have completed:

$$\text{ready}(v) \iff \forall u \in \text{Pred}(v), \text{state}(u) = \text{finished}. \quad (4.2)$$

Ready nodes without mutual dependencies may be scheduled concurrently by the C++ backend. This is the basis of parallel branch execution; it is different from OpenMP loop parallelization, which operates inside one custom function.

Table 4.2: Principal node families in an Accelera Pipe graph.

Node family	Architectural responsibility
Preprocess	Fits or applies a transformer and forwards transformed features.
Model	Fits a supervised or unsupervised estimator using upstream data.
Predict	Applies a fitted model method such as <code>predict</code> or <code>predict_proba</code> .
Metric	Evaluates predictions and supplies values used for path selection.
Branch/Merge	Expands alternatives or combines compatible branch outputs.

The graph also applies consumer-count-based lifetime management. If $c_t(u)$ is the number of unfinished downstream consumers of node u after execution step t , then completing consumer v updates the count by

$$c_{t+1}(u) = c_t(u) - 1, \quad u \in \text{Pred}(v). \quad (4.3)$$

Temporary output from u becomes releasable when $c_{t+1}(u) = 0$. Consequently, a transformed array shared by several model branches remains alive until the last model consumes it, then can be cleared without affecting correctness. Duplicate first pre-processing nodes may also be shared, allowing equivalent branches to reference one transformed result rather than repeat the same work.

After metric evaluation, the configured selection strategy determines the path retained in the `ExecutedGraph`. This inference graph keeps fitted preprocessors and models but does not need transient training arrays. It can be serialized as one pickle artifact and later called with new input data.

Code Parallelizer module

The Code Parallelizer is the code-level optimization module of the system. Its main role is to analyze supported loop-heavy code and attempt to accelerate it using C++ and OpenMP. The module can be used as a standalone tool or as an internal optimization step when Accelera Pipe receives a custom preprocessing function.

The Parallelizer accepts several kinds of input:

- C/C++ file or source code.
- Python file or Python source code.
- Python function.
- Custom transformer method.

Table 4.3: Parallelizer input dispatch and output behavior.

Input	Normalization	Result
C/C++ file	Read native source directly	Parallelized source file.
Python file	Convert supported Python AST to C++	Parallelized source file.
C/C++ or Python string	Detect syntax and convert Python when required	Parallelized C++ source string.
Python function	Capture source and helpers, flatten nested functions, then convert	Cached native Python-callable or original function on failure.
Custom transformer	Extract its custom <code>transform()</code> method	Optimized method attached to the same instance when compilation succeeds.

After receiving the input, the module first normalizes it into a form that can be analyzed. If the input is supported Python code, it is converted into C++ so that both

Python-originated and native C/C++ code can follow the same analysis path. The module then extracts loops using a Clang AST based analysis stage. After loop extraction, it computes features such as memory access patterns, dependencies, reductions, branches, nested loops, function calls, and vectorization potential.

These features are passed through a classifier and combined with deterministic rule-based safety checks. This is important because the classifier alone is not trusted to change program semantics. If the loop is considered safe, the module inserts an OpenMP pragma such as `#pragma omp parallel for`. If the loop matches a reduction pattern, the module can generate an OpenMP reduction clause. If the loop is unsafe or unsupported, it is left unchanged.

For each extracted loop L , feature extraction produces a fixed-dimensional vector $\mathbf{x}(L) \in \mathbb{R}^{40}$. The classifier proposes a class from

$$\hat{y}(L) \in \{\text{parallel_for}, \text{reduction}, \text{none}\}. \quad (4.4)$$

The proposal is then resolved with deterministic evidence. In simplified form, the implemented decision can be written as

$$D(L) = \begin{cases} \hat{y}(L), & \hat{y}(L) \neq \text{none}, \\ \text{reduction}, & L \text{ contains a supported scalar reduction,} \\ \text{parallel_for}, & L \text{ has independent array writes,} \\ \text{parallel_for}, & L \text{ is a supported consecutive nest,} \\ \text{none}, & \text{otherwise.} \end{cases} \quad (4.5)$$

The safety analysis rejects evidence such as loop-carried dependencies, indirect writes, and early exits before an independent-write rule is accepted. For supported reductions, the generated directive has the form

$$\text{\#pragma omp parallel for reduction}(\odot : r), \quad (4.6)$$

where r is the reduction variable and $\odot$ is the recognized operator.

The final pragma generation is intentionally rule-based. This makes the behavior safer and easier to inspect, especially when the available data for fully AI-generated pragma generation is limited or synthetic.

Interaction between Accelera Pipe and the Parallelizer

The two modules interact at custom preprocessing nodes. If an Accelera Pipe graph contains a preprocessing node with a supported custom method, the graph can send that method to the Parallelizer before storing or executing it. The Parallelizer then attempts to analyze, rewrite, compile, and cache an optimized version of the method. If successful, the optimized callable is attached back to the same graph node. If opti-

Table 4.4: Parallelizer stages.

Stage	Description
Input normalization	Accepts source code, files, functions, or transformer methods and prepares them for analysis.
Python-to-C++ conversion	Converts supported Python code into C++ so it can enter the same loop-analysis path as native C/C++ code.
Loop extraction	Uses compiler-style analysis to locate loops and record loop ranges.
Feature extraction	Computes indicators such as reductions, dependencies, array access, branches, function calls, and vectorization potential.
Safety decision	Combines classifier output with deterministic rule-based checks to avoid unsafe parallelization.
Pragma insertion	Adds OpenMP pragmas only when the loop is considered safe.
Compilation and caching	Builds optimized native code when needed and reuses cached results when possible.

mization fails, the original method is kept.

1. The user defines a graph workflow in `accelera_pipe`.
2. The graph detects a supported custom preprocessing method.
3. The method is sent to the Parallelizer.
4. The Parallelizer analyzes, rewrites, compiles, and caches the optimized implementation when possible.
5. The optimized callable is returned and reattached to the graph node.
6. The graph continues execution using either the optimized method or the original method if optimization fails.

This interaction connects workflow-level optimization with code-level optimization. Accelera Pipe controls the structure and execution of the machine learning graph, while the Parallelizer attempts to speed up supported custom code inside that graph. As a result, the system can optimize both the way the workflow is scheduled and the way expensive preprocessing code is executed.

The replacement contract preserves behavior at the pipeline boundary. Let f be the original Python preprocessing callable and let $\mathcal{C}(f)$ denote source capture, conversion, OpenMP rewriting, compilation, and module loading. The callable attached to the preprocessing node is

$$f_{\text{node}} = \begin{cases} \mathcal{C}(f), & \text{if every optimization stage succeeds,} \\ f, & \text{if source conversion, analysis, or compilation fails.} \end{cases} \quad (4.7)$$

This fallback contract gives the integration three useful properties:

- **Transparent use:** the graph calls the node through the same Python interface regardless of which implementation is attached.
- **Failure isolation:** unsupported syntax or a compiler failure disables the optimization for that callable without invalidating the graph.
- **Persistent reuse:** source-backed functions retain serializable Python source, while successful native modules can be recovered from cache.

Benchmark Platform flow

The Benchmark Platform is used for evaluation and ranking. It does not take a full user pipeline as input. Instead, it works with benchmark data links and user prediction submissions.

A benchmark owner creates a benchmark by providing the required CSV links:

- training CSV file link,
- testing CSV file link,
- true prediction or ground-truth CSV file link.

A participant submits:

- a predictions CSV file containing the prediction column,
- and optionally a GitHub repository URL for the project.

The platform compares the submitted prediction column with the true labels, computes the score, and updates the ranking. This makes the benchmark module focused on fair evaluation and leaderboard comparison, not on directly executing the user's pipeline.

Deployment flow

The Deployment block receives saved model artifacts and the active deployment configuration. It prepares the deployment service by collecting the model files, copying the serving source code, writing runtime requirements, generating a Docker image definition, and exposing a FastAPI application. Inside the serving container, requests pass

through schema validation before reaching the model service. After inference, telemetry is written so the user can inspect request counts, statuses, latencies, and prediction activity.

The block also separates the service package from the target environment. The same prepared service can be built locally in Docker, pushed to Heroku through container commands, or synchronized to an AWS EC2 virtual machine over SSH and run as a remote Docker container. This design makes Deployment the final architectural bridge between the outputs of AutoML or graph pipelines and real-world usage through a reachable HTTP endpoint.

The final deployment block produces a deployed model URL or service. This output has two possible meanings:

- **Standalone use:** the user deploys a saved model or pipeline directly and uses the endpoint.
- **End-to-end use:** the user selects a model or pipeline through the selection switch, deploys it, and receives the final E2E service output.

Overall architecture meaning

The main idea of the block diagram is that Accelera is modular but connected. Each module gives value by itself, but the full system becomes stronger when the modules are combined.

Table 4.5: Standalone use versus end-to-end use.

Mode	Description	Example
Standalone use	The user uses one module and stops at its output.	Use Auto Preprocessing only to generate processed data and reports.
End-to-end use	The user connects several modules together to move from data to deployed result.	Preprocess data, run AutoML or graph pipeline, select output, deploy service.
Integrated acceleration	A module internally uses another module to improve execution.	Accelera Pipe sends supported custom preprocessing methods to the Parallelizer.

Therefore, the block diagram communicates three important ideas. First, the system supports direct access to each module. Second, the modules can be connected into a complete machine-learning workflow. Third, the interaction between Accelera Pipe and the Parallelizer allows the project to combine graph-based execution with source-level acceleration.

4.3 Accelera Pipe Module

4.3.1 Functional Description

The Accelera Pipe module can be described formally as a directed acyclic graph:

$$G = (V, E)$$

where V is the set of computational nodes and E is the set of directed dependency edges between them. Each node $v_i \in V$ represents one pipeline operation, such as preprocessing, model fitting, prediction, merging, or metric evaluation. Each edge $(v_i, v_j) \in E$ means that node v_j depends on the output of node v_i . To guarantee valid execution, the graph must satisfy the acyclic condition:

$$\nexists (v_1, v_2, \dots, v_k) \text{ such that } v_1 = v_k$$

This condition ensures that the graph contains no circular dependencies. Therefore, the backend can execute the pipeline using a valid topological ordering. If $\tau(G)$ denotes a topological order of the graph, then every dependency must appear before the node that consumes it:

$$u \rightarrow v \Rightarrow \tau(u) < \tau(v)$$

During execution, each node applies its stored operation to the outputs received from its parent nodes. For a node v_i , the output can be represented as:

$$o_i = f_i(o_{p_1}, o_{p_2}, \dots, o_{p_m})$$

where f_i is the operation stored in the node, and $o_{p_1}, o_{p_2}, \dots, o_{p_m}$ are the outputs of its parent nodes. This formulation allows both sequential and branched workflows to be handled by the same execution model.

For branch-based execution, the pipeline may produce several candidate paths. If $P = \{p_1, p_2, \dots, p_n\}$ is the set of candidate paths and $M(p_i)$ is the metric value of path p_i , the selected path can be written as:

$$p^* = \arg \max_{p_i \in P} M(p_i)$$

for maximization metrics such as accuracy, or:

$$p^* = \arg \min_{p_i \in P} M(p_i)$$

for minimization metrics such as loss or error. The selected path p^* is stored inside the `ExecutedGraph`, which keeps only the fitted transformations and models needed for later inference.

At inference time, the executed graph applies only the selected fitted path to new

input data. If X_{new} is the new input, the inference output can be represented as:

$$\hat{y} = G^*(X_{\text{new}})$$

where G^* is the fitted executed graph selected during training. This separation reduces unnecessary training-time objects and keeps inference focused on the final reusable path.

4.3.2 Modular Decomposition

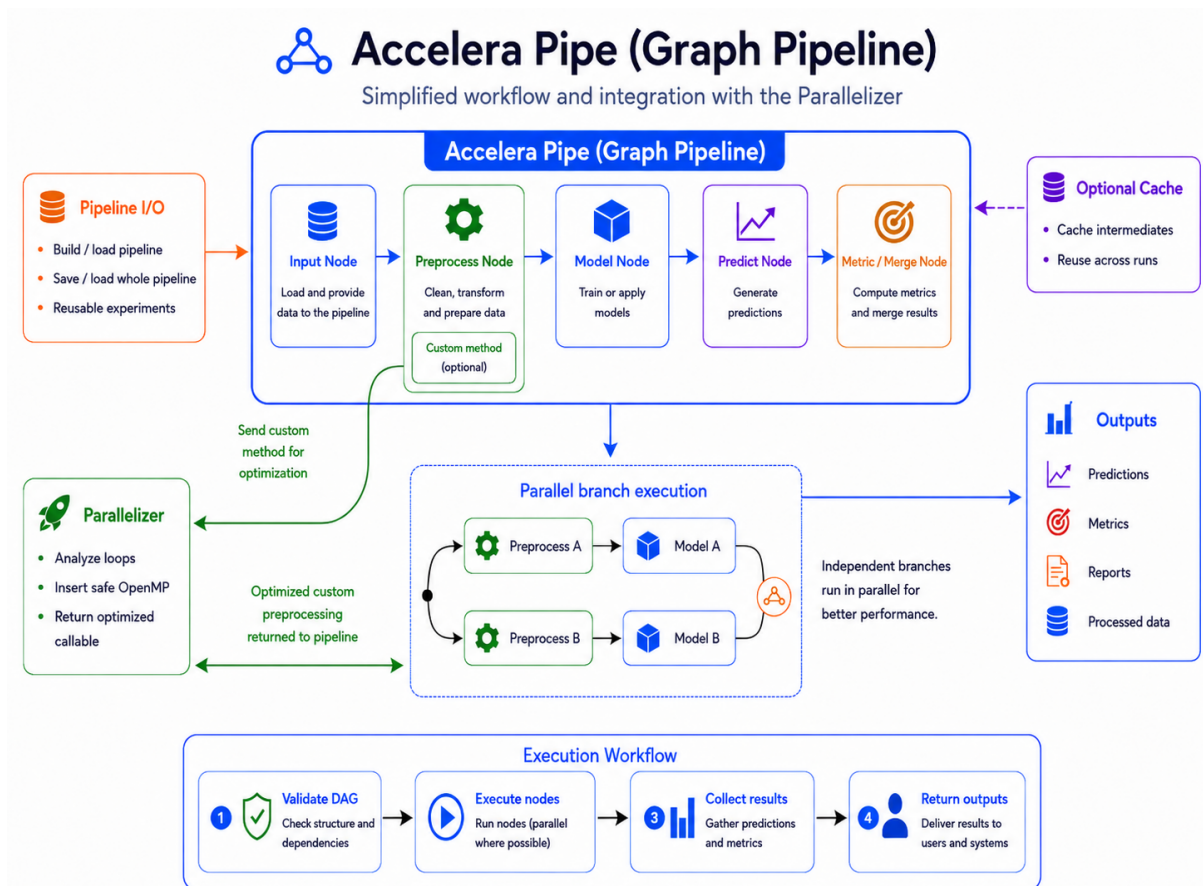


Figure 4.2: Accelera Pipe workflow with custom preprocessing parallelization and pipeline reuse.

The Accelera Pipe module is decomposed into two main layers: a Python API layer and a native C++ graph execution layer. As shown in Figure 4.2, the Python layer provides the user-facing builder interface used to define preprocessing, model, prediction, metric, branch, and merge nodes. It also handles metric wrapping, reports, serialization, save/load support, and integration with the Code Parallelizer.

The native C++ layer manages the internal graph representation and execution behavior. It stores nodes and edges, validates graph connections, computes the topological execution order, schedules independent branches for parallel execution, selects the

final fitted path, and manages intermediate memory. When a preprocessing node contains a supported custom method, Accelera Pipe can send it to the Parallelizer, receive an optimized callable, and attach it back to the graph before execution.

Table 4.6: Main components of the Accelera Pipe module

Component	Responsibility
Pipeline Builder Interface	Provides the user-facing fluent API used to define preprocessing stages, models, prediction steps, metrics, merge operations, and branches. It translates each user call into a typed computational node instead of immediately executing the operation.
Pipeline State and Persistence Manager	Owns the native graph object, maps high-level node categories to graph node types, controls graph execution settings, and provides save/load behavior for reusable pipeline objects.
Executed Graph Runtime	Represents the fitted execution path selected after training. It allows the same trained transformations and models to be reused for inference without keeping unnecessary validation data or temporary training branches.
Metric and Display Wrappers	Adapt supervised metrics, unsupervised metrics, arrays, figures, dictionaries, and reports into graph-compatible outputs that can be stored, selected, displayed, or used in branch-selection strategies.
Native Graph Engine	Stores the pipeline as a directed acyclic graph, validates node connections, computes execution order, schedules sequential or parallel execution, manages branch selection, and controls intermediate-memory lifetime.
Node Abstraction Layer	Defines the common behavior of input, preprocessing, model, prediction, merge, and metric nodes, including source-node relationships, stored data, GPU usage flags, and selected-path markers.

At the Python API level, each builder method creates or configures one graph operation. A preprocessing node stores a function or transformer together with execution parameters such as `execute_fit` and `cache`. Before a preprocessing function is stored, the module attempts to parallelize it through `parallelizer.parallelize`. Model nodes store estimators, prediction nodes store test data and the prediction method name, metric nodes store metric wrappers, and merge nodes store the merge strategy.

Table 4.7: Mapping from builder calls to graph node semantics

Builder method	Node type	Main role
<code>preprocess(name, func)</code>	PREPROCESS	Transform input data or selected columns.
<code>model(name, model)</code>	MODEL	Fit or apply a machine-learning estimator.
<code>predict(name, test_data)</code>	PREDICT	Run a model output function such as <code>predict</code> .
<code>metric(name, metric)</code>	METRIC	Evaluate predictions using a supervised or unsupervised metric.
<code>merge(name, strategy)</code>	MERGE	Combine outputs from multiple prediction branches.
<code>branch(name, ...)</code>	Multiple nodes	Create alternative candidate paths in the DAG.

4.3.3 Branching and Path Selection

The native backend uses a graph abstraction rather than a linear pipeline. The `Graph` class stores all nodes, tracks whether the graph has already been compiled, supports cloning, and exposes methods for adding nodes, splitting branches, executing, clearing, saving, loading, and enabling parallel execution. The graph computes a topological execution order before running. This guarantees that each node is executed only after its source nodes have produced their outputs. If the graph contains branches, the execution result is followed by path selection using one of the supported strategies: maximum metric, minimum metric, custom strategy, or all paths.

Algorithm 4.1: High-level Accelera Pipe training flow

Input: training data `X`, optional labels `y`, graph `G`, selection strategy `S`

Output: predictions or metric results `R`, executed graph `E`

- 1: Add `X` and `y` to the input node of `G`
- 2: Compile `G` if its topological order is not already available
- 3: Execute graph nodes sequentially or in parallel depending on `G` settings
- 4: Collect outputs from final leaf nodes
- 5: If `G` contains branches, select the path requested by `S`
- 6: Clone the selected fitted path into a compact executed graph `E`
- 7: Remove validation-only data and temporary training data
- 8: Return `R` and `E`

Branching is handled by the native `split` operation. Each branch is converted into

a chain of graph nodes starting from the current leaf. If the current leaf node is L , then a branch B_i can be represented as:

$$B_i = (L, n_{i1}, n_{i2}, \dots, n_{ik})$$

where $n_{i1}, n_{i2}, \dots, n_{ik}$ are the operations added to that branch. A branch may contain a single node or a list of nodes, so the same mechanism supports simple model comparison as well as full alternative sub-pipelines.

A useful optimization in branch construction is duplicate first-node sharing. If several branches begin with the same first node, the graph creates that first node once and connects the later branch consumers to its output. The sharing condition is:

$$n_{11} = n_{21} = \dots = n_{m1}$$

where n_{i1} is the first node of branch B_i . This is safe for the first node because all such branches receive the same original input. The optimization is deliberately conservative: later repeated nodes are not merged automatically because their inputs may already differ due to earlier branch operations.

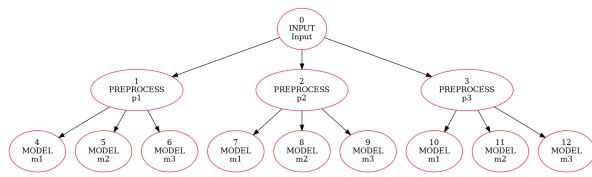


Figure 4.3: Before duplicate first-node sharing

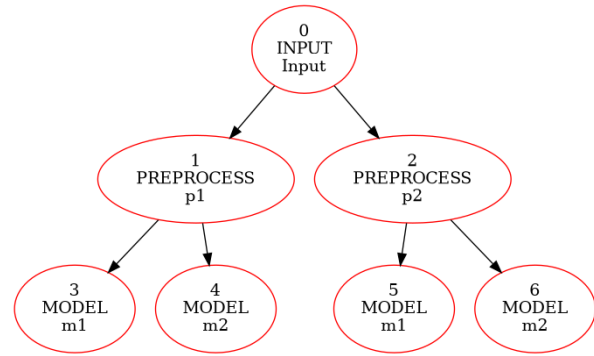


Figure 4.4: After duplicate first-node sharing

4.3.4 Intermediate Memory Management

Machine-learning pipelines can produce large intermediate arrays, especially when several preprocessing branches are evaluated. Accelera Pipe therefore tracks remaining consumers for each node output. Once the last downstream consumer finishes, the corresponding intermediate data can be released. Model and metric nodes are treated more carefully because they may hold fitted estimators or evaluation state, while input and preprocessing data can often be cleared after their consumers are done. This policy reduces memory pressure without changing the logical result of the graph.

 Algorithm 4.2: Consumer-count based intermediate release

Input: executed node N , remaining consumer table C

Output: graph with unused temporary data released

```

1:  for each source node  $S$  that feeds  $N$  do
2:       $C[S] = C[S] - 1$ 
3:      if  $C[S] == 0$  and  $S$  is releasable then
4:          clear the temporary data stored by  $S$ 
5:      end if
6:  end for
  
```

The consumer-count mechanism is used after each node finishes executing. Every source node keeps a counter representing how many downstream nodes still need its output. For a source node S , the remaining-consumer count can be written as:

$$C(S) = |\{N \mid S \rightarrow N \text{ and } N \text{ has not executed yet}\}|$$

When a downstream consumer completes, the counter is decreased:

$$C(S) \leftarrow C(S) - 1$$

If the counter reaches zero and the source node is safe to clear, the temporary output is released. This allows the graph to reduce memory pressure during branch-heavy training runs without changing the final predictions or selected path.

4.3.5 Parallel Graph Execution

The graph can run independent nodes in parallel, but a node can start only after all its parents have finished. For any node N , the readiness condition is:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N), \text{finished}(P) = \text{true}$$

The backend uses CPU thread semaphores, a separate GPU semaphore, and dependency checks over the topological order. This prevents a downstream node from reading incomplete data and also serializes GPU work when necessary. CPU nodes are allowed to execute concurrently as long as worker slots are available, while GPU nodes are handled more conservatively using a single execution slot. This makes the scheduler safe for mixed CPU/GPU graphs without changing the dependency semantics. This also allows the graph to use available hardware more efficiently when several branches are ready at the same time. At the same time, synchronization keeps the final execution result equivalent to a valid sequential topological run. The number of CPU threads is based on hardware concurrency but can be overridden using the `ACCELERA_NUM_THREADS` environment variable.

Algorithm 4.3: Parallel graph execution scheduling

Input: topological execution order T

Output: completed graph execution or reported error

```

1: Initialize finished and started state for all nodes
2: Build remaining-consumer counts for intermediate outputs
3: Create limited CPU worker slots and one GPU execution slot
4: while unfinished nodes still exist do
5:     for each node  $N$  in  $T$  do
6:         if  $N$  has not started and all parent nodes are finished then
7:             Acquire a CPU slot or the GPU slot depending on  $N$ 
8:             Mark  $N$  as started
9:             Launch  $N$  asynchronously
10:            After  $N$  finishes:
11:                Mark  $N$  as finished
12:                Release unused source outputs
13:                Release the CPU or GPU slot
14:            end if
15:        end for
16:    end while
17: Wait for all launched tasks
18: Rethrow any stored node execution error

```

Algorithm 4.3 describes the runtime scheduling idea implemented by the native graph backend. The graph is already arranged in topological order, but the scheduler does not simply execute nodes one by one. Instead, it repeatedly searches for nodes whose parents have finished and launches them asynchronously when a CPU or GPU execution slot is available. CPU nodes share a limited pool of worker slots, while GPU nodes use a separate single-slot semaphore to avoid unsafe concurrent GPU execution. After a node completes, the backend marks it as finished, releases any temporary source outputs that are no longer needed, and signals the scheduler so that newly ready child nodes can start. Any exception raised inside a node is stored and rethrown after all launched tasks are joined, which gives a clear error message containing the failed node name and whether it was executed as a CPU or GPU node.

4.3.6 Design Constraints

The Accelera Pipe module has several design constraints that guide how the graph is constructed, executed, and reused. The first constraint is that the internal workflow must always remain a valid directed acyclic graph. Since execution depends on topological ordering, cyclic dependencies are not allowed. For any edge from node u to

node v , the execution order must satisfy:

$$u \rightarrow v \Rightarrow \tau(u) < \tau(v)$$

where τ represents the topological order of the graph. This guarantees that every node is executed only after its required source nodes have already produced their outputs.

The second constraint is node-type validity. Not every node can be connected to any other node. For example, a prediction node must follow a model node, a metric node must follow a prediction or merge node, and a merge node should combine compatible prediction outputs. These restrictions preserve the semantic meaning of the pipeline and prevent invalid workflows from reaching the execution stage.

The third constraint is safe memory management. Since branch-heavy workflows can produce many temporary arrays, the backend must release intermediate data only when it is no longer needed. For a source node S , the remaining-consumer count is:

$$C(S) = |\{N \mid S \rightarrow N \text{ and } N \text{ has not executed yet}\}|$$

The output of S can be cleared only when $C(S) = 0$ and the node is safe to release. This reduces memory pressure without changing the final selected path or prediction results.

The fourth constraint is parallel execution safety. Independent nodes may run in parallel, but a node can start only after all of its parent nodes have completed:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N), \text{finished}(P) = \text{true}$$

This prevents downstream nodes from reading incomplete data. The backend also uses limited CPU worker slots and a separate GPU slot to avoid unsafe resource sharing during execution.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned into an `ExecutedGraph`, validation inputs are removed, metric targets are cleared, and temporary arrays are released. This keeps the saved inference graph compact and focused only on fitted transformations and models.

4.3.7 Summary

The Accelera Pipe module also supports persistence. The Python `PipelineBase` class saves and loads pipeline objects using pickle, while the C++ graph exposes its state as a Python dictionary containing node information, selected-path flags, source indices, cached data, graph compilation state, and parallel execution configuration. During loading, the graph rebuilds the nodes and reconnects them using the stored source indices, making the native graph serializable.

The module can also export a graph-like representation of the pipeline structure. The graph utility code writes an XML-style network description containing layers for

input, preprocessing, model, prediction, merge, and metric nodes, together with their edges. This representation is useful for debugging, visualization, and understanding the executable graph generated from the high-level pipeline.

In summary, Accelera Pipe is a hybrid Python/C++ pipeline execution system. Python provides the user-facing construction interface, while C++ provides graph semantics, execution scheduling, branch selection, memory-lifetime control, and parallel execution support, keeping the module both efficient and easy to use.

4.4 Code Parallelizer Module

4.4.1 Functional Description

The Code Parallelizer module is responsible for transforming supported loop-heavy source code into optimized C++ code annotated with OpenMP pragmas. It is designed to reduce the execution time of custom preprocessing functions and standalone code snippets by identifying loops that can be safely executed in parallel. The module accepts different input forms: a file path, a source string, a custom Python function, or a custom transformer instance. Regardless of the input form, the scientific goal is the same: normalize the input into a C++ loop-analysis path, detect parallelization opportunities, and return either parallelized source code or a compiled Python-callable native function.

This process can be viewed as a transformation from an original program P into an optimized program P' :

$$P' = \mathcal{T}(P)$$

where $\mathcal{T}$ represents the parallelization pipeline, including source normalization, loop extraction, loop classification, safety validation, and OpenMP pragma insertion. If no safe optimization is found, the transformation keeps the program behavior unchanged by returning the original code or callable.

For Python input, the module first lowers a restricted Python subset into C++. This conversion is intentionally limited to predictable language constructs such as arithmetic expressions, assignments, `for range(...)` loops, conditionals, function definitions, array-style indexing, and simple calls. The resulting C++ code is then analyzed by the same loop extraction path used for native C/C++ input. This design avoids building two separate parallelization pipelines and ensures that both Python and C++ inputs are judged using the same loop features and safety checks.

After normalization, the module extracts candidate loops and checks whether each loop can be parallelized safely. For a loop L_i , the final decision can be represented as:

$$d_i \in \{\text{sequential}, \text{parallel_for}, \text{reduction}\}$$

where sequential means that the loop is left unchanged, `parallel_for` means that a normal OpenMP parallel loop pragma can be inserted, and reduction means that the loop

can be parallelized using an OpenMP reduction clause. This decision is based on both learned classification and rule-based safety checks, so the module does not rely only on a model prediction when program correctness may be affected.

Table 4.8: Supported operations in the Python-to-C++ converter

Category	Supported forms and notes
Constants and names	None, booleans, integers, floats, strings, and variables.
Arithmetic	Addition, subtraction, multiplication, division, modulo, and power using <code>std::pow</code> .
Unary and boolean	Unary plus, unary minus, logical not, logical and, and logical or.
Comparisons	Equality, inequality, less-than, less-than-or-equal, greater-than, and greater-than-or-equal; chained comparisons rejected.
Calls and printing	Ordinary function calls and <code>print</code> ; keyword arguments are generally unsupported.
Access operations	Attribute access and indexing are supported; slicing is rejected.
Assignment	Single-target assignment and indexed assignment are supported; multi-target assignment is rejected.
Augmented assignment	Simple-name <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , and <code>%=</code> .
Control flow	<code>if/else</code> blocks and <code>return</code> statements.
Loops	<code>for i in range(...)</code> loops with one, two, or three range arguments.
Functions	Function definitions are emitted as C++ template-style functions; decorators are rejected.

After normalization, the module extracts loop information using native Clang AST bindings. Each loop is represented by its source code and source-line range. The Python orchestration layer then computes a fixed loop-feature representation, uses a classifier endpoint to predict whether the loop is a parallelization candidate, and validates the decision with rule-based checks. If the loop is safe, the module injects an OpenMP directive such as `#pragma omp parallel for`. For reduction-like loops, it can also emit a reduction clause when the reduction variable is valid.

The module has a fail-safe behavior. If the input cannot be converted, analyzed, classified, compiled, or loaded as a native extension, the system falls back to the original Python function or leaves the code unmodified. This is important because paral-

lelization is an optimization, not a reason to change the semantic meaning of the user’s program.

4.4.2 Modular Decomposition

The Code Parallelizer module is decomposed into six cooperating components: input normalization, Python-to-C++ conversion, Clang-based loop extraction, feature extraction and vectorization, classification with rule-based validation, and OpenMP/native-extension generation.

Table 4.9: Main components of the Code Parallelizer module

Component	Responsibility
Parallelization Orchestrator	Receives source strings, file paths, custom functions, and transformer instances. It coordinates conversion, loop extraction, feature analysis, classification, pragma insertion, caching, compilation, and fallback behavior.
Input Normalization Layer	Converts accepted input forms into a common source-analysis representation. Files are read, source strings are analyzed directly, functions are captured from source, and transformer instances are inspected through their transformation method.
Python-to-C++ Converter	Translates a restricted subset of Python into C++ so Python preprocessing logic can enter the same static loop-analysis pipeline as native C/C++ code.
Loop Extraction Engine	Uses a compiler-style front end to locate loops, record source ranges, and expose structured loop information for feature extraction and rewriting.
Feature Extraction and Embedding Generator	Computes structural, dependency, memory-access, control-flow, arithmetic, and reduction features, then maps them into a fixed-size numeric representation.
Parallelization Classifier and Safety Resolver	Combines classifier predictions with deterministic safety rules to decide whether a loop remains sequential, receives a parallel-for pragma, or receives a reduction pragma.
OpenMP Rewriter	Inserts the selected OpenMP directive while avoiding unsafe duplicate insertion, especially when an outer loop already contains selected inner loops.
Native Compilation and Loading Layer	Wraps optimized C++ into a Python-callable extension, adds OpenMP flags only when needed, builds the native module, loads it at runtime, and returns the optimized callable.
Caching Layer	Stores processed source and compiled native functions using source-based hashes to avoid repeated conversion and compilation.

Figure 4.5 summarizes the overall data flow. In general, supported inputs are normalized into a common C++ loop-analysis path, while callable Python inputs additionally proceed to PyBind11 extension generation.

Accelerera Parallelizer Module Workflow

Efficient loop coverage for C++ + loop-analysis paths;
only callable inputs are true to native Python-module integration.

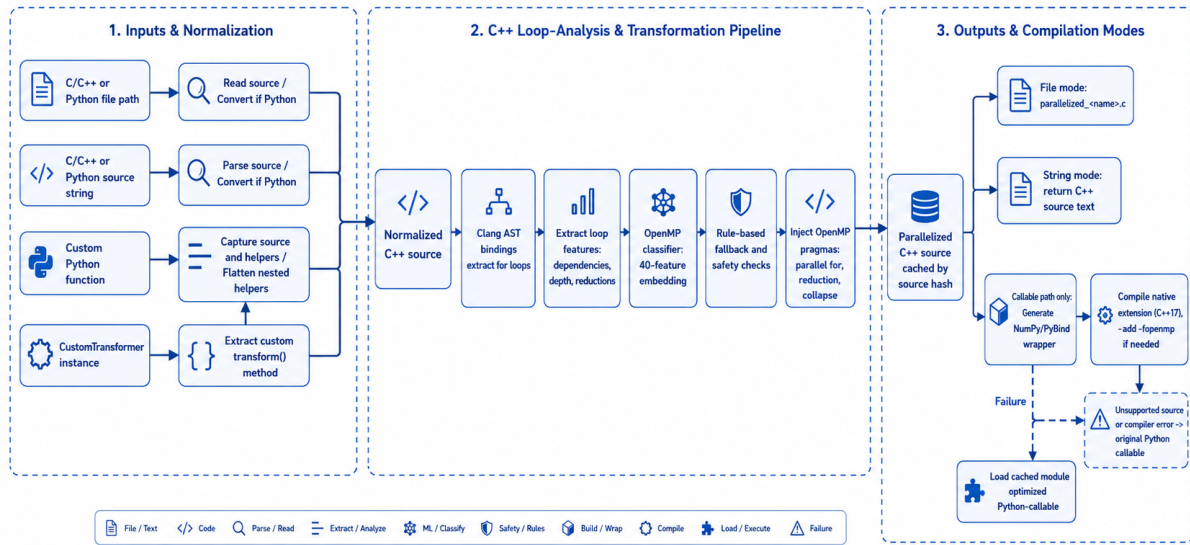


Figure 4.5: Code Parallelizer module workflow

The feature extraction stage analyzes both code structure and loop behavior, including reductions, array accesses, dependencies, indirect memory access, early exits, nesting depth, branches, function calls, arithmetic operations, and vectorization potential.

These indicators are encoded into a 40-dimensional vector for the classifier, then refined by deterministic safety rules that can still detect safe cases such as reductions, independent array writes, and nested loop patterns.

Algorithm 4.4: Loop classification and pragma insertion

Input: source program P

Output: source program P' with safe OpenMP pragmas

```

1:  if  $P$  is Python source then
2:      convert the supported Python subset into C++
3:  end if
4:  Extract for-loops using the Clang AST frontend
5:  for each extracted loop  $L$  do
6:      compute structural, memory, dependency, and reduction features
7:      send the 40-feature vector to the classifier service
8:      combine the predicted class with rule-based safety checks
9:      if the resolved class is parallel_for then
10:         insert #pragma omp parallel for
11:     else if the resolved class is reduction then
12:         insert #pragma omp parallel for reduction(...)

```

```

13:     else
14:         leave the loop unchanged
15:     end if
16: end for
17: Cache and return the transformed source code

```

The compilation path is used when the input is a Python callable. The parallelizer extracts or receives the function source, converts it to C++, parallelizes the C++ loop body, wraps the result in a PyBind11 module, compiles it as a shared native extension, loads the extension, and returns the optimized function. Caching is used at both the transformed-source level and the compiled method level. The cache key includes the compilation cache version, the function name, the compiler optimization level, the extension suffix, and the source code. This prevents recompiling the same function repeatedly.

4.4.3 Design Constraints

The most important design constraint of the Code Parallelizer is correctness. OpenMP pragmas can improve performance, but an incorrect pragma can produce wrong numerical results. Therefore, classifier output is not applied blindly. The module applies additional safety rules to avoid unsafe transformations when there are loop-carried dependencies, indirect accesses, early exits, side-effect function calls, or invalid reduction variables. Inner loops are also skipped when an outer selected loop already covers their range, preventing nested duplicate pragma insertion.

The second constraint is the restricted Python subset. Full Python semantics are too dynamic to translate safely into C++ using a lightweight source converter. The converter therefore rejects unsupported syntax such as decorators, slicing, chained comparisons, unsupported operators, non-range loop iterators, multi-target assignment, complex dynamic dispatch, and unsupported expression forms. This restriction is not a weakness in the design; it is a correctness boundary. The system optimizes code only when it can produce predictable C++.

The third constraint is compiler and platform compatibility. Native callable optimization depends on a working C++ compiler, Python include paths, PyBind11 headers, NumPy-compatible array wrapping, and platform-specific shared extension suffixes. The compiler path uses C++17 and adds `-fopenmp` only when the parallelized code actually contains OpenMP pragmas. On non-Windows platforms it also uses position-independent code flags suitable for shared libraries. If compilation fails, the original Python callable is returned.

The fourth constraint is input and output compatibility. The current compiled callable path is oriented toward functions with one named input parameter and NumPy-like two-dimensional numeric data. The wrapper rewrites array access to PyBind11 unchecked NumPy access and exposes the compiled function back to Python. This makes the generated extension useful for numeric preprocessing loops, but it also means that

arbitrary Python functions with complex object structures are outside the supported optimization target.

The fifth constraint is reproducibility and caching. Because native compilation can be expensive, the module caches processed C++ code and compiled functions using source hashes. The cache must include enough information to avoid stale or incorrect reuse when compiler options, cache version, function name, source code, or extension suffix changes. This design improves repeated execution time while keeping cache invalidation explicit.

4.4.4 Summary

The Code Parallelizer is directly integrated with Accelera Pipe. When a preprocessing node is added, the pipeline calls the parallelizer before storing the preprocessing object. If the object is a custom Python function, it is wrapped as a source-backed function whose runtime callable can be replaced by the compiled implementation. If the object is a custom transformer instance, the parallelizer attempts to optimize its `transform` method and attach the optimized method back to the instance. If the object is a library transformer or an unsupported callable, it is returned unchanged.

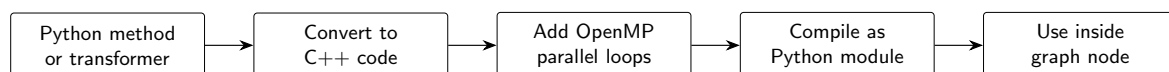


Figure 4.6: Integration flow for compiled preprocessing inside Accelera Pipe

Scientifically, the final design uses a hybrid approach rather than a purely generative code model. An earlier generator-based design attempted to emit parallelized code using a neural sequence model. That approach was rejected because the dataset size was not sufficient for reliable sequence generation and because generated synthetic samples risked introducing distribution bias. The current design is more controlled: the classifier only predicts the type of parallelization opportunity, while deterministic compiler-like rules decide how and whether the pragma is inserted. This separation makes the system easier to inspect, easier to debug, and safer for user code.

Overall, the Code Parallelizer acts as an optimization layer around user-defined numeric loops. It does not attempt to parallelize all Python programs. Instead, it targets the subset where static analysis, feature extraction, classifier prediction, rule validation, and native compilation can cooperate safely. This fits the broader Accelera design philosophy: keep the Python API simple, but move performance-sensitive and structurally analyzable work into optimized native execution when the system can do so without changing program semantics.

4.5 Auto Preprocessing Module

The Auto Preprocessing module automates the data understanding, exploratory data analysis, and preprocessing stages in the data science workflow. It helps reduce the repeated manual work needed to prepare datasets before modeling.

The module also helps the user understand the dataset before and after preprocessing by generating a report. This report shows important information about the input data and the preprocessing steps that were applied. For tabular data, the report shows information such as the number of rows and columns, data types, missing values, duplicate records, numerical and categorical features, basic statistics and EDA. For text data, it shows information such as the number of rows, target classes, class distribution, and number of missing. For image data, it shows information such as the number of images, image classes, class distribution, image sizes, and corrupted images. After preprocessing, the report shows the steps applied to the data, such as imputation, encoding, scaling, feature selection, text cleaning, TF-IDF feature extraction, image resizing, normalization, and image preparation for classification.

Another important design point is separating the training phase from the inference phase. In the training phase, the preprocessing objects are fitted on the training data and saved as artifacts. In the inference phase, these saved artifacts are loaded and applied to new data. This ensures that new data is processed in the same way as the training data before it is passed to the trained model for prediction.

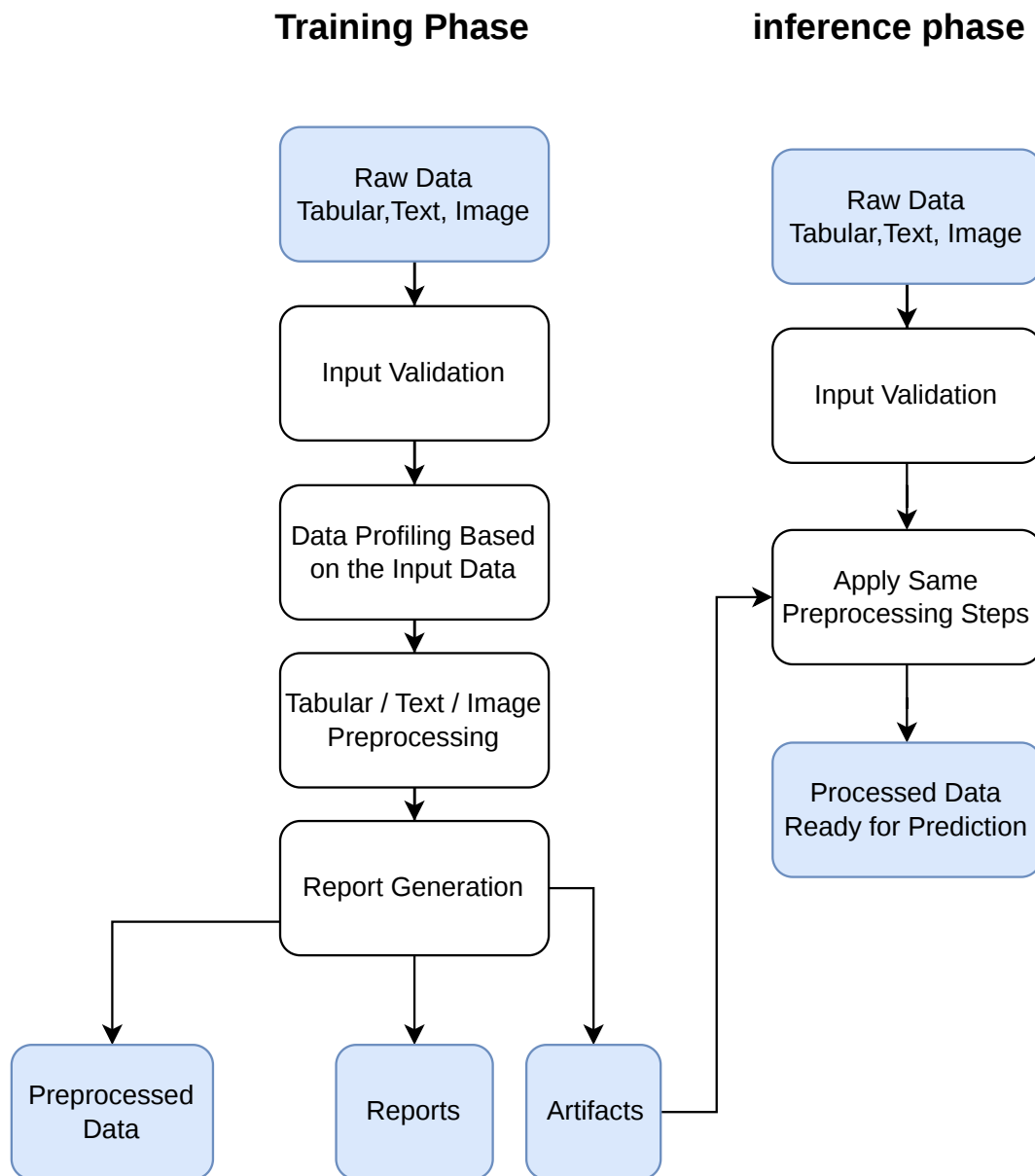


Figure 4.7: Auto Preprocessing Workflow

4.5.1 Functional Description

This module receives a dataset with its configuration, applies the suitable preprocessing workflow based on the data type and task, and returns processed data ready for training or inference. The module supports tabular, text, and image datasets. The module also generates a report when this option is enabled. The report summarizes information about the dataset and the preprocessing steps applied.

4.5.1.1 Tabular datasets

For tabular datasets it supports both classification and regression tasks. It analyzes the dataset, removes duplicate records, splits the data into training and validation sets, detects column types, handles missing values, encodes categorical features, scales numerical features, applies target preprocessing, performs feature selection, and saves the preprocessing objects needed later for test data.

4.5.1.2 Text datasets

It supports classification tasks. It cleans the text, tokenizes it, applies text preprocessing steps, and converts the text into a numerical representation suitable for machine learning models.

4.5.1.3 Image datasets

It supports image classification by reading images from class folders, validating image files, resizing images, applying augmentation, normalizing pixel values, and preparing the data loaders needed for training.

4.5.2 Modular Decomposition

The Auto Preprocessing module is divided based on the type of input data. Each data type has its own preprocessing workflow because every type has a different structure and needs different techniques:

- **Tabular data preprocessing:** handles structured datasets made of rows and columns.
- **Text data preprocessing:** handles text columns and converts cleaned text into numerical features.
- **Image data preprocessing:** handles image classification folders and prepares image loaders for training.

4.5.2.1 Tabular Data Preprocessing Functions

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process.
- **handel_bool_types():** This function handles columns that have bool data type. It finds these columns and converts them into integer values.
- **data_overview():** This function gives a general overview of the dataset before preprocessing.
- **drop_target_nulls():** This function drops rows have null value in the target.

- **drop_duplicates():** This function removes repeated rows from the dataset.
- **split_data():** This function splits the dataset into training and validation sets. The split is based on the validation size and random state selected by the user. The output of this function is `X_train`, `X_val`, `y_train`, and `y_val`.
- **get_data_info():** This function studies each column in the training data. It collects important information such as data type, number of unique values, missing values, and missing percentage. For numerical columns, it also calculates the mean, and median.
- **is_drop_column():** This function decides whether a column should be removed or kept. A column is removed if it is included in the user selected drop list, if it has only one constant value, or if its missing value percentage is higher than the selected threshold.
- **drop_col():** This function removes the selected columns from both the training and validation datasets. It also saves the dropped columns and their reasons in a file. This helps the system apply the same dropping step later on testing or new data.
- **detect_column_types():** This function detects the type of each feature. This step is important because each feature type needs a different preprocessing method. The features are divided into binary columns, ordinal columns, numerical columns, continuous columns, low cardinality categorical columns, high cardinality categorical columns, and unsupported columns. The function also stores the preprocessing steps for each column to be shown later in the report.
- **make_graphs():** This function creates graphs when the report option is enabled. The type of graph depends on the feature type and the problem type.

Table 4.10: Detected Column Types in Tabular Data

Feature Type	Description	Examples
Binary	Columns containing exactly two unique values.	yes/no, 1/0
Ordinal	Integer columns with a small number of unique values less than or equal to <code>max_ordinal_unique</code> .	Ratings or levels such as 1, 2, 3, 4
Numerical	Integer columns with a number of unique values greater than <code>max_ordinal_unique</code> .	Age, number of transactions
Continuous	Floating point columns.	Height = 175.5, price = 19.99, temperature = 36.6
Low Cardinality Categorical	Object or string columns with a number of unique values less than or equal to <code>cardinality_threshold</code> .	Product categories, country group
High Cardinality Categorical	Object or string columns with a number of unique values greater than <code>cardinality_threshold</code> .	Product names, cities
Other	Columns with unsupported or non standard data types. These columns are excluded from the preprocessing pipeline.	Dates, complex objects, unsupported formats

Table 4.11: EDA graphs generated for tabular preprocessing

Column type	Problem type	Generated graphs
Binary and categorical	Classification	Null vs non null pie chart, category count plot, and category count split by target class.
Binary and categorical	Regression	Null vs non null pie chart, category count plot, and target box plot for each category.
Ordinal	Classification	Null vs non null pie chart, ordered count plot, and ordered count plot split by target class.
Ordinal	Regression	Null vs non null pie chart, ordered count plot, and target box plot for each ordinal value.
Numerical and continuous	Classification	Null vs non null pie chart, histogram with KDE, and feature box plot grouped by class.
Numerical and continuous	Regression	Null vs non null pie chart, histogram with KDE, and scatter plot between feature and target.
Target column	Classification	Null vs non null pie chart and target class count plot.
Target column	Regression	Null vs non null pie chart, target histogram with KDE, and target box plot.
Numeric columns	Both	Correlation heatmap.

- **features_preprocessing():** This function applies the preprocessing steps to the input features. It creates different pipelines based on the detected feature types:
 - Numerical and continuous features are filled using the median and scaled using robust scaling.
 - Binary features are filled using the most frequent value and encoded using ordinal encoding.
 - Low cardinality categorical features are filled using the most frequent value and encoded using binary encoding.
 - High cardinality categorical features are filled using the most frequent value and encoded using frequency encoding.
 - Ordinal features are filled using the most frequent value and scaled using robust scaling.

These pipelines are combined using `ColumnTransformer`. The transformer is fitted on the training data and then applied to the validation data. The fitted preprocessor and the generated feature names are saved for later use.

- **target_preprocessing():** This function prepares the target column based on the problem type. For classification label encoding is used to convert class labels into numbers. For regression robust scaling is applied. The target preprocessor is saved so the same transformation can be used later.
- **features_importance():** This function selects the most useful features using mutual information with the target. Only features with scores greater than or equal to the selected threshold are kept. The selected features are saved in a file to use it in the inference step.
- **common_preprocessing():** This is the main function that controls the full tabular preprocessing workflow. It calls the previous functions in order. It starts by handling boolean columns, generating the data overview, drop rows have null target, removing duplicates, and splitting the data. Then, it extracts column information, drops unsuitable columns, detects column types, generates graphs, preprocesses the features, preprocesses the target, applies feature selection, and generates the report if enabled. It returns the processed training and validation data.

 Algorithm 4.5: General tabular preprocessing flow

Input: DataFrame D, target column y, problem type, other configurations

Output: X_train, X_val, y_train, y_val

- 1: Validate configuration
 - 2: Store original dataset columns
 - 3: Build data overview and report information
 - 4: Remove rows that have null target values
 - 5: Remove duplicate rows
 - 6: Split into training and validation sets
 - 7: Drop user-selected, high-missing, constant, and unsupported columns
 - 8: Detect column types from training data
 - 9: Generate feature and target graphs when report is enabled
 - 10: Fit feature preprocessing objects on X_train
 - 11: Transform X_train and X_val
 - 12: Fit target preprocessing object on y_train
 - 13: Transform y_train and y_val
 - 14: Select useful features using mutual information
 - 15: Save fitted objects and metadata
 - 16: Generate report if enabled
-

Figure 4.8 shows the full tabular preprocessing workflow. It starts from the input dataset, applies data cleaning and feature preprocessing steps, then saves the processed training and validation data with the preprocessing objects needed later for testing.

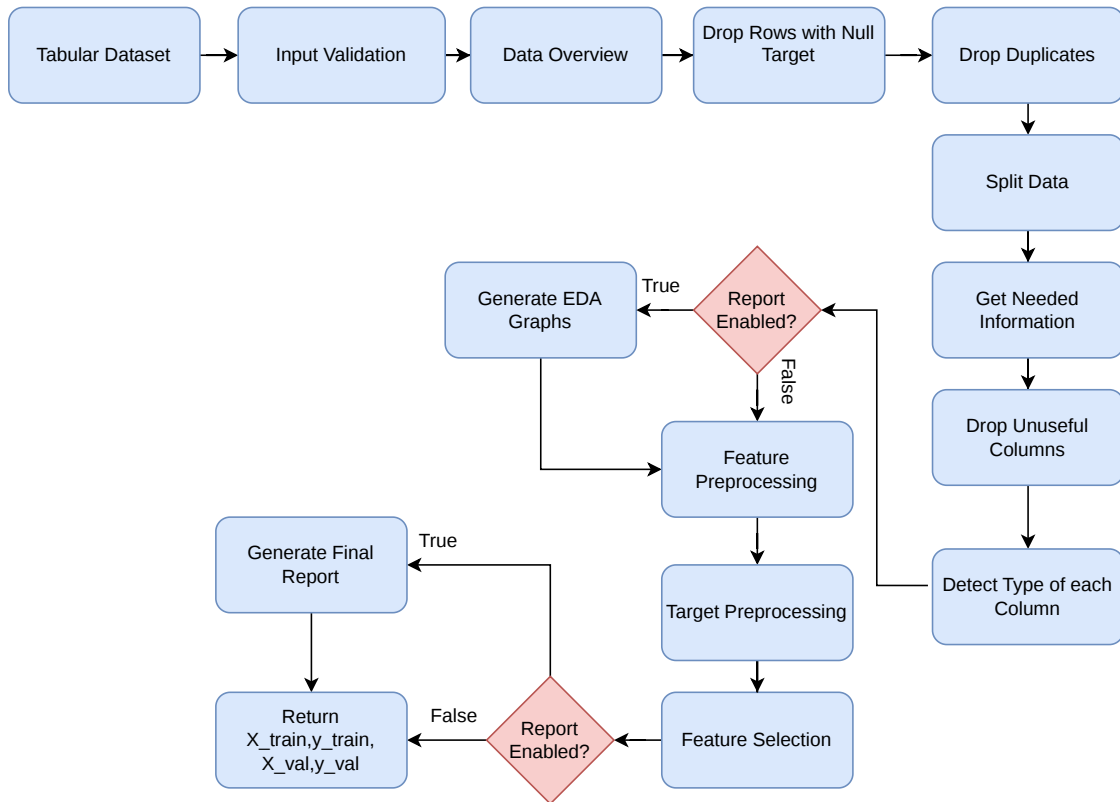


Figure 4.8: Tabular Preprocessing Workflow

4.5.2.2 Text Data Preprocessing Functions

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process. Also removes any columns that are not the text column or the target column. This keeps only the columns needed for text classification. It also downloads the required NLTK resources, such as `punkt` and `stopwords`.
- **make_graphs():** This function creates graphs if the report option is enabled. It creates a graph for the text column. It also creates a graph for the target column. These graphs help the user understand the text data and the target class distribution.
- **custom_text_tokenizer():** This function is used to clean the text and split it into words. It converts the text to lowercase, removes extra spaces and unnecessary symbols, and handles some contractions such as `can't` and `won't`. It also separates numbers from letters. Then, the text is tokenized into words. Stopwords are removed, but words like `not` and `no` are kept because they affect the meaning. After that, stemming is used to reduce words to their basic form.
- **features_preprocessing():** This function applies the preprocessing steps to the text column. It fills missing text values in the training and validation data with

empty strings. Then it uses the custom tokenizer . It also stores examples of the text before and after cleaning in the report. Then `TfidfVectorizer` is used to convert the cleaned text into numerical features. The vectorizer uses the user settings, such as the maximum number of features, n-gram range, maximum document frequency, and minimum document frequency. The vectorizer is fitted on the training data and applied to the validation data. The fitted vectorizer is saved so the same text transformation can be reused later on testing or new data.

- **target_preprocessing():** This function prepares the target column for classification this function applies label encoding to the target values. The fitted label encoder is saved. Also, the text column and target column information are saved . This helps the system apply the same target transformation later on testing or new data.
- **common_preprocessing():** This is the main function that controls the full text preprocessing workflow. It calls the other functions in the correct order. It creates the data overview. Then, it removes rows that have missing target values and removes duplicate rows. Then, it splits the dataset into training and validation sets. If the report option is enabled, graphs are created. Then, feature preprocessing is applied to clean and vectorize the text. Target preprocessing is applied to encode the class labels. Generates the report if enabled. It returns the processed training and validation data.

Algorithm 4.6: Text tokenizer flow

Input: raw text sentence

Output: cleaned token list

- 1: Convert text to lowercase and remove extra spaces
 - 2: Normalize apostrophe characters
 - 3: Expand common negative contractions
 - 4: Add spaces between letters and digits
 - 5: Remove characters except letters, digits, and apostrophes
 - 6: Split text into words
 - 7: Remove stopwords except "not" and "no"
 - 8: Remove very short tokens
 - 9: Apply Porter stemming
 - 10: Return cleaned tokens
-

Figure 4.9 summarizes the text preprocessing workflow. It shows how the text data is cleaned, tokenized, and converted into TF-IDF features before the final processed data is returned.

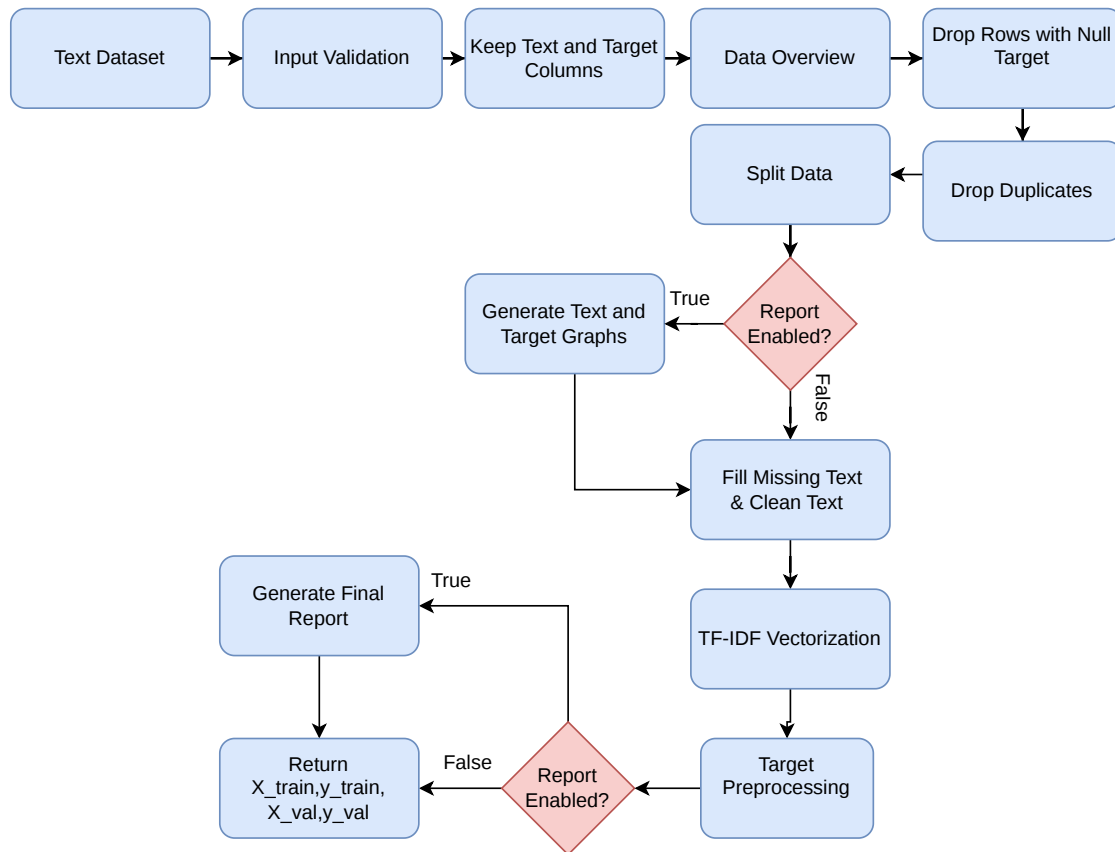


Figure 4.9: Text Preprocessing Workflow

4.5.2.3 Image Data Preprocessing

It is used for image classification datasets. In this module, the images are organized inside folders, where each folder represents one class. It is divided into a set of functions, and each function has a specific role in the image preprocessing workflow. The main functions are described as follows:

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process. It also prepares empty lists to store invalid images and their labels. It saves the image size information in a file so it can be reused later.
- **get_classes_mapping():** This function creates the class mappings. It gives each class name a numerical label. It creates two mappings: one from class name to label, and another from label to class name. These mappings are saved in files. This helps the system use the same labels during training, validation, testing, and prediction.
- **data_preparing():** This function reads the images from the class folders. It goes through each class folder and checks each image path. If the image is valid and

can be opened, its path is added to the list of valid image paths, and its class label is added to the labels list. If the image is invalid or corrupted, it is added to the invalid images list with its label. If there are no valid images, the function raises an error because the preprocessing process cannot continue without valid image data.

- **data_overview():** This function collects general information about the image dataset. For the training folder, it stores the class names, number of valid images, random sample images, number of invalid images, invalid image paths, and the class mapping. If a validation folder is provided, it also stores the same type of information for the validation folder. This information is used later in the final preprocessing report.

- **make_graphs_label_summary():** This function creates graphs that show the label distribution. It creates a label summary graph for the training folder. If a validation folder is provided, it also creates a label summary graph for the validation folder.

If the training data is split into training and validation sets, the function also creates graphs for the labels after splitting. These graphs help the user understand if the classes are balanced or not.

- **make_garphs_sample():** This function creates sample image graphs. It shows random samples from the training folder. If a validation folder is provided, it also shows random samples from the validation folder.

If the training data is split into training and validation sets, it also shows sample images after splitting. This helps the user visually check the image dataset.

- **make_graphs_loader():** This function creates sample images after using the DataLoader. It takes a small number of images from the training DataLoader and displays them.

If a validation DataLoader exists, it also displays sample images from it. This step helps confirm that the images are loaded correctly after resizing, normalization, and augmentation.

- **make_graphs():** This function controls all graph generation steps. It prepares the graph section in the report, then calls the label summary graphs, sample image graphs, and DataLoader sample graphs.

- **get_loaders():** This function creates the training and validation datasets and DataLoaders. First, it creates a `ClassificationImageDataset` for the training images using the image paths, labels, image size, and augmentation settings.

Then, it creates the training DataLoader with the selected batch size and uses shuffling. Shuffling is used for training because it helps the model see the data in a different order.

If validation data exists, the function creates a validation dataset and validation DataLoader. Augmentation is not applied to the validation data, and shuffling is not used because validation should be more stable.

- **common_preprocessing():** This is the main function that controls the full image preprocessing workflow. It starts by creating the class mappings. Then, it prepares the training images by collecting valid image paths and labels and saving invalid images.

If a validation folder is provided, it also prepares the validation images. After that, it creates the data overview, splits the data if needed, creates the DataLoaders, generates graphs, and creates the final image preprocessing report.

At the end, the function returns the training DataLoader and validation DataLoader. These outputs are ready to be used for training an image classification model.

Algorithm 4.7: Image training preprocessing flow

Input: training image folder, optional validation folder

Output: training and validation DataLoaders

- 1: Read class folders from the training directory
 - 2: Create class-to-label and label-to-class mappings
 - 3: Save mappings and image size information
 - 4: Validate image files and skip invalid images
 - 5: Build the data overview for the report
 - 6: Split training images if no validation folder is provided
 - 7: Create training dataset with resize and optional augmentation
 - 8: Create validation dataset without augmentation
 - 9: Build DataLoaders
 - 10: Generate label, sample, and loader graphs
 - 11: Generate the image preprocessing report
-

Figure 4.10 explains the image preprocessing workflow. It shows how image folders are read, invalid images are detected, class labels are created, images are resized and normalized, and final DataLoaders are prepared.

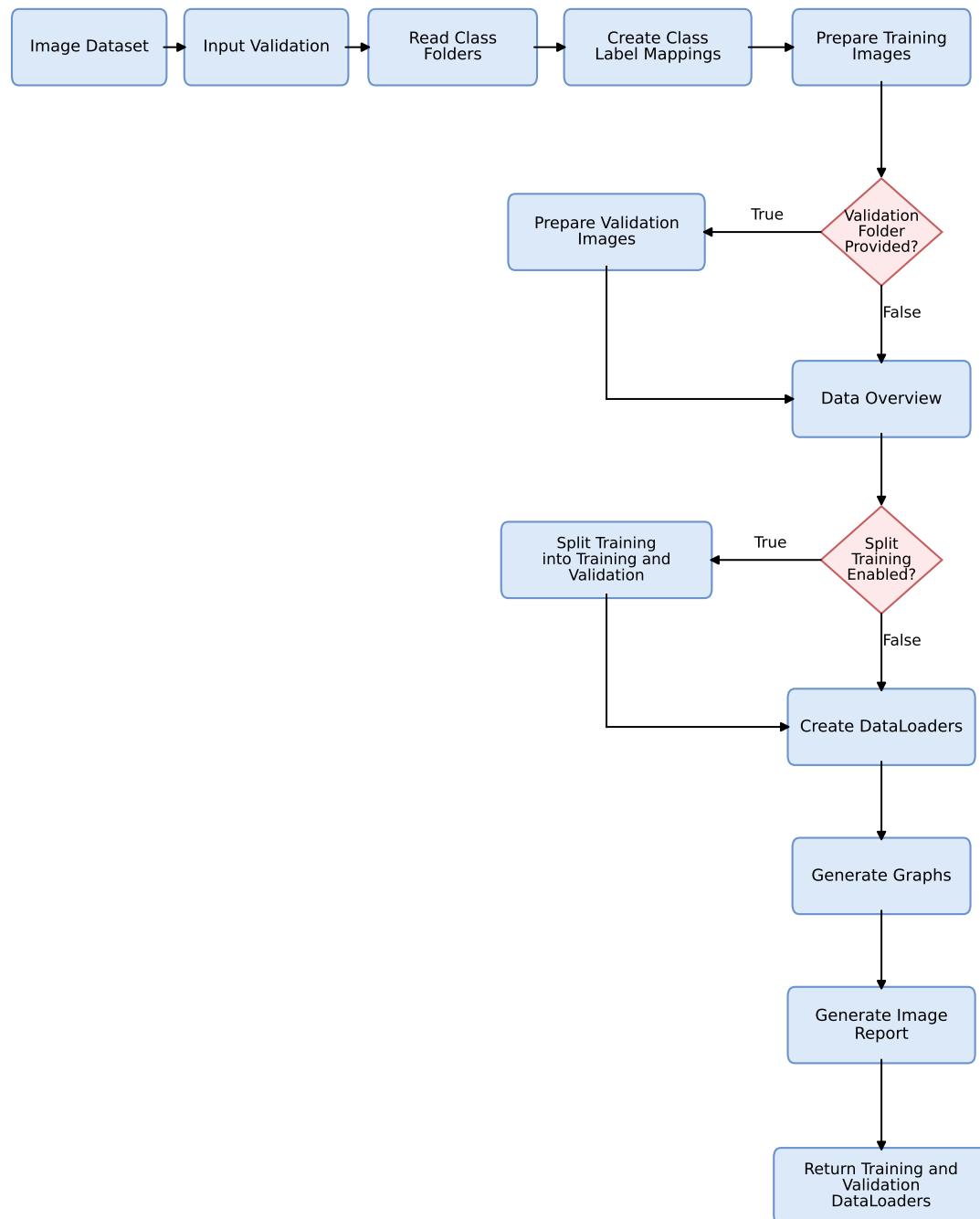


Figure 4.10: Image Preprocessing Workflow

4.5.2.4 Inference and Reusability:

For all supported data types, the saved preprocessing objects are reused during testing. This is important because the testing data must be processed in the same way as the training data.

- For tabular data, the system loads the saved preprocessors, selected features, dropped columns, and target preprocessor.
- For text data, the system loads the saved TF-IDF vectorizer and label encoder.
- For image data, the system loads the saved class mappings and image size.

This makes the preprocessing module reusable and consistent for training, validation, testing, and future prediction.

4.5.3 Design Constraints

The Auto Preprocessing module was designed under the following constraints:

- **Supported data types constraint:** The module is limited to the supported data types only, which are tabular, text, and image data. Other data types are not handled by this module.
- **Input structure constraint:** Each supported data type must follow the expected structure. Tabular and text data should be provided as structured datasets with a target column, while image data should be organized in folders based on class names.
- **Automatic decision constraint:** The module depends on rule-based preprocessing decisions. Therefore, the preprocessing choices are limited to the implemented rules and supported configuration options.

4.5.4 Module Summary

The Auto Preprocessing module provides one reusable preprocessing interface for tabular, text, and image data. It prepares data for training, generates reports when enabled, saves fitted preprocessing artifacts, and reuses the same artifacts during testing and prediction.

4.6 AutoML Module

4.6.1 Module Responsibility

The AutoML module automates model-family selection, hyperparameter search, preprocessing selection, and final model construction for supervised tabular classification

and regression. It exposes two Scikit-learn-style estimators: `AutoMLClassifier` and `AutoMLRegressor`. Both estimators support `fit`, `predict`, `score`, leaderboard inspection, configured scoring metrics, cross-validation folds, random seeds, time budgets, trial limits, meta-learning warm starts, and optional ensemble construction.

The main problem solved by this module is to produce a strong model under explicit time and trial constraints while avoiding unnecessary full-cost evaluations. The design separates public estimator behavior, task-specific evaluation, optimization, meta-learning, and ensembling so that each part can be extended independently.

4.6.2 Functional Description

The AutoML workflow starts by validating the input feature matrix X and target vector y . The estimator builds a task-specific engine, constructs a conditional configuration space, loads optional meta-learning warm starts, and evaluates candidate configurations through a staged optimizer. Each candidate contains a model family, active hyperparameters, and a preprocessing choice.

Every trial is evaluated using cross-validation and converted to an optimizer cost. For classification, accuracy is the default score and the cost is $1 - \text{accuracy}$. For regression, R^2 is the default score and the cost is the negative score because the optimizer minimizes cost. Failed trials are recorded in the run history instead of terminating the complete search.

4.6.3 Configuration Space Construction

The search space is conditional. The active hyperparameters depend on the selected model family, so random forest parameters are active only when `random_forest` is selected, SVM parameters are active only for SVM models, and so on. The implementation stores model definitions in task-specific component registries. Each component defines its public name, hyperparameters, conditions, and estimator factory.

The engine can also filter models according to dataset properties. For example, some expensive models can be disabled for large datasets, and models that require non-negative features can be skipped when negative input values are detected.

4.6.4 Three-Stage Fidelity Schedule

To avoid spending full resources on weak candidates, the optimizer evaluates configurations through three fidelity stages:

$$f = (s, q, k, b),$$

where s is the stage index, q is the sample fraction, k is the number of cross-validation folds, and b is a multiplicative model budget. The current schedule uses:

- stage 0: 20% of samples, 2 folds, and 25% model budget;

- stage 1: 50% of samples, 3 folds, and 60% model budget;
- stage 2: full samples, configured folds, and full model budget.

Promising candidates are promoted to higher stages. This makes the search more resource-aware than evaluating every sampled configuration at full cost.

4.6.5 Surrogate-Guided Candidate Selection

After initial trials are collected, the optimizer vectorizes configurations and augments each vector with its fidelity variables. A random-forest regressor is trained as a surrogate model over observed costs. Predictions from individual trees provide an empirical mean and standard deviation. The optimizer ranks a sampled candidate pool using expected improvement:

$$EI(x) = (\mu^* - \mu(x) - \xi)\Phi(z) + \sigma(x)\phi(z), \quad z = \frac{\mu^* - \mu(x) - \xi}{\sigma(x)},$$

where μ^* is the best observed cost, $\mu(x)$ and $\sigma(x)$ are the surrogate mean and uncertainty for candidate x , and Φ and ϕ are the normal cumulative distribution and density functions. This balances exploring uncertain regions with exploiting candidates predicted to improve the current best result.

4.6.6 Meta-Learning Warm Starts

The module computes basic metafeatures for the input dataset. Shared metafeatures include the number of instances and features, numeric and categorical feature ratios, missing-value ratios, skewness, kurtosis, and categorical symbol statistics. Classification adds class entropy and class probability statistics. Regression adds target mean, standard deviation, skewness, and kurtosis.

Packaged JSON metadata stores historical dataset descriptors and candidate configurations. At runtime, similar datasets are ranked by distance in the metafeature space, and the best historical configurations are converted into the current configuration space as warm-start candidates.

4.6.7 Parallel Execution and Timeout Control

The optimizer supports parallel candidate evaluation and per-trial timeout control. When timeout control is enabled, trials can be isolated in separate processes so that a slow or hanging estimator does not stop the parent search. The engine also passes internal job-count settings to estimators and ensembles to limit nested parallelism.

4.6.8 Final Model and Ensemble Selection

The strongest configuration is rebuilt and fitted on all available training data. If ensembles are disabled, this model is returned directly. If ensembles are enabled, successful

candidates from the run history are considered for voting or stacking.

For stacking, the module builds bagged base estimators and generates out-of-fold prediction blocks. Forward selection starts with the strongest single base model and adds candidates while they improve the meta-model score or until the minimum base-model requirement is satisfied. Classification uses a logistic-regression meta-model over probability blocks. Regression uses a ridge-regression meta-model over numeric prediction columns. The ensemble is added to the leaderboard and becomes the final model when its score is stronger than the best individual model.

4.6.9 Modular Decomposition

Table 4.12 summarizes the main implementation units of the AutoML module. The table helps connect the high-level AutoML workflow described above with the concrete files that implement estimator lifecycle management, configuration-space construction, trial evaluation, optimization, meta-learning, and ensemble construction.

Table 4.12: AutoML implementation units

File or directory	Responsibility
<code>base.py</code>	Shared estimator lifecycle, validation, fitted state, leaderboard, and warning handling.
<code>classification.py</code>	Classification defaults and engine construction.
<code>regression.py</code>	Regression defaults and engine construction.
<code>core/automl.py</code>	Search orchestration, final model fitting, model filtering, and ensemble selection.
<code>components/</code>	Model registries, hyperparameter definitions, conditions, and estimator factories.
<code>configspace_search_space.py</code>	Task-specific configuration-space assembly and configuration conversion.
<code>evaluation.py</code>	Fidelity-aware estimator construction, preprocessing comparison, cross-validation, and cost conversion.
<code>optimization/smac_optimizer.py</code>	Trial scheduling, surrogate modelling, expected improvement, promotions, parallel workers, and timeouts.
<code>meta_learning/</code>	Metafeature extraction, similarity ranking, and warm-start retrieval.
<code>stacked_ensemble.py</code>	Classification adapters, bagging, forward selection, and stacked classification.
<code>stacked_ensemble_regression.py</code>	Regression stacking and forward selection.
<code>meta_learning_data/</code>	Packaged historical classification and regression meta-data.

4.6.10 Design Constraints

- The current module targets supervised tabular classification and regression.
- Inputs must be compatible with Scikit-learn validation utilities.
- Optional libraries such as XGBoost, LightGBM, and CatBoost are used only when available in the environment.
- Trial failures and estimator exceptions must be recorded rather than crashing the full search.
- Search should remain reproducible when a random state is provided.
- Warm-start metadata must be stored relative to the package so it works after installation.

4.6.11 User-Facing Example

Algorithm 4.8: Using Accelera AutoML for classification

```
from accelera.src.accelera_automl import AutoMLClassifier

model = AutoMLClassifier(
    time_budget=300,
    n_trials=20,
    cv=3,
    random_state=42,
    use_meta_learning=True,
    use_ensemble=True,
    ensemble_strategy="stacked",
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
print(model.score(X_test, y_test))
print(model.return_leaderboard(top_n=3))
```

4.7 Deployment Module

4.7.1 Module Responsibility

The Deployment module is the part of Accelera that turns a trained machine-learning result into a usable software service. Without deployment, the output of a machine-learning workflow remains a local artifact, such as a `.pkl` file or a saved preprocessing object. These artifacts may be technically valuable, but they cannot serve real users until they are loaded by a backend application, validated against expected input formats, exposed through an API, and executed in an environment that can be reached by clients.

The module therefore completes Accelera's end-to-end workflow. It receives deployment inputs from the workspace, including `config.json` and model pickle files under `models/`. It can snapshot these files using a lightweight model VCS, prepare a FastAPI serving application, package the service inside Docker, and execute deployment workflows for local Docker, Heroku, and AWS EC2. At runtime, the generated service validates incoming requests using Great Expectations, executes model inference through `ModelService`, and records prediction telemetry through the request tracker.

Figure 4.11 summarizes this responsibility. The figure is important because it shows the Deployment module as a complete bridge between saved workspace artifacts and running deployment targets. The upper part of the flow represents local inputs and version control, the middle part represents service generation and serving behavior, and the lower part represents the targets on which the service can run.

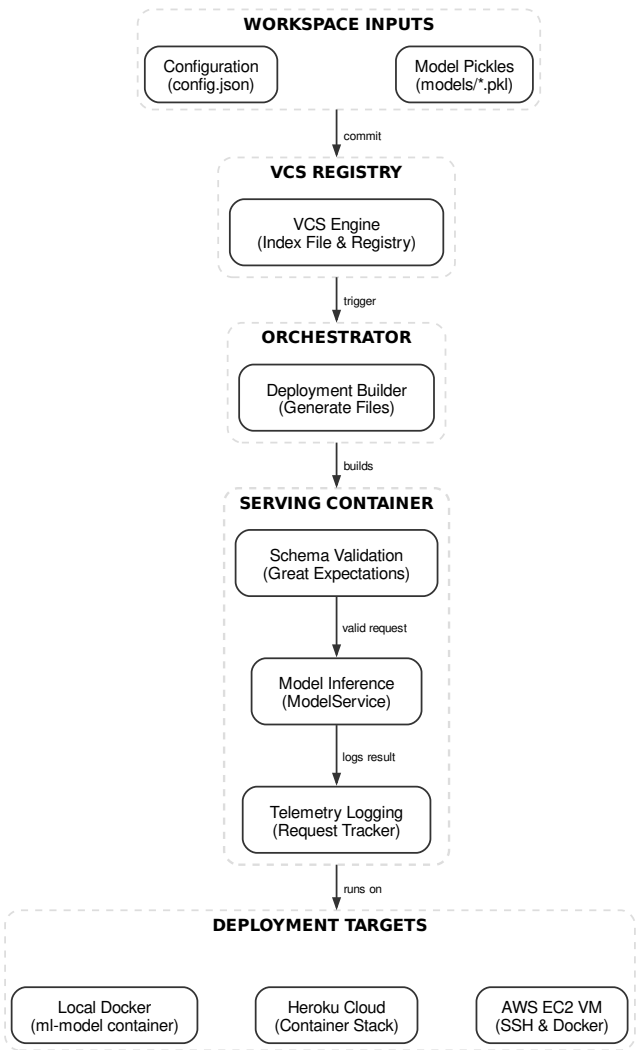


Figure 4.11: Deployment module overview from workspace artifacts to serving targets.

The core files under `accelera/src/deployment/` are listed in Table 4.13. This table

connects the conceptual architecture with the actual source files in the repository and shows how each file owns a clear part of the deployment workflow.

Table 4.13: Deployment module implementation units.

File	Responsibility
<code>server.py</code>	Defines the FastAPI application, prediction endpoints, CSV inference endpoint, health endpoint, tracking summary endpoint, and interactive GUI route.
<code>modelservice.py</code>	Loads <code>config.json</code> , deserializes model and preprocessor artifacts, runs preprocessing transforms, and calls the final estimator's prediction method.
<code>schema_validation.py</code>	Validates request columns, required values, allowed values, numeric bounds, and type coercion using Great Expectations and local validation logic.
<code>tracking.py</code>	Records prediction activity, request status, row counts, latencies, and tracking summaries in line-separated JSON records.
<code>deployment.py</code>	Prepares deployment files, writes requirements, generates the Dockerfile, builds and runs local containers, and orchestrates Heroku and EC2 deployments.
<code>vcs.py</code>	Creates deployment commits, stores model/configuration snapshots, records the active HEAD, and restores historical snapshots for rollback.

4.7.2 Service Architecture

The serving subsystem is built around FastAPI, Uvicorn, `ModelService`, schema validation, and telemetry tracking. FastAPI receives HTTP requests, `ModelService` loads the configured model artifacts, the validation layer checks input quality before inference, and the tracker records each request outcome. This design gives the deployed model a clear runtime boundary: clients interact with API endpoints, while the model itself remains isolated behind validation and service logic.

Before presenting the internal stages, Figure 4.12 shows the serving architecture used inside the container. The diagram explains how requests flow from the HTTP layer to validation, model inference, and tracking. This separation is important because it prevents the model from being called directly with untrusted or malformed data.

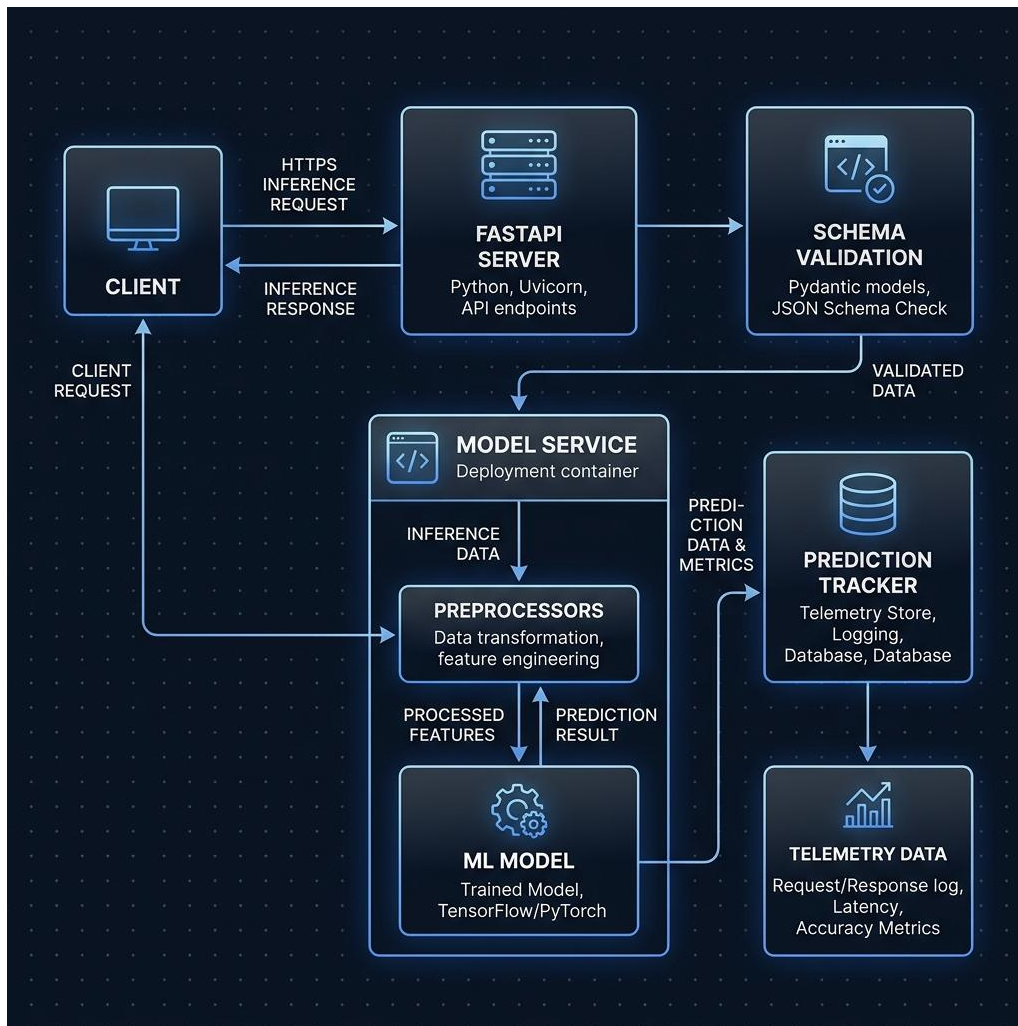


Figure 4.12: Model serving architecture and execution flow.

The service exposes several routes. The `/predict` endpoint accepts JSON payloads, while `/predict/csv` accepts uploaded CSV files for batch prediction. The `/health` endpoint reports whether the service is loaded, which is useful for local checks and remote deployment verification. The `/tracking/summary` endpoint exposes basic request-tracking information, and `/gui` renders an interactive form generated from the configured schema.

4.7.3 Configuration and Artifact Preparation

The deployment process begins from two concrete workspace inputs: `config.json` and model artifacts. The configuration file describes the models to load, the validation schema, and the tracking configuration. The model files are serialized Python objects, typically Scikit-learn estimators, preprocessors, or feature selectors stored as `.pkl` files. The module does not assume one fixed estimator type; instead, `ModelService` loads configured artifacts and treats the object with a `predict` method as the final model, while objects with `transform` methods are treated as preprocessors.

Algorithm 4.9 describes the preparation workflow. This algorithm matters because

deployment should not begin until the module confirms that every configured model path exists and that all runtime files are available inside the deployment workspace.

Algorithm 4.9: Preparing deployment artifacts

Input: workspace root, config.json, configured model paths

Output: deployment workspace with runtime files, models, requirements, and Dockerfile

```

1: Read config.json from the deployment workspace
2: Extract the models mapping from the configuration
3: For each configured model path do
4:     Resolve the path relative to the configuration file
5:     If the file does not exist, stop with a descriptive error
6:     Copy the artifact into the deployment models directory
7:     Rewrite the model path as a deployment-relative path
8: End for
9: Copy server.py, modelservice.py, schema_validation.py, tracking.py,
and deployment.py
10: Synchronize the accelera package into the serving workspace
11: Write runtime requirements for FastAPI, Uvicorn, Great Expectations,
Pandas, NumPy, and model libraries
12: Generate a Dockerfile that copies the service, configuration, and
model files
13: Return the prepared deployment workspace

```

After this algorithm completes, the local workspace has a repeatable service package. The important result is that the deployment engine no longer depends on scattered project files; it has a prepared runtime directory, a Dockerfile, dependency information, the active configuration, and the model artifacts needed by the service.

4.7.4 Runtime Validation, Inference, and Tracking

Runtime validation is a central part of the Deployment module because machine-learning models are sensitive to input shape, feature order, data type, and value ranges. A request may be syntactically valid JSON but still invalid for the trained model. The validation layer handles this by checking expected columns, missing values, unexpected fields, allowed values, and numeric bounds. It also coerces values into the expected data types where possible.

Algorithm 4.10 summarizes the prediction path followed by the serving application. The algorithm is written at the service level rather than as raw code because the key contribution is the sequence of validation, inference, and telemetry responsibilities.

Algorithm 4.10: Validating a request, running inference, and tracking the result

Input: API payload or uploaded CSV, active deployment configuration
 Output: prediction response or structured API error

- 1: Receive the request at /predict or /predict/csv
 - 2: Record the request start time
 - 3: Convert the payload into a tabular representation
 - 4: Validate required feature names and detect extra or missing features
 - 5: Coerce values to the configured types when possible
 - 6: Check numeric ranges and allowed categorical values
 - 7: If validation fails, record a validation_error event and return HTTP 422
 - 8: Convert validated rows into a NumPy-compatible array
 - 9: Apply loaded preprocessors in the configured order
 - 10: Call predict on the final model artifact
 - 11: Compute request latency in milliseconds
 - 12: Record endpoint, status, row count, latency, and predictions in telemetry
 - 13: Return row count and predictions to the client
-

The result is a deployed model service that is safer and more observable than a minimal prediction script. Invalid inputs are rejected with structured errors, successful predictions are returned through a consistent API response, and request activity is recorded for later inspection.

4.7.5 Model Version Control

The Deployment module includes a lightweight VCS dedicated to model deployment artifacts. This system is different from Git because it does not track all source-code changes. Instead, it snapshots the active `config.json` file and model files into an `experiments/` directory, assigns a short SHA-1 commit identifier, and records meta-data in `experiments.json`. This allows a user to return to a previous deployed model configuration without manually copying files.

Figure 4.13 gives the high-level VCS structure. It shows the relationship between the active deployment workspace, the index file, the committed experiment directories, and the restored deployment state.

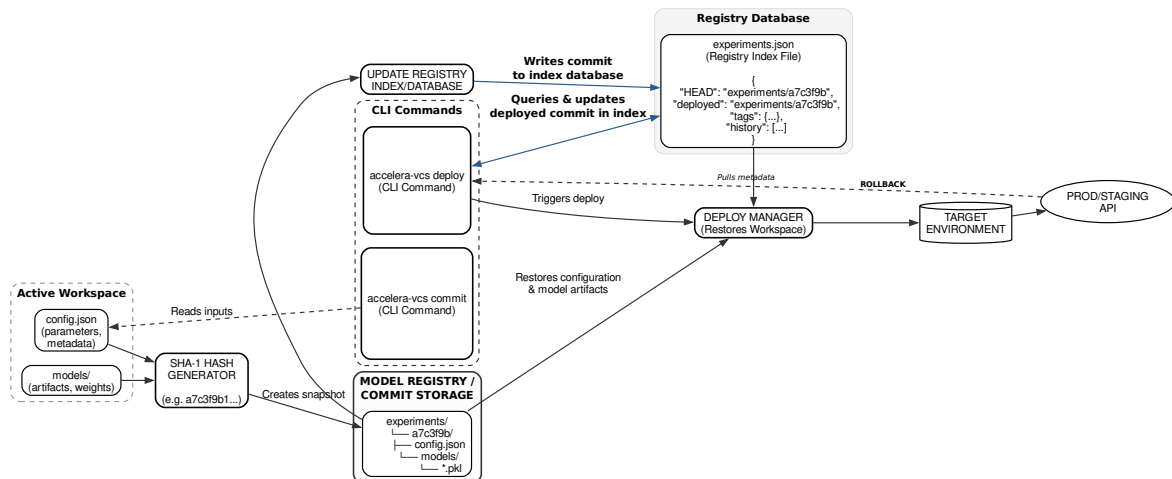


Figure 4.13: Deployment VCS registry structure.

The commit workflow is shown in Figure 4.14. Before this figure, it is useful to note that a deployment commit is not only a text label. It is a reproducibility unit containing both configuration and model binaries.

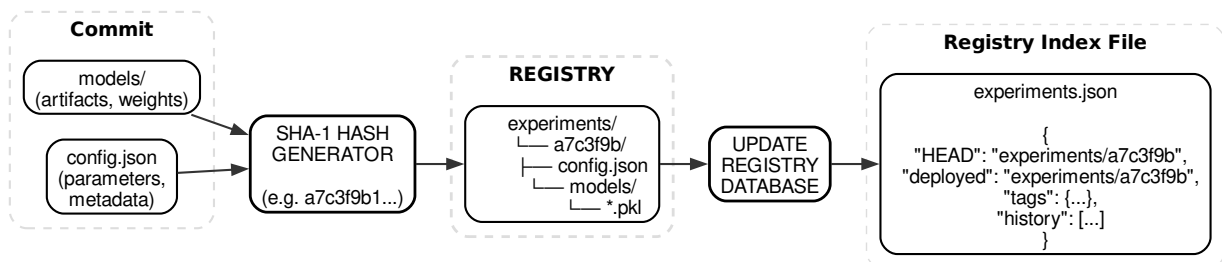


Figure 4.14: VCS commit workflow for deployment artifacts.

Algorithm 4.11 describes the commit process in pseudocode. This workflow supports reproducibility by ensuring that the model files referenced by a configuration are copied into the same commit directory as the configuration itself.

Algorithm 4.11: Creating a deployment VCS commit

Input: commit message, active config.json, active model files

Output: new deployment commit and updated experiments index

- 1: Require a non-empty commit message
- 2: Load and parse config.json
- 3: Resolve every model path listed in the models mapping
- 4: If any configured artifact is missing, stop the commit

- 5: Load `experiments.json`
- 6: Generate a short SHA-1 commit identifier from timestamp and message
- 7: Create `experiments/<commit-hash>/models/`
- 8: Copy model artifacts into the commit models directory
- 9: Rewrite model paths in the committed configuration as relative paths
- 10: Write `config.json` and `metadata.json` inside the commit directory
- 11: Append the commit record to `experiments.json`
- 12: Move HEAD to the new commit hash

After committing, the deployment workspace has an auditable model history. The history can be listed, inspected, and used to restore an earlier model snapshot. Figure 4.15 shows the deploy/rollback workflow that copies a selected snapshot back into the active runtime workspace.

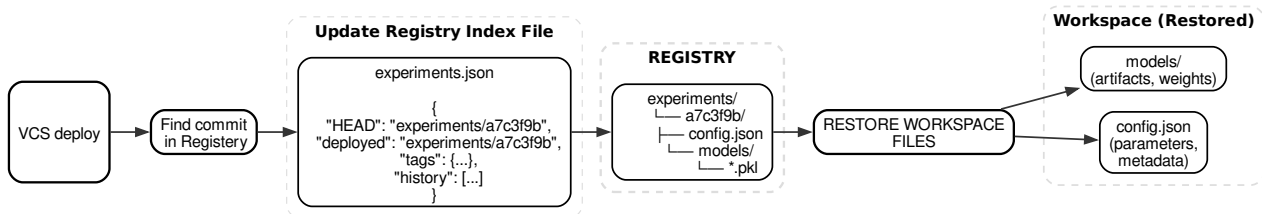


Figure 4.15: VCS deploy and rollback workflow.

4.7.6 Deployment Orchestration

The deployment orchestrator is responsible for converting the prepared service package into a running service. Its local path uses Docker to build and run the container on the developer machine. Its Heroku path uses the Heroku CLI and Container Registry. Its EC2 path uses SSH and `rsync` to synchronize files to a remote virtual machine, build the Docker image on the server, start the container, and verify the service through a health request.

Figure 4.16 presents the supported deployment targets. The figure is placed before the detailed algorithms because it helps the reader understand that the same prepared model service can be sent to different execution environments.

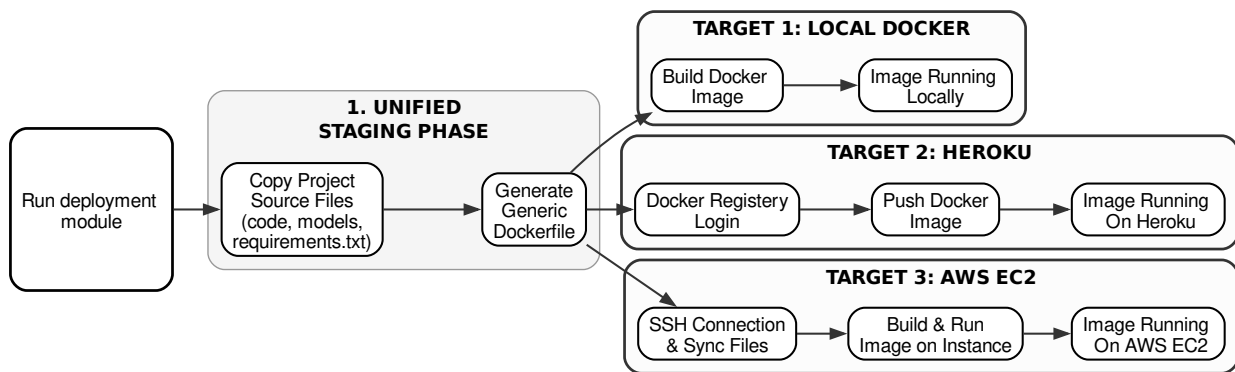


Figure 4.16: Multi-target deployment orchestration flow.

Algorithm 4.12 describes the local Docker workflow. This path is useful for testing the deployment service before moving it to a cloud or server environment.

Algorithm 4.12: Building and running the local deployment container

Input: prepared deployment workspace and target port

Output: running local Docker container

- 1: Prepare deployment files and Dockerfile
- 2: Build the Docker image named ml-model
- 3: Read the service port from the PORT environment variable or use 8000
- 4: Stop any currently running container that publishes the same port
- 5: Start the ml-model container with the selected port mapping
- 6: Print the local API URL and GUI URL
- 7: Keep the container process attached for local inspection

Once this workflow completes, the user can test the model through the local /predict, /health, or /gui routes on the selected port. This local path is especially valuable because it validates the complete serving package before a remote deployment is attempted.

Algorithm 4.13 describes the EC2 workflow. EC2 deployment is more complex because it involves a remote machine, SSH authentication, network ports, remote Docker execution, and health verification.

Algorithm 4.13: Deploying the model service to AWS EC2

Input: EC2 host, SSH user, optional private key, remote directory, port, image name, container name

Output: running EC2-hosted deployment service or a diagnostic error

- 1: Prepare local deployment files and Dockerfile

- 2: Build the SSH command using host, user, key, and host-key settings
 - 3: Create the remote deployment directory over SSH
 - 4: Synchronize Dockerfile, config.json, accelera_deployment, and models using rsync
 - 5: If requested, install and start Docker on the EC2 host
 - 6: Build the Docker image on the remote host
 - 7: Remove any old container with the target container name
 - 8: Start the new container with restart policy and port mapping
 - 9: Probe the container health endpoint from inside the host
 - 10: Print docker ps status, public application URL, and GUI URL
 - 11: Probe the public /health URL from the local machine
 - 12: If the public check fails, report that the TCP port may need to be opened
-

The result of the EC2 workflow is a public service URL when the instance, firewall rules, and Docker runtime are configured correctly. If the service is running internally but the public health check fails, the module gives a useful diagnostic message so the user can inspect security-group or port settings.

4.7.7 Command-Line Interface

The Deployment module exposes its behavior through command-line interfaces. This is appropriate for deployment because users often need to run the same operational steps repeatedly during testing, rebuilding, release, and rollback. Table 4.14 summarizes the orchestrator commands, while Table 4.15 summarizes the model VCS commands.

Table 4.14: Deployment orchestrator commands.

Command	Purpose
<code>prepare</code>	Copy serving files, synchronize the package, write requirements, validate model paths, and generate the Dockerfile.
<code>build</code>	Build the local Docker image, optionally without cache.
<code>run-local</code>	Start the local Docker container and publish the selected port.
<code>local</code>	Run <code>prepare</code> , <code>build</code> , and <code>run-local</code> as one local deployment workflow.
<code>heroku-create</code>	Create a Heroku app configured for the container stack.
<code>heroku-push</code> and <code>heroku-release</code>	Push the web container to Heroku and release it for traffic.
<code>heroku-deploy</code>	Execute the combined Heroku deployment sequence.
<code>ec2-deploy</code>	Synchronize files to EC2, build the container remotely, start the service, and verify health.
<code>ec2-stop</code> and <code>ec2-logs</code>	Stop the remote container or stream its logs for inspection.

Table 4.15: Deployment VCS commands.

Command	Purpose
<code>init</code>	Initialize the deployment experiment directory and empty <code>experiments.json</code> index.
<code>commit</code>	Snapshot the active <code>config.json</code> and model files under a short hash and commit message.
<code>status</code>	Display registry status, including total commits, HEAD, and deployed commit when available.
<code>log</code>	Print recorded commits in reverse chronological order.
<code>show</code>	Display a specific commit's metadata, configuration, and model files.
<code>deploy</code>	Restore a selected commit's configuration and model files into the active deployment workspace.

These commands make the module usable from scripts, demos, and repeatable terminal workflows. They also make the deployment process more transparent: users can run preparation, local testing, remote deployment, logging, stopping, committing, and rollback as separate steps instead of treating deployment as a single hidden operation.

4.7.8 Technical Value of the Module

The Deployment module is valuable because it pushes Accelera beyond model construction. It gives the project a practical final stage where trained models become reachable services with validation, observability, versioning, and target-specific deployment workflows. This makes Accelera stronger than a normal training-only tool: it supports the path from data preparation and model selection to an API that can be tested through a browser, called by another application, monitored through tracking summaries, and moved between local and remote environments.

The design is intentionally lightweight. It does not claim to replace large enterprise MLOps platforms, Kubernetes serving systems, or managed cloud AI services. Instead, it provides a focused deployment layer that fits the scale of the project and connects directly to Accelera's generated artifacts. For a graduation project, this is a significant engineering contribution because it requires combining machine-learning artifact management, web serving, validation, telemetry, Docker packaging, cloud/server operations, and testing into one coherent module.

4.8 Benchmark Platform Module

The Benchmark Platform module is designed to provide a shared environment where users can create machine learning benchmarks, submit predicted results, and compare submissions using selected evaluation metrics. The module includes a backend API, database schemas, Python validation and scoring scripts, and a frontend interface for users and admins.

The module separates benchmark creation from submission evaluation. This makes the workflow clearer: first, the benchmark creator defines the task and evaluation rules; then, users submit predictions under the same conditions. This separation helps ensure that all users are evaluated using the same ground-truth file and the same metric configuration. Another important design point is the use of Python scripts for file validation and metric scoring. The backend is implemented using Node.js and Express, while Python handles the machine learning evaluation part through Scikit-learn metrics and dataset loading utilities.

4.8.1 Functional Description

The main function of this module is to support fair comparison between different machine learning solutions. A user can create a benchmark by providing the benchmark title, description, problem type, target column, dataset link, test dataset link, ground-truth target link, and evaluation metric. The system validates the provided links and checks that the test set does not contain the target column while the target file contains the expected target column.

After a benchmark is created, other users can submit their prediction file and repository link. The system validates the submitted prediction file, calculates the score using

the selected Scikit-learn metric, stores the submission, and displays submissions on a leaderboard. The leaderboard is ordered based on the metric configuration, where some metrics are better when their value is higher and others are better when their value is lower.

4.8.2 Modular Decomposition

The Benchmark Platform is decomposed into several smaller components:

- **Benchmark Management:** This component handles benchmark creation, listing, filtering, viewing, and deletion. It stores benchmark information such as title, description, problem type, dataset links, target column, selected metric, metric parameters, creation date, and creator.
- **Metric Management:** This component allows admin users to define evaluation metrics. Each metric contains a display name, the corresponding Scikit-learn metric function name, needed parameters, problem type, and whether a higher or lower score is better. Metrics are linked to benchmarks and used during submission scoring.
- **Submission Management:** This component allows users to submit prediction files for a specific benchmark. Each submission stores the benchmark id, submitting user, repository link, prediction file link, score, and submission date. Users can also update their existing prediction file or delete their submission if they have permission.
- **Validation and Scoring Scripts:** The backend calls Python scripts to validate benchmark files and score submissions. During benchmark creation, the validation script checks that the test file does not include the target column and that the target file contains the required target column. During submission, the scoring script loads the ground-truth file and the submitted prediction file, checks their length and target column, then computes the selected Scikit-learn metric.
- **Leaderboard Interface:** The frontend displays benchmark submissions in a leaderboard. It shows the submitter information, repository link, score, and submission date. The sorting direction depends on the selected metric, so the platform can support both metrics where higher is better and metrics where lower is better.

Algorithm 4.14: Benchmark creation flow

Input: benchmark form data

Output: saved benchmark or validation error

- 1: Receive benchmark creation request
 - 2: Validate problem type
 - 3: Check title uniqueness
 - 4: Validate dataset, test set, and target label links
 - 5: Run `validate_user_links.py`
 - 6: If the test file contains the target column, reject benchmark
 - 7: If the target file does not contain the target column, reject benchmark
 - 8: Save benchmark document in MongoDB
-

Algorithm 4.15: Submission scoring flow

Input: benchmark target file, submitted prediction file, metric name

Output: score or validation error

- 1: Retrieve true target file
 - 2: Retrieve submitted prediction file
 - 3: Check that both files have the same number of rows
 - 4: Check that submitted file contains the target column
 - 5: Load metric function from `sklearn.metrics`
 - 6: Compute `metric(true_target, predicted_target, metric_parameters)`
 - 7: Return score to the backend as JSON
 - 8: Save submission with score
-

4.8.3 Design Constraints

Several constraints affected the design of the Benchmark Platform:

- **Task and data constraint:** The platform is limited to classification and regression benchmarks that use tabular files. This includes the training data, test data, prediction file, and target file.
- **Storage and access constraint:** Dataset and prediction files are handled through accessible links to reduce server storage usage.
- **Scoring constraint:** The platform uses Scikit-learn metrics for scoring. Therefore, the selected metric must be supported by Scikit-learn and must match the benchmark problem type.

4.8.4 Module Summary

The Benchmark Platform module provides a controlled environment for benchmark creation, prediction submission, validation, scoring, and leaderboard ranking. It keeps the web management logic in the backend and frontend, while the metric calculation logic is handled by Python scoring scripts.

Chapter 5: System Testing and Verification

This chapter explains how the project was tested to make sure that the main parts work correctly. The testing was not done in one way only, because each part of the project has a different output. The main purpose of testing was to make sure that each part can receive the input, apply the required steps, save the needed files, and return the expected output without errors.

5.1 Testing Setup

The testing was done mainly in a Python environment. The main tools used in testing were Python, Pytest, Pandas, NumPy, Scikit-learn, PyTorch, Matplotlib, and the libraries used in the implementation. For the benchmark platform, Node.js, Express, React, MongoDB, and Python scoring scripts were also checked. Some tests were written and run automatically using Pytest. Other parts were checked manually by comparing the actual output with the expected output.

5.2 Testing Plan and Strategy

The testing plan was based on testing the project step by step. First, each module was tested separately. After that, the outputs were checked in a more complete workflow. For example, preprocessing output was passed to machine learning models, generated reports were opened and inspected, and benchmark submissions were checked after scoring.

Different testing methods were used depending on the part being tested. Some parts were tested using unit tests with Pytest. Other parts were checked manually by comparing the actual output with the expected output. In some cases, the results were compared with another tool, to check if the preprocessing results were reasonable.

5.2.1 Module Testing

5.3 Accelera Pipe Module

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 5.1: Accelera Pipe latency scaling in seconds

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 5.1 shows the main performance trend without compressing the font. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate.

Table 5.2: Accelera Pipe RSS memory scaling in MB

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

Table 5.2 shows the current cost of graph-level parallelism. Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section, especially consumer-count based release, duplicate first-node sharing, and guarded preprocessing copies, target exactly this issue.

Table 5.3: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler → SVC	0.7900	0.8250
5,000	MinMaxScaler → SVC	0.9160	0.9100
10,000	MinMaxScaler → SVC	0.9385	0.9260
50,000	MinMaxScaler → SVC	0.9565	0.9598
100,000	StandardScaler → SVC	0.9675	0.9709
250,000	StandardScaler → SVC	0.9774	0.9768

Table 5.3 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from MinMax scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5.4: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5.4 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler followed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelera Pipe trades memory for parallel branch execution and native graph state.

5.4 Code Parallelizer Module

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 5.5: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 5.5 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 5.6: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 5.6 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×

Table 5.7: Linux parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

Table 5.7 reports the Linux direct `examples/parallel_accpipe.py` run after normalizing the example execution directory to the repository root. The Python implementation

takes 6.83 seconds on 500,000 samples. The first optimized process in the measurement sequence takes 5.88 seconds because it still pays process-local setup and cache lookup costs. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The validation in the example confirmed that the optimized output matched the Python output. This result shows why the integration needs two cache layers: one layer avoids repeating Python-to-C++ conversion and OpenMP insertion, while the lower compiled-module cache avoids recompiling the pybind extension.

Table 5.8: Linux direct example-script verification runs

Example	Baseline time (s)	Optimized time (s)	Validation
accpipe_demo	2.48	1.79	same accuracy, 0.9260
parallel_accpipe, run 1	6.83	5.88	outputs match
parallel_accpipe, run 2	6.83	0.08	outputs match
code_parallelizer, run 1	5.81	0.014	outputs match
code_parallelizer, run 2	6.05	0.015	outputs match
save_load, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
save_load, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 5.8 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`; the C path also reported about 0.26–0.27 seconds of compilation time, outside the listed executable run-time. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 5.9: Windows direct example correctness and path timing

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
accpipe_demo	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
parallel_accpipe	isolated cache, run 1	6.66	6.00	outputs match
parallel_accpipe	isolated cache, run 2	6.58	0.09	outputs match
code_parallelizer	isolated cache, run 1	5.59	1.04	outputs match
code_parallelizer	isolated cache, run 2	5.61	1.05	outputs match
save_load	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 5.10: Windows compilation and process overhead

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
accpipe_demo	normal	–	6.53	graph example startup
parallel_accpipe	isolated cache, run 1	included	14.37	cold compile path
parallel_accpipe	isolated cache, run 2	cached	8.33	warm cache path
code_parallelizer	isolated cache, run 1	0.87	8.74	temporary executable
code_parallelizer	isolated cache, run 2	1.04	9.00	temporary executable
save_load	normal	–	2.71	persistence example

Tables 5.9 and 5.10 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table remains readable. The examples were run one by one in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold cache behavior and the second run represents warm cache behavior. The strongest cache effect appears in `parallel_accpipe.py`: the cold end-to-end Accelera path takes 6.00 seconds because it must convert the function, select loops, compile a pybind extension, link it, and load it into Python. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages.

The Windows `code_parallelizer_demo.py` runs show a different limitation. The generated C/OpenMP executable is compiled into a temporary directory on every run, so the second run does not reuse the final linked program. The measured C executable subprocess takes about 1.04 seconds, while the instrumented main-body timer inside the generated program reports only about 0.024–0.025 seconds. This gap is Windows process startup, OpenMP runtime initialization, dynamic library loading, and executable loading overhead. The compile/link step itself adds about 0.87–1.04 seconds. On Linux, the same benchmark has lower launch and linking overhead in the reported measurements, so executable runtime is closer to the timed main-body work. For this reason, Windows results should be interpreted as end-to-end toolchain latency results, not only as loop-kernel speed results.

5.4.0.1 Auto Preprocessing Module

Most of the preprocessing functions were tested using unit tests. These tests were used to check validation rules, artifact saving, and feature transformation. Real datasets were also used to test the system in a more practical way. These datasets were collected from Kaggle for tabular, text, and image data.

How the Auto Preprocessing testing works The Auto Preprocessing testing was done as a practical full-flow test. The idea was to run preprocessing on real datasets,

then pass the processed output to a model. This means the test does not only check that preprocessing runs without errors, but also checks that the returned data can actually be used for training, validation, and prediction.

For tabular datasets, the test starts by loading each dataset with its target column and problem type. The preprocessing flow splits the data, fits the needed encoders and scalers, transforms the features and target, applies feature selection, and saves the preprocessing artifacts. After that, a machine learning model is trained on the processed training data and evaluated on the processed validation data. Classification datasets are evaluated using Macro F1-score, and regression datasets are evaluated using the R-squared score. The same datasets are also processed by a baseline automated cleaning tool, so the final scores can be compared and the Accelera output can be checked against an existing preprocessing approach.

For text datasets, the test checks that raw text can be converted into features that a classifier can understand. The preprocessing flow receives the text column and target column, cleans the text, tokenizes it, applies TF-IDF, encodes the labels, and returns processed training and validation data. A classifier is then trained on the transformed text features and evaluated using Macro F1-score. This confirms that the text preprocessing output is numeric, consistent, and usable for a normal machine learning model.

For image classification, the test checks the full image preparation flow. The preprocessing reads images from class folders, validates image files, skips corrupted images, creates class-to-label mappings, resizes the images, applies augmentation when enabled, and returns PyTorch DataLoaders. A neural network model is then trained using these loaders. The inference path is also checked by loading the saved model and applying the testing preprocessing to new images. This verifies that the saved image size and class mapping can be reused during testing, not only during training.

The main concept in all three tests is to check preprocessing through the next step in the machine learning pipeline. If the tabular arrays, TF-IDF matrices, labels, image tensors, or saved artifacts were wrong, the model training or evaluation step would fail or give invalid results.

Table 5.11: Datasets Used in Module Testing

Dataset Name	Shape	Dataset Type	Problem Type
startup_founder_burnout_2026	(50000, 29)	Tabular	Classification
healthy_diet_calorie_intake	(6000, 15)	Tabular	Classification
main gaming_addiction	(250, 49)	Tabular	Classification
student_exam_performance_dataset	(10000, 17)	Tabular	Classification
heart	(1025, 14)	Tabular	Classification
Dataset_spine	(310, 7)	Tabular	Classification
diabetes_dataset	(100000, 31)	Tabular	Classification
student_placement_synthetic	(100000, 18)	Tabular	Classification
Titanic-Dataset	(891, 12)	Tabular	Classification
customer_purchase_data	(1500, 9)	Tabular	Classification
global_ai_jobs	(90000, 35)	Tabular	Regression
CarPrice_Assignment	(205, 26)	Tabular	Regression
Housing	(545, 13)	Tabular	Regression
PetImages	(24998,)	Image	Classification
DailyDialog	(11327, 2)	Text	Classification
TestReviews	(4321, 2)	Text	Classification

Table 5.12: Models and Evaluation Metrics Used in Module Testing

Dataset Name	Model Used	Evaluation Metric	Accelera Result
startup_founder_burnout_2026	RandomForest	Macro F1-score	0.9999
healthy_diet_calorie_intake	RandomForest	Macro F1-score	1.0000
gaming_addiction	RandomForest	Macro F1-score	1.0000
student_exam_performance_dataset	RandomForest	Macro F1-score	0.8573
heart	RandomForest	Macro F1-score	0.7860
Dataset_spine	RandomForest	Macro F1-score	0.7840
diabetes_dataset	RandomForest	Macro F1-score	0.9985
student_placement_synthetic	RandomForest	Macro F1-score	0.9986
Titanic-Dataset	RandomForest	Macro F1-score	0.7906
customer_purchase_data	RandomForest	Macro F1-score	0.9385
global_ai_jobs	RandomForest	R-squared Score	0.9227
CarPrice_Assignment	RandomForest	R-squared Score	0.9562
Housing	RandomForest	R-squared Score	0.6037
PetImages	ResNet model	Accuracy	0.9700
DailyDialog	XGBClassifier	Macro F1-score	0.9000
TestReviews	XGBClassifier	Macro F1-score	0.6080

These results show that the preprocessing output can be used by different models after transformation. The tabular results were strong in many datasets, especially the larger classification datasets and the car price regression dataset. The lower scores on datasets such as TestReviews show that preprocessing alone does not solve every modeling problem. Some datasets still need better features, model tuning, or more suitable models.

5.4.0.2 AutoML Module

Experimental Objectives The AutoML experiments compare Accelera AutoML with Auto-sklearn, FLAML, and TPOT on supervised tabular classification and regression tasks. The evaluation focuses on two questions: whether the systems reach comparable predictive quality, and how much wall-clock time they require. Classification is evaluated mainly by accuracy. Regression is evaluated by R^2 , root mean squared error (RMSE), and mean absolute error (MAE). Runtime is reported in seconds.

Benchmark Protocol The classification experiments were executed through the AutoMLBenchmark (AMLB) runner using the `flaml_paper_39` classification suite and the `flaml_paper_1h` constraint. The reported classification evaluation covers 12 datasets. The regression benchmark was built from the Penn Machine Learning Benchmarks (PMLB) repository and executed through AMLB using the `pmlb_regression` suite with the `pmlb_30m` constraint. The reported regression evaluation covers 10 datasets.

All systems were run through the same benchmark interface. The recorded setup used one fold and one allocated core per task. The regression time budget was 1,800 seconds per dataset, while the classification constraint allowed up to one hour.

Classification Results Table 5.13 summarizes the stored classification results. Auto-sklearn obtained the highest mean accuracy at 0.8842. Accelera reached a mean accuracy of 0.8791, a difference of about 0.0051, while recording the lowest mean duration at 1,201.7 seconds. Its mean runtime was approximately one third of the runtimes recorded for the other systems.

Table 5.13: AutoML classification benchmark summary

System	Mean accuracy	Mean time (s)	Accuracy wins	Speed wins
Auto-sklearn	0.884178	3615.71	3	3
FLAML	0.881427	3655.63	6	0
Accelera	0.879076	1201.72	6	15
TPOT	0.877654	3953.78	8	1

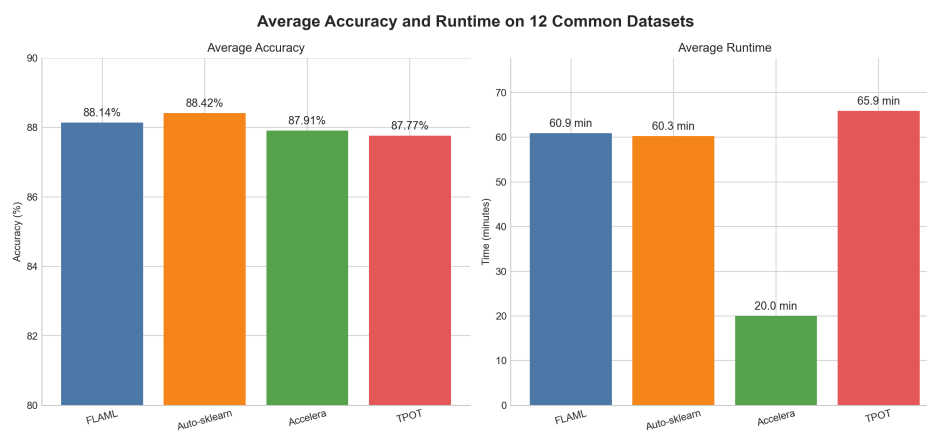


Figure 5.1: Mean classification accuracy and runtime for the evaluated AutoML systems

Regression Results Table 5.14 reports the regression summary over ten datasets. Auto-sklearn obtained the highest mean R^2 at 0.5609, followed closely by TPOT at 0.5592. Accelera reached 0.5294 and recorded the lowest mean runtime at 892.64 seconds, compared with 1,820–1,864 seconds for the competing systems. Accelera also achieved six speed wins, the most of any evaluated system.

Table 5.14: AutoML regression benchmark summary over ten datasets

System	Mean R^2	Median R^2	Mean time (s)	R^2 wins	RMSE wins	MAE wins	Speed wins
Auto-sklearn	0.560947	0.494362	1820.18	2	2	2	2
TPOT	0.559238	0.493144	1849.70	2	2	2	0
Accelera	0.529389	0.432577	892.64	2	2	2	6
FLAML	0.522743	0.497530	1863.85	4	4	4	2

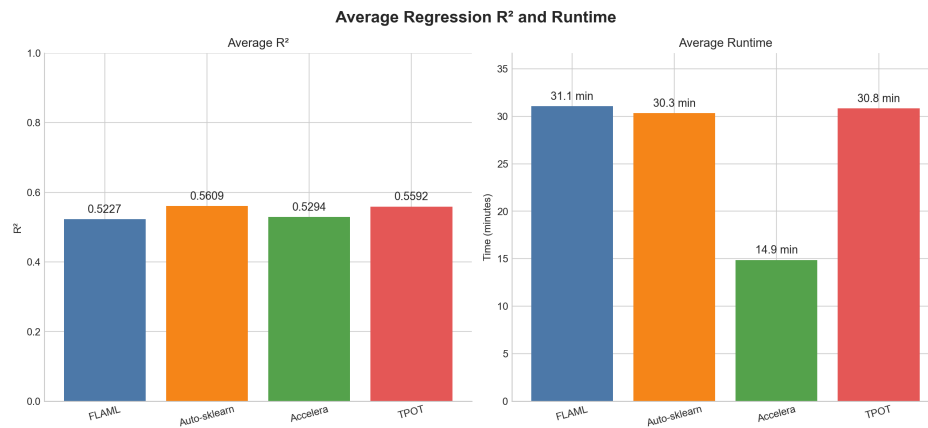


Figure 5.2: Mean regression R^2 and runtime for the evaluated AutoML systems

Threats to Validity The current results use a single benchmark fold, so they do not quantify variation across repeated splits or random seeds. Runtime is sensitive to hardware, operating-system load, framework startup overhead, and timeout handling. Aggregate means can also hide dataset-level failures or missing measurements. Therefore, the tables should be read as a summary of the stored runs rather than a universal ranking of AutoML systems.

5.4.0.3 Deployment Module

5.4.1 Experimental Setup and Test Suite

The Deployment module was verified using a focused `pytest` test suite that exercises the module as a serving, packaging, versioning, and orchestration component. The goal of this verification is not only to check individual functions, but also to confirm that the deployment workflow can move from saved artifacts and configuration files to a prepared API service in a repeatable manner.

The test environment is intentionally isolated from external cloud services. Temporary workspaces created with the `tmp_path` fixture are used to hold generated configuration files, model artifacts, VCS registries, Dockerfile outputs, and telemetry logs. The `monkeypatch` fixture replaces external dependencies such as subprocess calls, SSH execution, Heroku commands, and EC2 health checks. This allows the test suite to

verify the correctness of the deployment logic without requiring a live Heroku account, a running EC2 instance, or active Docker containers during every test run.

The command used to execute the deployment test suite is shown below. The explicit `PYTHONPATH` value is important because the deployment tests load modules both through the package path and through the isolated serving runtime path.

Algorithm 5.1: Deployment test-suite execution command

```
PYTHONPATH=accelera/src/deployment:accelera/src pytest accelera/src/
deployment/tests
```

This setup verifies the Deployment module in a way that is suitable for headless execution and continuous integration. Instead of depending on the availability of external platforms, the tests check that Accelera generates the correct files, calls the correct commands, validates inputs correctly, records telemetry correctly, and preserves model snapshots correctly.

5.4.2 Verification Objectives

The Deployment module contains several responsibilities that must work together. Table 5.15 summarizes the main verification objectives used in the test suite. The table is included before the detailed results to show how each group of tests supports a concrete quality requirement of the module.

Table 5.15: Deployment module verification objectives.

Objective	Reason	Evidence in tests
Artifact reproducibility	The deployed service must be based on known model files and a known configuration.	VCS tests check commit creation, parent links, HEAD, deployed markers, and rollback restoration.
Serving correctness	The service must load models and preprocessors in the expected order before prediction.	Model service tests load serialized artifacts, verify idempotent loading, and check preprocessing before prediction.
Input reliability	Invalid payloads must be rejected before reaching the model.	Schema validation tests cover missing columns, unexpected columns, type coercion, booleans, integers, numeric ranges, and allowed values.
API behavior	The deployed service must expose predictable routes for prediction, health, GUI, CSV upload, and tracking.	Server tests verify successful responses, validation errors, internal errors, row counting, GUI rendering, and tracking summary behavior.
Operational repeatability	Deployment commands must produce correct Docker, Heroku, and EC2 command flows.	Orchestration tests inspect Docker build/run calls, Heroku sequencing, SSH transport, remote scripts, <code>rsync</code> , <code>stop</code> , and <code>log</code> commands.

5.4.3 Subsystem Verification Details

The test suite targets the major components of the Deployment module: model VCS, model loading, runtime validation, API serving, telemetry, and deployment orchestration. These components were tested separately because each one represents a failure point in a real deployment workflow.

- Model Version Control verification:** The VCS tests verify the stability of short SHA-1 commit identifiers, prefix resolution, index save/load behavior, workspace initialization, commit creation, parent tracking, log formatting, status reporting, and deployment rollback. The most important result is that committed model artifacts and configuration files can be restored into the active deployment workspace, which supports repeatable deployment and rollback.
- Model Service verification:** The `ModelService` tests verify that the service reads

`config.json`, loads serialized preprocessors and models, identifies the predict-capable artifact, and applies preprocessing before prediction. They also verify that loading is idempotent and that a clear runtime error is raised when no loaded artifact supports prediction.

3. **Runtime Schema Validation verification:** The schema tests verify both disabled and enabled validation modes. They confirm that missing columns and unexpected columns are detected, that numeric and integer values are coerced safely, that boolean representations are normalized, and that invalid values produce structured validation errors. These checks are important because the deployed API must protect the model from malformed inputs.
4. **Web Serving Endpoints verification:** The server tests verify the behavior of the FastAPI routes without requiring a live server process. They check that the root path redirects to the GUI, that `/health` and `/tracking/summary` report service state, that missing prediction input is rejected, that successful predictions record telemetry events, and that validation or internal errors map to the expected HTTP error behavior.
5. **Prediction Tracking verification:** The telemetry tests verify that a disabled tracker performs no writes, while an enabled tracker writes valid JSON lines and can summarize request counts, total rows, status counts, and the last recorded event. This confirms that the deployed service has a lightweight observability layer rather than only a prediction endpoint.
6. **Deployment Orchestration verification:** The orchestration tests verify the command-building and file-generation logic of `deployment.py`. They check valid and invalid ports, model path validation, service-source synchronization, generated `requirements.txt`, generated Dockerfile, Docker build and run commands, Heroku deployment order, SSH configuration, remote EC2 script generation, `rsync` synchronization, EC2 deployment flow, and remote stop/log operations.

5.4.4 Command-Flow Verification

In addition to unit-level behavior, the tests verify important command flows that a user would execute during deployment. This is significant because deployment failures often happen not inside model inference, but in the surrounding operational steps: building containers, selecting ports, synchronizing files, and starting services on remote hosts.

Table 5.16 summarizes the command flows covered by the orchestration tests. The table shows that the Deployment module was tested as an operational tool, not only as a collection of Python helper functions.

Table 5.16: Verified deployment command flows.

Flow	Verified behavior	Result
Local Docker build	The build command includes Docker build arguments, image naming, and optional cache bypass.	Command construction passed.
Local Docker run	The module checks for containers already publishing the selected port, stops conflicting containers, and starts a new mapped container.	Port-handling flow passed.
Heroku deployment	The complete deployment sequence runs login, optional app creation, container login, push, release, and open in the expected order.	Heroku orchestration passed.
EC2 remote deployment	The module prepares files, creates the remote directory, synchronizes build inputs, runs the remote deployment script, and checks the public URL.	EC2 orchestration passed.
EC2 operations	The stop and logs commands generate the expected remote Docker operations.	Operational commands passed.

5.4.5 Deployment Test Results

When the deployment test suite is executed in an environment with the required Python dependencies available, all 51 test cases pass. The result is a strong indication that the module's core responsibilities are implemented consistently: artifacts are versioned, models are loaded, inputs are validated, API behavior is predictable, telemetry is recorded, and deployment commands are generated correctly.

Table 5.17 presents the final test distribution. The table is organized by test file so that each subsystem can be traced back to its verification source in the repository.

Table 5.17: Deployment Module Test Suite Verification Summary

Test File	Cases	Passed	Failed	Verified Feature
vcs_test.py	14	14	0	Registry snapshots, HEAD, status, and rollback restoration
modelservice_test.py	6	6	0	Artifact loading, idempotent loading, and preprocessor sequencing
schema_validation_test.py	8	8	0	Column checks, type coercion, booleans, ranges, and allowed values
server_test.py	9	9	0	FastAPI routes, prediction responses, GUI rendering, and telemetry events
tracking_test.py	3	3	0	JSONL telemetry writing and summary aggregation
deployment_test.py	11	11	0	Docker, Heroku, SSH, EC2, model-path, and generated-file workflows
Total	51	51	0	All deployment checks passed

The successful result confirms that the Deployment module operates as a reliable final stage for Accelera. It does not merely save model files; it validates the artifacts needed for serving, builds a deployable runtime package, protects inference through schema validation, exposes API behavior, records prediction activity, and verifies the command paths needed for local and remote deployment. This strengthens Accelera’s position as an end-to-end machine-learning system that continues beyond training and evaluation into practical service delivery.

5.4.5.1 Accelera Pipeline Reporting Module

The pipeline reporting part was tested by generating reports from executed pipeline results. The test checks that the report can read the serialized graph file, create the graph image, detect branches, show the selected path, and display metric results in the correct format.

Different metric result types were checked, such as single numbers, arrays, dictionaries, strings, tuples, and plotted figures. The expected output was a `report.html` file with readable tables and figures. The graph image was also checked manually to make sure that selected nodes and normal nodes were shown using different colors.

5.4.5.2 Benchmark Platform Module

The Benchmark Platform module was evaluated using manual functional testing to check its main features from the user side. The testing focused on creating benchmark tasks, validating dataset links, submitting prediction files, calculating scores, and displaying the results in the leaderboard.

The leaderboard was tested by submitting different results and checking whether they were ordered correctly based on the selected evaluation metric. This included metrics where higher values are better and metrics where lower values are better.

The scoring process was also tested manually by using different target and prediction files. The platform was checked to make sure it rejects invalid submissions, such as files with different row counts or missing target columns. These tests showed that the

benchmark platform can handle result submission, calculate scores, validate files, and display leaderboard results correctly.

5.4.6 Integration Testing

Integration testing was done by connecting the output of one part with another part. For Auto Preprocessing, the transformed data was passed to machine learning models to check that the output was usable. For pipeline reporting, the report was generated after graph serialization and metric calculation. For benchmarking, the backend called the Python scoring script and then saved the returned score in MongoDB.

These checks were important because a function can pass a unit test but still fail when used with the rest of the system.

5.4.7 Testing Schedule

The testing was done during implementation and after completing each module. First, small functions were tested. Then complete module flows were tested. After that, final outputs such as reports, saved files, and leaderboard results were checked manually. The final comparison with AutoCleanML was done after the tabular preprocessing flow became stable.

5.4.8 Comparative Results to Previous Work

5.4.8.1 Auto Preprocessing

Auto Preprocessing was compared with AutoCleanML to check the performance of the proposed preprocessing approach. The comparison was done using several tabular datasets collected from Kaggle. The same datasets and the same machine learning models were used for both tools. The comparison was applied only on tabular datasets because AutoCleanML mainly works with tabular data. For classification datasets, the Macro F1-score was used as the evaluation metric. For regression datasets, the R-squared score was used to measure how well the model predicts the target values. After preprocessing the datasets, the output data was tested using four machine learning models. For classification problems, Logistic Regression, Random Forest, Decision Tree, and KNN were used. For regression problems, Linear Regression, Random Forest, Decision Tree, and KNN were used. This helped test the preprocessing output with different model types, not only one model. The results were saved and plotted as comparison graphs between Accelera and AutoCleanML. These graphs show the score of each tool on the selected datasets. The purpose of this comparison was to check whether Accelera can produce preprocessing results that are close to, or better than, an existing preprocessing tool.

Table 5.18: Classification comparison using Logistic Regression and Random Forest

Dataset	Logistic Regression		Random Forest	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Startup founder burnout	1.0000	1.0000	0.9999	0.9999
Healthy diet calorie intake	0.9917	0.9890	1.0000	1.0000
Gaming addiction	0.8937	0.8937	0.9646	1.0000
Student exam performance	0.8667	0.8657	0.8579	0.8573
Heart	0.8019	0.8007	0.7689	0.7860
Spine dataset	0.8691	0.7943	0.8360	0.7840
Diabetes dataset	0.9995	0.9985	0.9996	0.9985
Student placement synthetic	0.5657	0.5879	0.6241	0.9986
Titanic dataset	0.7734	0.7864	0.7906	0.7906
Customer purchase data	0.8484	0.8375	0.9386	0.9385

Table 5.19: Classification comparison using Decision Tree and KNN

Dataset	Decision Tree		KNN	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Startup founder burnout	1.0000	1.0000	0.9824	0.9881
Healthy diet calorie intake	1.0000	1.0000	0.9379	0.9184
Gaming addiction	1.0000	1.0000	0.6905	0.8512
Student exam performance	0.7987	0.8063	0.8423	0.8338
Heart	0.7213	0.7704	0.7689	0.7860
Spine dataset	0.7686	0.7632	0.8239	0.8042
Diabetes dataset	0.9995	0.9972	0.9925	0.9980
Student placement synthetic	0.5827	0.9968	0.5995	0.8348
Titanic dataset	0.6979	0.7877	0.7203	0.7562
Customer purchase data	0.8486	0.8449	0.8544	0.8806

Table 5.20: Regression comparison using Linear Regression and Random Forest

Dataset	Linear Regression		Random Forest	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Global AI jobs	0.9022	0.7498	0.9288	0.9227
Car price assignment	0.8886	0.8943	0.9535	0.9562
Housing	0.6506	0.6482	0.6087	0.6037

Table 5.21: Regression comparison using Decision Tree and KNN

Dataset	Decision Tree		KNN	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Global AI jobs	0.8640	0.8412	0.7586	0.7510
Car price assignment	0.9269	0.8825	0.7905	0.7478
Housing	0.3819	0.3634	0.5312	0.5336

From the classification and regression results, Accelera preprocessing achieved competitive performance compared to AutoCleanML. The comparison was tested using different types of models, including linear models, tree based models, Random Forest, and KNN. In many cases, the results of Accelera were very close to AutoCleanML, and in some cases Accelera achieved better scores.

These results show that the implemented preprocessing steps in Accelera are able to prepare data in a valid way for different machine learning techniques. The performance was not limited to one specific model type, but appeared across several models and datasets. This means that Accelera successfully reached results close to an existing automated preprocessing tool, while still using its own preprocessing approach.

However, the results also show that no preprocessing tool gives the best score in all cases. The final performance can still depend on the dataset, the problem type, and the model used after preprocessing.

Chapter 6: Conclusions and Future Work

6.1 Faced Challenges

The main implementation challenges were preserving correctness while sharing branch computations, releasing intermediate arrays only after their final consumer, and limiting generated OpenMP code to loops that pass dependency and conversion checks. For the Deployment module, the main challenge was combining model artifacts, serving code, validation, telemetry, Docker packaging, and remote execution into one repeatable workflow. The experiments also identify memory usage during branch-level parallel execution as the main remaining graph-backend challenge.

6.2 Gained Experience

6.3 Conclusions

Accelera provides a connected framework for preparing data, selecting models, building graph-based machine learning workflows, improving supported custom code, deploying trained models, and comparing results through a benchmark platform. The work shows that the project is not only a collection of separate tools, but a workflow where each module can support a different stage of the machine learning process.

The Auto Preprocessing module reduces repeated manual preparation work for tabular, text, and image datasets. It saves fitted artifacts so that testing data can be transformed in the same way as training data. The AutoML module adds automated model search under time and trial limits, which helps compare candidate models more systematically. Accelera Pipe represents workflows as graphs, so shared preprocessing, branches, predictions, and metrics can be organized more clearly. The Code Parallelizer adds a source-level optimization path for supported loop-heavy code while keeping conservative safety checks.

The Deployment module and Benchmark Platform complete the workflow by supporting model serving, version tracking, submission validation, scoring, and leaderboard comparison. The testing and experimental results show that the implemented modules can produce usable outputs, generate reports, evaluate models, and compare results. Overall, Accelera gives a practical base for a more organized and reusable machine learning workflow. Accelera Pipe provides a graph-based execution model for branch-heavy machine-learning pipelines and supports fitted executed graphs. The Code Parallelizer provides a source-to-source path from supported Python or C++ loops to OpenMP-annotated C++ using converter rules, loop feature extraction, classi-

fier guidance, and safety checks. The Deployment module completes the workflow by turning saved models and generated artifacts into validated API services that can run locally or on supported cloud/server targets. The available experiments show runtime improvements on large branch-selection workloads while preserving selected models, accuracy, and output equivalence, and the deployment tests verify repeatable serving, validation, telemetry, and orchestration behavior.

6.4 Future Work

Future work should continue reducing graph memory consumption while preserving parallel branch execution, and should extend safe converter , loop-analysis coverage without weakening correctness checks, The preprocessing pipeline can be improved by adapting the preprocessing steps based on the selected model type and by adding techniques to handle imbalanced datasets. In the future, the benchmark platform can be improved by saving uploaded datasets on the server instead of using Google Drive links. It can also allow users to upload code files directly, instead of submitting only links. The Deployment module can also be extended with additional targets such as managed container services, richer monitoring, authentication for prediction endpoints, and more advanced model registry metadata.

Reference

- [1] Matthieu Komorowski, Dominic C. Marshall, Justin D. Saliccioli, and Yves Crutain. “Exploratory data analysis.” In *Secondary Analysis of Electronic Health Records*, pages 185–203. Springer, 2016.
- [2] Mohammad Hossin and Md Nasir Sulaiman. “A review on evaluation metrics for data classification evaluations.” *International Journal of Data Mining and Knowledge Management Process*, 5(2):1, 2015.
- [3] Navin Sabharwal and Amit Agrawal. “Introduction to Natural Language Processing.” In *Hands-on Question Answering Systems with BERT: Applications in Neural Networks and Natural Language Processing*, pages 1–14. Springer, 2021.
- [4] Koukaras Paraskevas and Tjortjis Christos. “Data preprocessing and feature engineering for data mining: Techniques, tools, and best practices.” *AI*, 6(10):257, 2025. MDPI AG.
- [5] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms.” In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013.
- [6] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. “Efficient and robust automated machine learning.” In *Advances in Neural Information Processing Systems*, 2015.
- [7] Randal S. Olson, Nathan Bartley, Ryan J. Urbanowicz, and Jason H. Moore. “Evaluation of a tree-based pipeline optimization tool for automating data science.” In *Applications of Evolutionary Computation*, 2016.
- [8] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. “FLAML: A fast and lightweight AutoML library.” In *Proceedings of Machine Learning and Systems*, 2021.
- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] T. Kadosh, N. Hasabnis, P. Soundararajan, V. A. Vo, M. Capota, N. Ahmed, Y. Pinter, and G. Oren, “OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation,” arXiv preprint arXiv:2409.14771, 2024.

- [11] Nick Erickson et al. “AutoGluon-Tabular: Robust and accurate AutoML for structured data.” *arXiv preprint arXiv:2003.06505*, 2020.
- [12] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Sequential model-based optimization for general algorithm configuration.” In *Learning and Intelligent Optimization*, 2011.
- [13] James Bergstra and Yoshua Bengio. “Random search for hyper-parameter optimization.” *Journal of Machine Learning Research*, 13:281–305, 2012.
- [14] David H. Wolpert. “Stacked generalization.” *Neural Networks*, 5(2):241–259, 1992.

Chapter A: Development Platforms and Tools

A.1 Hardware Platform

A.2 Software Platform

The implemented modules use Python, C++, PyBind11, CMake, Clang-based AST analysis, OpenMP, NumPy, Pandas, Matplotlib, PyTorch, Scikit-learn, Node.js, React, MongoDB, FastAPI, Uvicorn, Great Expectations, Docker, Heroku CLI, SSH, and AWS EC2 workflows. The Auto Preprocessing module depends mainly on Python data-science libraries. The Pipeline Reporting module uses Python, HTML report generation, Matplotlib, Pandas tables, and Graphviz for graph visualization. The Benchmark Platform uses a Node.js and Express backend, MongoDB for storing benchmarks and submissions, React for the frontend, and Python scripts for dataset validation and metric scoring. The Deployment module uses FastAPI and Uvicorn for serving, Great Expectations for runtime validation, Docker for container packaging, and SSH/rsync for EC2 synchronization. The available experiments include Linux and Windows execution paths.

Chapter B: Use Cases

B.1 Auto Preprocessing Use Cases

The user can use Auto Preprocessing when they have a dataset and want to prepare it before training a model. For tabular data, the module can clean missing values, encode categorical columns, scale numerical columns, select features, save artifacts, and generate a report. For text data, it can clean sentences, tokenize them, apply TF-IDF, encode labels, and generate a report. For image data, it can validate images, map class folders to labels, resize images, apply augmentation, and create DataLoaders.

B.2 Accelera Pipe Use Cases

Accelera Pipe is suitable when a machine-learning experiment is better represented as a graph than as one fixed sequence of operations. The user can place alternative preprocessing objects, estimators, or metrics in branches, execute independent paths concurrently, and select a fitted path using validation results. The executed graph can then be reused for inference without retraining the selected model.

Table B.1: Representative Accelera Pipe use cases.

Use case	Graph construction	Expected benefit
Candidate comparison	Branch over preprocessing methods, models, or both, then evaluate every path with the same metric.	A consistent search procedure without manually constructing and executing each candidate pipeline.
Validation-based selection	Attach prediction and metric nodes containing validation data, then use a maximum, minimum, or custom selection strategy.	A fitted executable graph containing the path chosen by the required objective.
Parallel experimentation	Enable the default parallel graph execution when independent branches are sufficiently large.	Concurrent evaluation of graph paths while retaining a serial mode for debugging or small workloads.
Ensemble construction	Connect model branches to a converge node using a supported voting strategy.	Combined predictions from several estimators inside the same graph.
Reusable inference	Save the selected executed graph and load it in a later process.	Persistence of fitted preprocessing and model objects in one file.

Continued on the next page

Use case	Graph construction	Expected benefit
Custom transformation	Add a Python function or custom transformer as a preprocessing node.	Automatic Parallelizer optimization when the callable is supported, with fallback to its original Python implementation.

B.3 Code Parallelizer Use Cases

The Code Parallelizer is intended for loop-heavy transformations whose iterations are independent or form a supported scalar reduction. It can be used as a standalone source-to-source tool or as an optimization service inside Accelera Pipe. The source form determines whether the result is generated code or an executable callable.

Table B.2: Representative Code Parallelizer use cases.

Use case	Accepted input	Produced result
Existing source file	Path to a supported Python, C, or C++ source file.	A <code>parallelized_<name>.c</code> file written to the selected output directory.
Source inspection	In-memory Python or C/C++ source text.	Parallelized C++ source returned as a string for inspection or later compilation.
Python acceleration	A supported top-level Python function containing loop-oriented numerical code.	A compiled Python-callable wrapper backed by C++ and OpenMP, or the original function if optimization fails.
Transformer acceleration	A custom transformer with a supported <code>transform</code> method.	The transformer instance with its transform method replaced by the optimized callable when compilation succeeds.
Pipeline integration	A custom function or transformer passed to <code>Pipeline.preprocess</code> .	Transparent optimization before the callable is stored in the preprocessing node.

The module is most useful for element-wise array updates, row-wise numerical transformations, nested independent loops, and sum or product reductions. A loop with cross-iteration dependencies, indirect writes, unsupported Python syntax, or state that cannot be translated safely should remain sequential.

B.4 Pipeline Reporting Use Cases

The user can use the pipeline report after running an Accelerera pipeline. The report helps show the graph structure, branches, selected path, and metric results. This is useful when the pipeline contains more than one branch and the user wants to understand which path was selected.

B.5 Benchmark Platform Use Cases

The benchmark platform can be used when different users or models need to be compared on the same task. A benchmark creator defines the dataset, test file, target column, and metric. Users then submit prediction files, and the platform calculates the score and displays the leaderboard.

Chapter C: User Guide

C.1 Using Accelera Pipe

C.1.1 Constructing and selecting a graph

An Accelera Pipe program is assembled by chaining node-building methods. A node passed with `branch=True` is returned as a branch description; `branch()` inserts those descriptions into the graph. The following example compares two scalers under the same classifier and selects the path with the highest validation accuracy.

Code Example C.1: Building and selecting an Accelera Pipe graph

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import MinMaxScaler, StandardScaler

from accelera.src.accelera_pipe.core.pipeline import Pipeline

pipe = Pipeline()
pipe.branch(
    "preprocessing",
    pipe.preprocess("standard", StandardScaler(), branch=True),
    pipe.preprocess("minmax", MinMaxScaler(), branch=True),
).model(
    "classifier", LogisticRegression(max_iter=1000)
).predict(
    "validation_prediction", test_data=X_val
).metric(
    "validation_accuracy", "accuracy_score", y_true=y_val
)

validation_results, executed_graph = pipe(
    X_train, y_train, select_strategy="max"
)
test_results = executed_graph(X_test, y_true=y_test)
```

The training call returns a list of graph outputs and an `ExecutedGraph`. The latter contains the fitted objects from the selected path and accepts new feature data for inference. The available path selection modes are `all`, `max`, and `min`; a user-defined function can be supplied through `custom_strategy`. Parallel graph execution is enabled by default. It can be disabled for deterministic debugging with `pipe.disable_parallel_execution()`, while `pipe.set_multicore_threshold(n)` controls the workload threshold used by the native scheduler.

Column-specific preprocessing can be declared without a custom adapter by passing column names to `preprocess`. Internally, the node wraps the transformer in a Scikit-learn `ColumnTransformer` and passes all remaining columns through unchanged.

Code Example C.2: Applying a transformer to selected columns

```
from sklearn.preprocessing import OneHotEncoder

pipe.preprocess(
    "encode_categories",
    OneHotEncoder(handle_unknown="ignore", sparse_output=False),
    columns=["city", "category"],
)
```

The `cache` argument of preprocessing and model nodes is disabled by default. It should be enabled only when repeated executions reuse the same expensive input and the retained data is worth the additional memory.

C.1.2 Saving training graphs and executed graphs

Both graph forms are saved as one pickle file. Saving a `Pipeline` preserves its graph recipe for later training. Saving an `ExecutedGraph` preserves the fitted selected path for inference.

Code Example C.3: Saving and loading Accelera Pipe graphs

```
from accelera.src.accelera_pipe.core.executed_graph import ExecutedGraph
from accelera.src.accelera_pipe.core.pipeline import Pipeline

pipe.save("training_graph.pkl")
loaded_pipe = Pipeline.load("training_graph.pkl")
_, fitted_graph = loaded_pipe(X_train, y_train, select_strategy="max")

fitted_graph.save("inference_graph.pkl")
loaded_graph = ExecutedGraph.load("inference_graph.pkl")
predictions = loaded_graph(X_test, y_true=y_test)
```

Top-level custom preprocessing functions and simple lambdas are represented by source-backed wrappers when possible. Functions that capture closure variables cannot be reconstructed safely and should be rewritten as top-level functions before the pipeline is saved.

C.2 Using the Code Parallelizer

C.2.1 Parallelizing files and source strings

The native bindings and a compiler with OpenMP support must be available before using the module. Passing a file path writes generated source to the repository root by default; `output_dir` selects another directory. Passing source text instead returns the transformed C++ code directly.

Code Example C.4: Parallelizing a source file and a code string

```
from accelera.src.parallelizer.parallelizer import parallelizer

parallelizer.parallelize(
    "data/hard_test_files/hard_eval_002.py",
    output_dir="generated",
)

source = """
double sum(const double *x, int n) {
    double total = 0.0;
    for (int i = 0; i < n; ++i) total += x[i];
    return total;
}
"""
parallelized_source = parallelizer.parallelize(source)
print(parallelized_source)
```

For Python files and Python source strings, the supported subset is converted to C++ before loop extraction. C and C++ inputs enter loop extraction directly. The output should always be compiled and tested against the original program, because parallel execution is valid only when the loop iterations preserve the same observable result.

C.2.2 Parallelizing a Python callable

Passing a supported custom function returns a callable that can be invoked like the original Python function. The Parallelizer extracts its source and helper functions, converts them to C++, injects OpenMP directives, compiles a PyBind11 module, and caches the compiled artifact.

Code Example C.5: Compiling a custom numerical function

```
import numpy as np

from accelera.src.parallelizer.parallelizer import parallelizer

def square_values(X):
    for i in range(len(X)):
        for j in range(len(X[i])):
            X[i][j] = X[i][j] * X[i][j]
    return X

optimized_square = parallelizer.parallelize(square_values)
result = optimized_square(np.ones((10000, 25), dtype=np.float64))
```

If extraction, conversion, analysis, compilation, or import fails, callable optimization prints a warning and returns the original Python implementation. This fallback protects functional execution, but users should compare outputs and timings to confirm both correctness and a useful speedup.

C.2.3 Using automatic pipeline integration

No direct Parallelizer call is required for custom preprocessing. Accelera Pipe passes custom functions and custom transformer instances through `parallelizer.parallelize()` when the preprocessing node is created.

Code Example C.6: Adding an automatically optimized custom function

```
from accelera.src.accelera_pipe.core.pipeline import Pipeline

pipe = Pipeline()
pipe.preprocess("square_values", square_values)
outputs, executed_graph = pipe(X_train, y_train)
```

For a custom transformer, only a class-defined `transform` method is considered for replacement. Standard Scikit-learn objects and unsupported callables pass through unchanged, preserving their normal behavior.

C.3 Using Tabular Auto Preprocessing

To use tabular preprocessing, the user first loads the dataset as a Pandas DataFrame. Then the user gives the target column, problem type, and output folder. The output folder is important because the module saves the fitted preprocessing objects and the HTML report inside it.

Code Example C.7: Calling tabular training preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.classical_training_preprocessing import (
    ClassicalTrainingPreprocessing,
)

df = pd.read_csv("dataset.csv")

preprocessor = ClassicalTrainingPreprocessing(
    df=df,
    target_col="target",
    problem_type="classification",
    folder_path="outputs/tabular_run",
    val_size=0.2,
    random_state=42,
    is_report=True,
)

X_train, y_train, X_val, y_val = preprocessor.common_preprocessing()
```

For regression, the user only changes the problem type:

Code Example C.8: Changing tabular preprocessing to regression

```
preprocessor = ClassicalTrainingPreprocessing(
    df=df,
    target_col="price",
    problem_type="regression",
    folder_path="outputs/regression_run",
)
```

After training preprocessing, the same folder should be used for test data. This makes the test data use the saved training transformers instead of fitting new ones.

Code Example C.9: Calling tabular testing preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.classical_testing_preprocessing import (
    ClassicalTestingPreprocessing,
)

test_df = pd.read_csv("test.csv")

test_preprocessor = ClassicalTestingPreprocessing(
    df=test_df,
    folder_path="outputs/tabular_run",
)

X_test, y_test = test_preprocessor.common_preprocessing()
```

C.4 Using Text Auto Preprocessing

For text preprocessing, the user provides the text column and the target column. The text column must contain text values, and the target is used for classification.

Code Example C.10: Calling text training preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.text_training_preprocessing import (
    TextTrainingPreprocessing,
)

df = pd.read_csv("reviews.csv")

text_preprocessor = TextTrainingPreprocessing(
    df=df,
    target_col="label",
    text_col="review",
    folder_path="outputs/text_run",
    tfidf_max_features=500,
    is_report=True,
)
```

```
X_train, y_train, X_val, y_val = text_preprocessor.common_preprocessing()
```

For testing text data, the user again passes the same folder path. If the test data does not contain the target column, the class returns the transformed text features and None for the labels.

Code Example C.11: Calling text testing preprocessing

```
from accelera.src.auto_preprocessing.core.text_testing_preprocessing import (
    TextTestingPreprocessing,
)

test_text = pd.read_csv("reviews_test.csv")

text_test_preprocessor = TextTestingPreprocessing(
    df=test_text,
    folder_path="outputs/text_run",
)

X_test, y_test = text_test_preprocessor.common_preprocessing()
```

C.5 Using Image Auto Preprocessing

For image preprocessing, the dataset should be arranged in class folders. For example:

```
Training/
  Cats/
  Dogs/
```

The user passes the training folder and the output folder. If a validation folder already exists, it can be passed also. If not, the module can split the training folder.

Code Example C.12: Calling image training preprocessing

```
from accelera.src.auto_preprocessing.core.classification_image_training_preprocessing
import (
    ClassificationImageTrainingPreprocessing,
)

image_preprocessor = ClassificationImageTrainingPreprocessing(
    training_folder_images="data/PetImages/Training",
    folder_path="outputs/image_run",
    split_training=True,
    val_size=0.2,
    images_size=(224, 224),
    batch_size=16,
    augment=True,
)

train_loader, val_loader = image_preprocessor.common_preprocessing()
```

For image testing, the user provides a list of image paths. If class names are available, they can be passed too. If not, the loader can still be used for prediction.

Code Example C.13: Calling image testing preprocessing

```
from accelera.src.auto_preprocessing.core.classification_image_testing_preprocessing
    import (
        ClassificationImageTestingPreprocessing,
    )

image_paths = ["test/cat1.jpg", "test/dog1.jpg"]
class_names = ["Cats", "Dogs"]

image_test_preprocessor = ClassificationImageTestingPreprocessing(
    image_paths=image_paths,
    image_class_names=class_names,
    folder_path="outputs/image_run",
    batch_size=4,
)

test_loader, invalid_images = image_test_preprocessor.common_preprocessing()
```

C.6 Reading Auto Preprocessing Reports

After running preprocessing with `is_report=True`, the user opens the generated `report.html` file from the output folder. The report shows the dataset overview, missing values, duplicate rows, dropped columns, graphs, preprocessing decisions, selected features, and processed data samples.

C.7 Using Pipeline Reports

To generate a pipeline report, the pipeline graph should be serialized first. Then the user passes the XML graph path, output folder, and metric results to `GraphReport`.

Code Example C.14: Calling the pipeline graph report

```
from accelera.src.accelera_pipe.wrappers.graph_report import GraphReport

report = GraphReport(
    folderpath="outputs/pipeline_report",
    xmlpath="outputs/pipeline_graph.xml",
    results=metric_results,
)

report.execute()
```

The output is a `report.html` file. It contains the graph image, branch tables, selected branch, and metric summary.

C.8 Using the Benchmark Platform

The benchmark platform is used from the web interface. The benchmark creator enters the title, problem type, target column, dataset link, test-set link, target-label link, and evaluation metric. A user submits a prediction file link and repository link. The backend calls the Python scoring script, saves the score, and displays the result in the leaderboard.

The prediction file must contain the target column name expected by the benchmark. Also, the number of rows in the prediction file must match the number of rows in the target-label file.

Chapter D: Code Documentation

Chapter E: Feasibility Study